

FC7xxx MCU User Manual

Rev.0.8

Revision History

Revision	Date	Changes
0.1	2023/07/15	Initial release
0.2	2023/08/15	Updated for MCAL V0.2.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.4	2023/11/22	Updated for MCAL V0.4.0
0.5	2024/01/22	Updated for MCAL V0.4.0
0.6	2024/03/29	Added support for FC7240
0.7	2025/01/15	Added support for FC7300DQ
0.8	2025/10/10	Added support for FC7300F4MDDT1C

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 MCU Introduction	5
1.1 Requirement Tracing	5
1.2 Supported Derivatives	5
1.3 Hardware Summary	5
Chapter 2 Software Design	6
2.1 Rejected Requirements	6
2.2 File Structure	7
2.3 Macros	7
2.3.1 Macros in Mcu.h	7
2.3.2 Macros in Mcu_Hw.h	8
2.3.3 Macros in Mcu_version.h	8
2.3.4 Macros in MCU_Cfg.h	9
2.3.5 Macros in Mcu_Hw_Types.h	14
2.4 Enums	14
2.4.1 Enumerations in Mcu_Hw_Types.h	14
2.4.2 Enumerations in Mcu_Cfg.h	14
2.5 Typedefs	15
2.5.1 Typedefs in Mcu_Cfg.h	15
2.6 Structures	16
2.6.1 Mcu_ConfigType	16
2.6.2 Mcu_DemConfigType	16
2.6.3 Mcu_ClockType	17
2.6.4 Mcu_ClockConfigTypes	17
2.6.5 Mcu_ModuleConfigType	18
2.6.6 Mcu_RamSectionType	18
2.6.7 Mcu_RamConfigType	18
2.6.8 Mcu_ModeType	18
2.6.9 Mcu_ModeConfigType	18
2.6.10 Mcu_RGM_ConfigType	19
2.6.11 Mcu_DeinitClockType	19
2.7 API Functions	19
2.7.1 Functions in Mcu.h	19
2.8 Hardware Functions	33
2.8.1 Functions in Mcu_Hw.h	33
2.8.2 Peripheral Functions	45
2.8.3 Interrupt Functions	70
Chapter 3 Tresos Configuration Items	73

3.1	Container Inclusion Relation	73
3.2	Containers and Variables.....	73
3.2.1	McuGeneralConfiguration	74
3.2.2	McuPublishedInformation.....	77
3.2.3	McuEcucPartitionRef	77
3.2.4	McuResetReasonConf	78
3.2.5	McuModuleConfiguration	78
3.2.6	CommonPublishedInformation	164
Chapter 4 Configuration Guides.....		168
4.1	MCU Usage Common Steps.....	168

Chapter 1 MCU Introduction

1.1 Requirement Tracing

The design of this module follows the specifications of the MCU driver specified in AUTOSAR Classic Platform Release 4.6.0. For detailed requirements, refer to the *AUTOSAR_SWS_MCUDriver*.

1.2 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of FLAGCHIP Semiconductors:

- FC7300 series including FC7300F8MDQ, FC7300F8MDT, FC7300F4MDD and FC7300F8MDS with different packages.
- FC7240F2MDS with different packages.

1.3 Hardware Summary

The MCU driver provides MCU services for Clock and RAM initialization. In the MCU configuration set, the MCU specific settings for the Clock (i.e. PLL setting) and RAM (i.e. section base address and size) shall be configured.

MCU shall provide a service to enable and set the MCU clock. (i.e. Cpu clock, Peripheral Clock, Prescalers, Multipliers have to be configured in the MCU), For Flagchip MCU, MCU may configure the whole clock tree confirmation options and selections for peripheral usage. SCG, PCC, CMU, CSC, SCM would be involved during this procedure.

The MCU module shall provide a service to provide MCU reset and get the reset reason. RGM and CM7 core reset function integrated in MCU module.

The MCU module shall provide a service to activate MCU's reduced power modes. So SMC WKU and PMC functions are integrated in the MCU module.

Chapter 2 Software Design

2.1 Rejected Requirements

Rejected Requirement 1 **SWS_Mcu_00215**

Description	The type definitions for Mcu_Lcfg.c and Mcu_PBcfg.c are located in the file Mcu.h
Rejection Reason	Driver design doesn't include Link-time configuration support.

Rejected Requirement 2 **SWS_Mcu_00216**

Description	Mcu_Lcfg.c shall include Mcu_Cbk.h for a link time configuration if the call back function is linked to the module via the ROM structure
Rejection Reason	Driver design doesn't include Link-time configuration support.

Rejected Requirement 3 **SWS_Mcu_00218**

Description	Mcu_PBcfg.c shall include Mcu_Cbk.h for post build time configuration if the call back function is linked to the module via the ROM structure.
Rejection Reason	Driver design doesn't include Mcu_Cbk.h, the declaration for the callback function is in the Mcu_Cfg.h.

Rejected Requirement 4 **SWS_Mcu_00138**

Description	The function Mcu_InitClock shall start the PLL lock procedure (if PLL shall be initialized) and shall return without waiting until the PLL is locked.(BSW12208)
Rejection Reason	MCU clock initialize function combined MCU core clock and bus clock selection, thus we should wait all source clock and PLL stable before it's ready to be selected.

Rejected Requirement 5 **SWS_Mcu_00140**

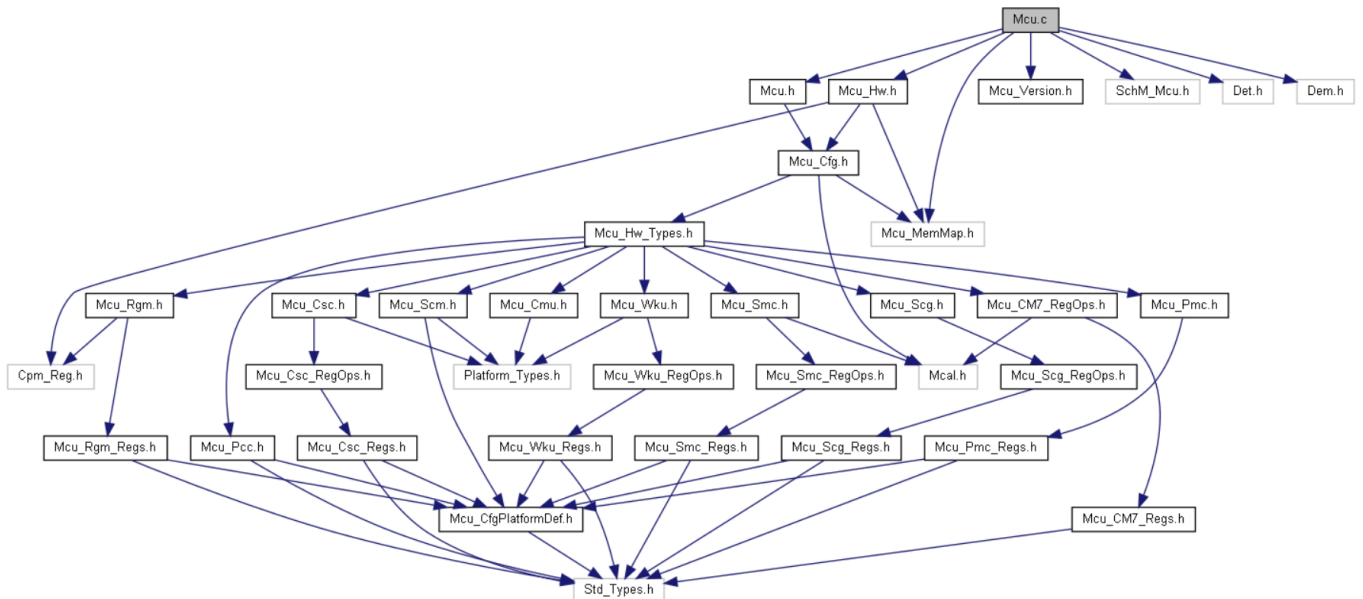
Description	The function Mcu_DistributePllClock shall activate the PLL clock to the MCU clock distribution.
Rejection Reason	Driver design doesn't do anything in Mcu_DistributePllClock function, clock configuration and distribute had done in Mcu_InitClock function, because the clock need to wait stable and then the peripheral clock can be configured

Rejected Requirement 6 **ECUC_Mcu_00190**

Description	SWS Item	ECUC_Mcu_00190		
	Name	McuRamSectionWriteSize		
	Description	This parameter shall define the size in bytes of data which can be written into RAM at once.		
	Multiplicity	1		
	Type	EcucIntegerParamDef		
	Range	0 .. 4294967295		
	Default value	–		
	Post-Build Variant Value	true		
	Value Configuration Class	Pre-compile time	X	VARIANT-PRE-COMPILE
		Link time	–	

		Post-build time	X	VARIANT-POST-BUILD
	Scope / Dependency	scope: local		
Rejection Reason	RAM Section API opened, but suggest user initialize ram sections at startup stage before MCU initialize function called.			

2.2 File Structure



2.3 Macros

2.3.1 Macros in Mcu.h

- `#define MCU_E_PARAM_CONFIG ((uint8)0x0AU)`
- `#define MCU_E_PARAM_CLOCK ((uint8)0x0BU)`
- `#define MCU_E_PARAM_MODE ((uint8)0x0CU)`
- `#define MCU_E_PARAM_RAMSECTION ((uint8)0x0DU)`
- `#define MCU_E_PLL_NOT_LOCKED ((uint8)0x0EU)`
- `#define MCU_E_UNINIT ((uint8)0x0FU)`
- `#define MCU_E_PARAM_POINTER ((uint8)0x10U)`
- `#define MCU_E_INIT_FAILED ((uint8)0x11U)`
- `#define MCU_E_ALREADY_INITIALIZED ((uint8)0x12U)`
- `#define MCU_E_CORE_MISMATCH ((uint8)0x20U)`
- `#define MCU_INSTANCE_ID ((uint8)0x0U)`

- `#define MCU_INIT_ID ((uint8)0x00U)`
- `#define MCU_INITRAMSECTION_ID ((uint8)0x01U)`
- `#define MCU_INITCLOCK_ID ((uint8)0x02U)`
- `#define MCU_DISTRIBUTEPLLCLOCK_ID ((uint8)0x03U)`
- `#define MCU_GETPLLSTATUS_ID ((uint8)0x04U)`
- `#define MCU_GETRESETREASON_ID ((uint8)0x05U)`
- `#define MCU_GETRESETRAWVALUE_ID ((uint8)0x06U)`
- `#define MCU_PERFORMRESET_ID ((uint8)0x07U)`
- `#define MCU_SETMODE_ID ((uint8)0x08U)`
- `#define MCU_GETVERSIONINFO_ID ((uint8)0x09U)`
- `#define MCU_GETRAMSTATE_ID ((uint8)0x0AU)`
- `#define MCU_ENABLECMU_ID ((uint8)0x0BU)`
- `#define MCU_DISABLECMU_ID ((uint8)0x0CU)`
- `#define MCU_INITIALIZED ((boolean)0x01U)`
- `#define MCU_UNINITIALIZED ((boolean)0x00U)`

2.3.2 Macros in `Mcu_Hw.h`

N/A

2.3.3 Macros in `Mcu_version.h`

- `#define MCU_AR_RELEASE_MAJOR_VERSION 4`
- `#define MCU_AR_RELEASE_MINOR_VERSION 6`
- `#define MCU_AR_RELEASE_REVISION_VERSION 0`
- `#define MCU_SW_MAJOR_VERSION 1`

- `#define MCU_SW_MINOR_VERSION 0`
- `#define MCU_SW_PATCH_VERSION 3`
- `#define MCU_VENDOR_ID 174`
- `#define MCU_MODULE_ID 101`

2.3.4 Macros in MCU_Cfg.h

- `#define MCU_CFG_VENDOR_ID 174U`
- `#define MCU_CFG_AR_RELEASE_MAJOR_VERSION 4U`
- `#define MCU_CFG_AR_RELEASE_MINOR_VERSION 6U`
- `#define MCU_CFG_AR_RELEASE_REVISION_VERSION 0U`
- `#define MCU_CFG_SW_MAJOR_VERSION 1U`
- `#define MCU_CFG_SW_MINOR_VERSION 0U`
- `#define MCU_CFG_SW_PATCH_VERSION 3U`
- `#define MCU_DEV_ERROR_DETECT (STD_OFF)`
- `#define MCU_MULTICORE_ERROR_DETECT (STD_ON)`
- `#define MCU_GET_RAM_STATE_API (STD_OFF)`
- `#define MCU_INIT_CLOCK (STD_ON)`
- `#define MCU_NO_PLL (STD_OFF)`
- `#define MCU_PERFORM_RESET_API (STD_ON)`
- `#define MCU_VERSION_INFO_API (STD_OFF)`
- `#define MCU_ENABLE_RGM (STD_ON)`
- `#define MCU_ENABLE_SMC (STD_ON)`

- #define **MCU_ENABLE_WKU** (STD_ON)
- #define **MCU_ENABLE_CMU** (STD_ON)
- #define **MCU_RESET_CALLOUT_USED** (STD_OFF)
- #define **MCU_RESET_CALLOUT** (ResetCallout_Func)
- #define **MCU_PMC_ISR_NOTIFICATION_USED** (STD_OFF)
- #define **MCU_PMC_VOLTAGE_ERROR_NOTIFICATION** Pmc_Notification
- #define **MCU_RGM_ISR_NOTIFICATION_USED** (STD_OFF)
- #define **MCU_RGM_DELAY_NOTIFICATION** (Rgm_Notification)
- #define **MCU_DISABLE_DEM_REPORT_ERROR_STATUS** (STD_ON)
- #define **MCU_MAX_MODECONFIGS** ((uint32)7U)
- #define **MCU_MAX_RAMCONFIGS** ((uint32)2U)
- #define **MCU_MAX_NORAMCONFIGS**
- #define **MCU_PRECOMPILE_SUPPORT** (STD_OFF)
- #define **MCU_ENABLE_USER_MODE_SUPPORT** (STD_OFF)
- #define **MCU_RAW_RESET_DEFAULT** ((uint32)0xFFFFFFFFU)
- #define **MCU_RAW_RESET_SYSRST_TOUT** ((uint32)0x80000000U)
- #define **MCU_RAW_RESET_PINRST_TOUT** ((uint32)0x40000000U)
- #define **MCU_RAW_RESET_FSM_ERR** ((uint32)0x20000000U)
- #define **MCU_RAW_RESET_LBIST** ((uint32)0x00008000U)
- #define **MCU_RAW_RESET_CMU** ((uint32)0x00004000U)
- #define **MCU_RAW_RESET_SACKERR** ((uint32)0x00002000U)

- `#define MCU_RAW_RESET_SYSAP ((uint32)0x00000800U)`
- `#define MCU_RAW_WATCHDOG1 ((uint32)0x00001000U)`
- `#define MCU_RAW_RESET_JTAG ((uint32)0x00000100U)`
- `#define MCU_RAW_RESET_POR ((uint32)0x00000080U)`
- `#define MCU_RAW_RESET_PIN ((uint32)0x00000040U)`
- `#define MCU_RAW_RESET_HSM_WDOG ((uint32)0x00000020U)`
- `#define MCU_RAW_RESET_FCSMU ((uint32)0x00000010U)`
- `#define MCU_RAW_RESET_CLKERR0 ((uint32)0x00000008U)`
- `#define MCU_RAW_RESET_CLKERR1 ((uint32)0x00000004U)`
- `#define MCU_RAW_RESET_LVD ((uint32)0x00000002U)`
- `#define MCU_RAW_RESET_WAKEUP ((uint32)0x00000001U)`
- `#define McuClockSettingConfig_0 ((Mcu_ClockType)0U)`
- `#define McuConf_McuClockSettingConfig_McuClockSettingConfig_0 ((Mcu_ClockType)0U)`
- `#define McuModeSettingConf_0 ((Mcu_ModeType)0U)`
- `#define McuConf_McuModeSettingConf_McuModeSettingConf_0 ((Mcu_ModeType)0U)`
- `#define McuModeSettingConf_1 ((Mcu_ModeType)1U)`
- `#define McuConf_McuModeSettingConf_McuModeSettingConf_1 ((Mcu_ModeType)1U)`
- `#define McuModeSettingConf_2 ((Mcu_ModeType)2U)`
- `#define McuConf_McuModeSettingConf_McuModeSettingConf_2 ((Mcu_ModeType)2U)`
- `#define McuModeSettingConf_3 ((Mcu_ModeType)3U)`

- `#define McuConf_McuModeSettingConf_McuModeSettingConf_3 ((Mcu_ModeType)3U)`
- `#define McuModeSettingConf_4 ((Mcu_ModeType)4U)`
- `#define McuConf_McuModeSettingConf_McuModeSettingConf_4 ((Mcu_ModeType)4U)`
- `#define McuModeSettingConf_5 ((Mcu_ModeType)5U)`
- `#define McuConf_McuModeSettingConf_McuModeSettingConf_5 ((Mcu_ModeType)5U)`
- `#define McuModeSettingConf_6 ((Mcu_ModeType)6U)`
- `#define McuConf_McuModeSettingConf_McuModeSettingConf_6 ((Mcu_ModeType)6U)`
- `#define McuRamSectorSettingConf_0 ((Mcu_RamSectionType)0U)`
- `#define McuConf_McuRamSectorSettingConf_McuRamSectorSettingConf_0 ((Mcu_RamSectionType)0U)`
- `#define McuRamSectorSettingConf_1 ((Mcu_RamSectionType)1U)`
- `#define McuRamSectorSettingConf_2 ((Mcu_RamSectionType)2U)`
- `#define McuConf_McuRamSectorSettingConf_McuRamSectorSettingConf_2 ((Mcu_RamSectionType)2U)`
- `#define McuConf_McuRamSectorSettingConf_McuRamSectorSettingConf_1 ((Mcu_RamSectionType)1U)`
- `#define McuConf_McuResetReasonConf_MCU_SYSRST_TOUT_RESET ((uint8)31U)`
- `#define McuConf_McuResetReasonConf_MCU_PINRST_TOUT_RESET ((uint8)30U)`
- `#define McuConf_McuResetReasonConf_MCU_FSM_ERR_RESET ((uint8)29U)`
- `#define McuConf_McuResetReasonConf_MCU_LBIST_RESET ((uint8)15U)`
- `#define McuConf_McuResetReasonConf_MCU_CMU_RESET ((uint8)14U)`
- `#define McuConf_McuResetReasonConf_MCU_STOP_ACKNOWLEDGE_ERROR_RESET ((uint8)13U)`
- `#define McuConf_McuResetReasonConf_MCU_WATCHDOG1_RESET ((uint8)12U)`
- `#define McuConf_McuResetReasonConf_MCU_SYSAP_RESET ((uint8)11U)`
- `#define McuConf_McuResetReasonConf_MCU_JTAG_RESET ((uint8)8U)`

- `#define McuConf_McuResetReasonConf_MCU_POWER_ON_RESET ((uint8)7U)`
- `#define McuConf_McuResetReasonConf_MCU_PIN_RESET ((uint8)6U)`
- `#define McuConf_McuResetReasonConf_MCU_HSMDOG_RESET ((uint8)5U)`
- `#define McuConf_McuResetReasonConf_MCU_FCSMU_RESET ((uint8)4U)`
- `#define McuConf_McuResetReasonConf_MCU_CLOCK_ERROR0_RESET ((uint8)3U)`
- `#define McuConf_McuResetReasonConf_MCU_CLOCK_ERROR1_RESET ((uint8)2U)`
- `#define McuConf_McuResetReasonConf_MCU_LOW_VOLTAGE_DETECT_RESET ((uint8)1U)`
- `#define McuConf_McuResetReasonConf_MCU_WAKEUP_RESET ((uint8)0U)`
- `#define McuConf_McuResetReasonConf_MCU_NO_RESET_REASON ((uint8)32U)`
- `#define McuConf_McuResetReasonConf_MCU_MULTIPLE_RESET_REASON ((uint8)33U)`
- `#define McuConf_McuResetReasonConf_MCU_RESET_UNDEFINED ((uint8)34U)`
- `#define McuConf_McuResetReasonConf_MCU_SW_RESET ((uint8)20U)`
- `#define McuConf_McuResetReasonConf_MCU_INTM_RESET ((uint8)19U)`
- `#define McuConf_McuResetReasonConf_MCU_WATCHDOG_RESET ((uint8)18U)`
- `#define McuConf_McuResetReasonConf_MCU_SYSRST_RESET ((uint8)17U)`
- `#define McuConf_McuResetReasonConf_MCU_LOCKUP_RESET ((uint8)16U)`
- `#define McuConf_McuResetReason2Reg_Default 0xFFU`
- `#define McuConf_McuResetReason2Reg_Max 32U`
- `#define McuConf_McuResetReason2Reg_Map {\n`
`McuConf_McuResetReasonConf_MCU_WAKEUP_RESET,\n`
`McuConf_McuResetReasonConf_MCU_LOW_VOLTAGE_DETECT_RESET,\n`
`McuConf_McuResetReasonConf_MCU_CLOCK_ERROR1_RESET,\n`
`McuConf_McuResetReasonConf_MCU_CLOCK_ERROR0_RESET,\n`
`McuConf_McuResetReasonConf_MCU_FCSMU_RESET,\n`
`McuConf_McuResetReasonConf_MCU_HSMDOG_RESET,\n`
`McuConf_McuResetReasonConf_MCU_PIN_RESET,\n`
`McuConf_McuResetReasonConf_MCU_POWER_ON_RESET,\n`
`McuConf_McuResetReasonConf_MCU_JTAG_RESET,\n`

```

McuConf_McuRstRsn2Reg_Default,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuResetReasonConf_MCU_SYSAP_RESET,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuResetReasonConf_MCU_STOP_ACKNOWLEDGE_ERROR_RESET,\
McuConf_McuResetReasonConf_MCU_CMU_RESET,\
McuConf_McuResetReasonConf_MCU_LBIST_RESET,\
McuConf_McuResetReasonConf_MCU_LOCKUP_RESET,\
McuConf_McuResetReasonConf_MCU_SYSRST_RESET,\
McuConf_McuResetReasonConf_MCU_WATCHDOG_RESET,\
McuConf_McuResetReasonConf_MCU_INTM_RESET,\
McuConf_McuResetReasonConf_MCU_SW_RESET,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuRstRsn2Reg_Default,\
McuConf_McuResetReasonConf_MCU_FSM_ERR_RESET,\
McuConf_McuResetReasonConf_MCU_PINRST_TOUT_RESET,\
McuConf_McuResetReasonConf_MCU_SYSRST_TOUT_RESET,\}

```

2.3.5 Macros in Mcu_Hw_Types.h

- #define **Mcu_PowerModeType** **SMC_ModeType**

2.4 Enums

2.4.1 Enumerations in Mcu_Hw_Types.h

2.4.1.1 Mcu_PllStatusType

Enumeration	Mcu_PllStatusType
Description	This structure is used to represent the states of PLL0. This Enumeration type contains the PLL0's possible states.
Diagram	N/A
Values	• MCU_PLL_LOCKED PLL is locked. • MCU_PLL_UNLOCKED PLL is unlocked. • MCU_PLL_STATUS_UNDEFINED PLL is unlocked
Requirements	SWS_MCU_00250, SWS_MCU_00231
Reference by	Mcu_PllStatusType Mcu_GetPllStatus(void)

2.4.2 Enumerations in Mcu_Cfg.h

2.4.2.1 Mcu_RamStateType

Enumeration	Mcu_RamStateType
Description	Ram State of the micro controller. This is the Ram State data type returned by the function Mcu_GetRamState() of the Mcu module.

Diagram	N/A
Values	- MCU_RAM_STATE_INVALID : RAM content is not valid or unknown (default). - MCU_RAM_STATE_VALID : RAM content is valid.
Requirements	SWS_MCU_00256
Reference by	Mcu_RamStateType Mcu_GetRamState(void)

2.4.2.2 Mcu_ResetType

Enumeration	Mcu_ResetType
Description	The type Mcu_ResetType, represents the different reset that a specified MCU can have
Diagram	N/A
Values	- MCU_WAKEUP_RESET: WAKEUP Reset. RGM_SRS[WAKEUP]. - MCU_LOW_VOLTAGE_OR_POR_RESET: Low-Voltage Detect Reset or Power On Reset Reset. RGM_SRS[LVD]. - MCU_LOSS_OF_CLOCK_RESET: Loss-of-Clock Reset. RGM_SRS[LOC]. - MCU_LOSS_OF_LOCK_RESET: Loss-of-Lock Reset. RGM_SRS[LOL]. - MCU_FCSMU_RESET: FCSMU Reset. RGM_SRS[FCSMU]. - MCU_HSMDOG_RESET: HSM WDOG Reset. RGM_SRS[HSM_WDOG]. - MCU_PIN_RESET: External Reset Pin. RGM_SRS[PIN]. - MCU_POWER_ON_RESET: Power-on reset. RGM_SRS[POR]. - MCU_JTAG_RESET: JTAG generated reset. RGM_SRS[JTAG]. - MCU_MDMAP_SYSTEM_RESET: MDM-AP System Reset Request . RGM_SRS[MDM_AP]. - MCU_STOP_ACKNOWLEDGE_ERROR_RESET: Stop Acknowledge Error reset . RGM_SRS[SACKERR]. - MCU_CMU_RESET: CMU Reset . RGM_SRS[CMU]. - MCU_LBIST_RESET: Logic BIST Reset . RGM_SRS[LBIST]. - MCU_LOCKUP_RESET: CPUx LOCKUP Reset . RGM_Cx_SRS[Cx_LOCKUP]. - MCU_SYSRST_RESET: CPUx SYSRST Reset . RGM_Cx_SRS[Cx_SYSRST]. - MCU_WATCHDOG_RESET: Watchdog reset. RGM_Cx_SRS[Cx_WDOG]. - MCU_INTM_RESET: CPUx INTM Reset . RGM_Cx_SRS[Cx_INTM]. - MCU_SW_RESET: CPUx SW Reset . RGM_Cx_SRS[Cx_SWRST]. - MCU_FSM_ERR_RESET: FSM Error Reset . RGM_Cx_SRS[FSM_ERR]. - MCU_PINRST_TOUT_RESET: Pin Reset Request Timeout Reset . RGM_SRS[PINRST_TOUT]. - MCU_SYSRST_TOUT_RESET: System Reset Request Timeout Reset . RGM_SRS[SYSRST_TOUT]. - MCU_NO_RESET_REASON: No reset reason found - MCU_MULTIPLE_RESET_REASON: More than one reset events are logged except "Power on event". - MCU_RESET_UNDEFINED: Undefined reset source. - MCU_RESET_MAX: Default reset source.
Requirements	SWS_MCU_00252, SWS_MCU_00234, SWS_MCU_00134
Reference by	Mcu_ResetType Mcu_GetResetReason(void)

2.5 Typedefs

2.5.1 Typedefs in Mcu_Cfg.h

- typedef uint32 **Mcu_RamSectionType**
Definition(IDs) for a RAM section, configure via the configuration structure.
(Requirements : SWS_MCU_00255, SWS_MCU_00239, SWS_MCU_00240)

- typedef uint16 **Mcu_ModeType**
The Mcu_ModeType specifies the identification (ID) for a MCU mode, configured via configuration structure.
(Requirements : SWS_MCU_00254, SWS_MCU_00237, SWS_MCU_00238)
- typedef uint32 **Mcu_RawResetType**
The type Mcu_RawResetType specifies the reset reason in raw register format, read from a reset status register.
(Requirements : SWS_MCU_00253, SWS_MCU_00235, SWS_MCU_00236)
- typedef uint8 **Mcu_ClockType**
Identification (ID) for clock setting configured via the configuration structure.
(Requirements : SWS_MCU_00251, SWS_MCU_00232, SWS_MCU_00233)

2.6 Structures

2.6.1 Mcu_ConfigType

Structure	Mcu_ConfigType
Description	Mcu_ConfigType_struct, used for Mcu_init API.
Diagram	
Data Fields	• Mcu_DemConfigType* Mcu_pDemConfig • Mcu_ClockType Mcu_u8NoClkConfigs • Const Mcu_ClockConfigType * Mcu_apClockConfig • Const Mcu_ModuleConfigType * Mcu_pModuleConfig • Mcu_RamSectionType Mcu_NoRamConfigs • Const Mcu_RamConfigType(* Mcu_apRamConfig)[MCU_MAX_RAMCONFIGS] • Mcu_ModeType Mcu_NoModeConfigs • Const Mcu_ModeConfigType * Mcu_apModeConfig
Requirements	SWS_MCU_00249, SWS_MCU_00131, SWS_MCU_00054, SWS_MCU_00035, SWS_MCU_00031
Reference by	void Mcu_Init(const Mcu_ConfigType * pConfigPtr)

2.6.2 Mcu_DemConfigType

Structure	Mcu_DemConfigType
-----------	-------------------

Description	DEM error reporting configuration
Diagram	N/A
Data Fields	Mcal_DemErrorType Mcu_E_Clock_FailureCfg
Requirements	SWS_MCU_00051, SWS_MCU_00152, SWS_MCU_00166, ECUC_MCU_00187, ECUC_MCU_00188, FUNC_MCU_0046
Reference by	Mcu_ConfigType (A member of Mcu_ConfigType structure)

2.6.3 Mcu_ClockType

Structure	Mcu_ClockType
Description	Mcu identification (ID) for clock setting configured via the configuration structure
Diagram	N/A
Data Fields	uint8
Requirements	SWS_MCU_00251, SWS_MCU_00232, SWS_MCU_00155
Reference by	Mcu_ConfigType (A member of Mcu_ConfigType structure)

2.6.4 Mcu_ClockConfigTypes

Structure	Mcu_ClockConfigType
Description	Mcu clock configuration structure, including SCG, PCC, clock part of SMISC and CMU.
Diagram	<pre> graph LR Mcu_ClockConfigType --> SCG_ClockType Mcu_ClockConfigType --> PCC_GrpConfigType Mcu_ClockConfigType --> CLOCK_CSC0_ConfigType Mcu_ClockConfigType --> CLOCK_SCM_ConfigType Mcu_ClockConfigType --> CLOCK_CMU_ConfigType SCG_ClockType --> SCG_ClockCtrlType SCG_ClockType --> SCG_Sirc32kType SCG_ClockType --> SCG_PllType SCG_ClockType --> SCG_SoscType SCG_ClockType --> SCG_FoscType SCG_ClockType --> SCG_FircType SCG_ClockType --> SCG_SircType PCC_GrpConfigType --> PCC_CLKCtrlType PCC_GrpConfigType --> PCC_GRP_pConfig CLOCK_CSC0_ConfigType --> CSC0_tClkOutConfig CLOCK_CSC0_ConfigType --> CSC0_tAonClkConfig CLOCK_CSC0_ConfigType --> CSC0_tLPWakeUpPadConfig CLOCK_CSC0_ConfigType --> CSC0_LPWakeUpPadType CLOCK_SCM_ConfigType --> SCM_tEnetClkConfig CLOCK_SCM_ConfigType --> SCM_tHsmClkConfig CLOCK_SCM_ConfigType --> SCM_tTraceClkConfig CLOCK_CMU_ConfigType --> CMU1_Config CLOCK_CMU_ConfigType --> CMU2_Config CLOCK_CMU_ConfigType --> CMU3_Config CLOCK_CMU_ConfigType --> CMU4_Config CLOCK_CMU_ConfigType --> CMU0_Config </pre> The diagram illustrates the hierarchical structure of the Mcu_ClockConfigType structure. It is composed of several key components, each represented by a box, with dashed arrows indicating the relationships and data flow between them. The main structure, Mcu_ClockConfigType, is shown on the right. It branches out to five primary configuration types: SCG_ClockType, PCC_GrpConfigType, CLOCK_CSC0_ConfigType, CLOCK_SCM_ConfigType, and CLOCK_CMU_ConfigType. Each of these further branches into specific configuration structures and types. For example, SCG_ClockType includes SCG_ClockCtrlType, SCG_Sirc32kType, SCG_PllType, SCG_SoscType, SCG_FoscType, SCG_FircType, and SCG_SircType. Similarly, PCC_GrpConfigType includes PCC_CLKCtrlType and PCC_GRP_pConfig. The CLOCK_CSC0_ConfigType includes CSC0_tClkOutConfig, CSC0_tAonClkConfig, CSC0_tLPWakeUpPadConfig, and CSC0_LPWakeUpPadType. The CLOCK_SCM_ConfigType includes SCM_tEnetClkConfig, SCM_tHsmClkConfig, and SCM_tTraceClkConfig. Finally, CLOCK_CMU_ConfigType includes CMU1_Config, CMU2_Config, CMU3_Config, CMU4_Config, and CMU0_Config. The CMU_CfgType is also shown as a separate component related to the CMU configuration.
Data Fields	- Const SCG_ClockType * const SCG_pClockConfig - Const PCC_GrpConfigType * const PCC_GRP_pConfig - Const CLOCK_CSC0_ConfigType *const CLOCK_CSC0_pConfig; - Const CLOCK_SCM_ConfigType *const CLOCK_SCM_pConfig; - Const CLOCK_CMU_ConfigType * const CLOCK_CMU_Config
Requirements	SWS_MCU_00137

Reference by	Mcu_ConfigType (A member of Mcu_ConfigType structure)
---------------------	---

2.6.5 Mcu_ModuleConfigType

Structure	Mcu_ModuleConfigType
Description	AUTOSAR module configuration structure, include RGM, PMC and WKU modules
Diagram	<pre> graph TD Base[Mcu_ModuleConfigType] -.-> RGM_pResetConfig RGM[Mcu_RGM_ConfigType] Base -.-> WKU_pConfig WKU[WkuConfigType] Base -.-> PMC_pConfig PMC[PMC_InterruptType] </pre>
Data Fields	• Const Mcu_RGM_ConfigType * const RGM_pResetConfig • Const PMC_InterruptType * const PMC_pConfig • Const WkuConfigType *const WKU_pConfig;
Requirements	FUNC_MCU_0035, FUNC_MCU_0042, FUNC_MCU_0044
Reference by	Mcu_ConfigType (A member of Mcu_ConfigType structure)

2.6.6 Mcu_RamSectionType

Structure	Mcu_RamSectionType
Description	Definition(IDs) for a RAM section, configure via the configuration structure
Diagram	N/A
Data Fields	uint32
Requirements	SWS_MCU_00255, SWS_MCU_00239, SWS_MCU_00154
Reference by	Mcu_ConfigType (A member of Mcu_ConfigType structure)

2.6.7 Mcu_RamConfigType

Structure	Mcu_RamConfigType
Description	Definition of a RAM section within the configuration structure.
Diagram	N/A
Data Fields	• Mcu_RamSectionType Mcu_RamSectorId • uint8(* Mcu_pu8RamBaseAddr)[1U] • Mcu_RamSizeType Mcu_RamSize • uint8 Mcu_u8RamDefaultValue
Requirements	SWS_MCU_00030
Reference by	Mcu_ConfigType (A member of Mcu_ConfigType structure)

2.6.8 Mcu_ModeType

Structure	Mcu_ModeType
Description	The Mcu_ModeType specifies the identification (ID) for a MCU mode
Diagram	N/A
Data Fields	uint16
Requirements	SWS_MCU_00161, SWS_MCU_00254, SWS_MCU_00237
Reference by	Mcu_ConfigType (A member of Mcu_ConfigType structure)

2.6.9 Mcu_ModeConfigType

Structure	Mcu_ModeConfigType
Description	Definition of a MCU mode section in the configuration structure.
Diagram	N/A
Data Fields	• Mcu_ModeType Mcu_ModeConfigId • Mcu_PowerModeType u32PowerMode

Requirements	SWS_MCU_00249
Reference by	Mcu_ConfigType (A member of Mcu_ConfigType structure)

2.6.10 Mcu_RGM_ConfigType

Structure	Mcu_RGM_ConfigType
Description	Reset control Module data configuration type
Diagram	N/A
Data Fields	- uint8 u8BusClockFilterWidth; - boolean bResetfiltAonLp; - RGM_FilterClkSrc eFilterClkSrc; - uint16 u16SysResetInterruptCfg; - uint16 au16CoreResetInterruptCfg[CORE_NUM]; - RGM_ResetDelayType eDelay;
Requirements	FUNC_MCU_0043, FUNC_MCU_0044
Reference by	Mcu_ModuleConfigType (A member of Mcu_ModuleConfigType structure)

2.6.11 Mcu_DeinitClockType

Structure	Mcu_DeinitClockType
Description	MCU SCG de-initialize clock type
Diagram	N/A
Data Fields	SCG_DeInitClockType tDeInitClockConfig;
Requirements	FUNC_MCU_0011
Reference by	Std_ReturnType Mcu_DeInitClock(const Mcu_DeinitClockType* pDeInitClockPtr)

2.7 API Functions

[SWDESG_MCU_011] The MCU peripheral property names shall match the names in MCU datasheet when possible.

2.7.1 Functions in Mcu.h

2.7.1.1 Mcu_Init

Function	void Mcu_Init (const Mcu_ConfigType *pConfigPtr)
Description	MCU driver initialization function.
Diagram	<pre> graph LR Mcu_Init[Mcu_Init] --> Mcu_Hw_PmcPowerInit[Mcu_Hw_PmcPowerInit] Mcu_Init --> Mcu_Hw_RgmInit[Mcu_Hw_RgmInit] Mcu_Init --> Mcu_Init_CheckParameterValid[Mcu_Init_CheckParameterValid] Mcu_Hw_PmcPowerInit --> Pmc_Init[Pmc_Init] Pmc_Init --> Pmc_HWA_SetLVCSRRegister[PMC_HWA_SetLVCSRRegister] Pmc_Init --> Pmc_HWA_SetPmcConfiguration[PMC_HWA_SetPmcConfiguration] Mcu_Hw_RgmInit --> RGM_EnableInt[RGM_EnableInt] RGM_EnableInt --> RGM_HWA_EnableCore0ResetInterrupt[RGM_HWA_EnableCore0ResetInterrupt] RGM_EnableInt --> RGM_HWA_EnableCore1ResetInterrupt[RGM_HWA_EnableCore1ResetInterrupt] RGM_EnableInt --> RGM_HWA_EnableCore2ResetInterrupt[RGM_HWA_EnableCore2ResetInterrupt] RGM_EnableInt --> RGM_HWA_EnableGlobalResetInterrupt[RGM_HWA_EnableGlobalResetInterrupt] RGM_EnableInt --> RGM_HWA_EnableSysResetInterrupt[RGM_HWA_EnableSysResetInterrupt] RGM_EnableInt --> RGM_HWA_SetResetDelay[RGM_HWA_SetResetDelay] RGM_EnableInt --> RGM_HWA_WriteRstfltReg[RGM_HWA_WriteRstfltReg] RGM_EnableInt --> RGM_EnableResetFilter[RGM_EnableResetFilter] </pre>

Parameters	Parameter	Description
	pConfigPtr	Pointer to Mcu Configuration Value.
Register Accessed	RGM_RSTFLT(rw), RGM_SRIE(rw), PMC_LVSCR(rw), PMC_CONFIG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	Void	
Architecture Requirements	SWR_ARCHI_001, SWR_ARCHI_003, SWR_ARCHI_004, SWR_ARCHI_013, SWR_ARCHI_050, SWR_ARCHI_022, SWR_ARCHI_026, SWR_ARCHI_045, SWR_ARCHI_047, SWR_ARCHI_048, FMR_MCU_0001, SWS_MCU_00153, SWS_MCU_00012, SWS_MCU_00026, SWS_MCU_00116, SWS_MCU_00245, SWS_MCU_00163, SWS_MCU_00126, FUNC_MCU_0002, FUNC_MCU_0005	

[SWDESG_MCU_012] Mcu_Init function check the input parameters, if invalid, report errors.

[SWDESG_MCU_013] Mcu_Init function checks whether input parameter pConfigPtr is not empty, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_014] Mcu_Init function checks whether MCU module initialize success, if not, the function report DET error to DET module with the error code MCU_E_INIT_FAILED.

[SWDESG_MCU_015] Mcu_Init function checks whether MCU module has initialized, if not, the function report DET error to DET module with the error code MCU_E_ALREADY_INITIALIZED.

[SWDESG_MCU_016] Mcu_Init function call Mcu_Init_CheckParameterValid function to check the configuration settings for power, clock, and ram sections visible within the MCU module.

[SWDESG_MCU_017] Mcu_Init function needs to initialize the hardware allows for only one usage of the register.

[SWDESG_MCU_018] Mcu_Init function needs to initialize the registers which affect several hardware modules, and it is not an I/O register.

[SWDESG_MCU_019] Mcu_Init function report DET error by the function Det_ReportError.

[SWDESG_MCU_020] Mcu_Init function always have a pointer as a parameter when the value of macro MCU_PRECOMPILE_SUPPORT is STD_OFF. Instead, a NULL pointer shall be passed to the initialization function.

[SWDESG_MCU_021] Mcu_Init function provide low voltage detection.

[SWDESG_MCU_178] A pointer to such a structure is provided to the MCU initialization routines for configuration.

[SWDESG_MCU_179] The definitions for each RAM section within the structure Mcu_ConfigType shall contain:RAM section base address, Section size, Data pre-setting to be initialized.

2.7.1.2 Mcu_InitRamSection

Function	Std_ReturnType Mcu_InitRamSection (Mcu_RamSectionType RamSection)	
Description	MCU driver initialization of Ram sections.	
Diagram	N/A	
Parameters	Parameter	Description
	RamSection	Mcu_RamSectorId
Register Accessed	N/A	
Returns	Std_ReturnType	
Architecture Requirements	SWR_ARCHI_003, FMR_MCU_0002, SWS_MCU_00154, SWS_MCU_00011, SWS_MCU_00012, SWS_MCU_00136, SWS_MCU_00125, SWS_MCU_00226, SWS_MCU_00017, SWS_MCU_00021	

[SWDESG_MCU_022] Mcu_InitRamSection function initialize the ram section specified by the input parameter.

[SWDESG_MCU_023] Mcu_InitRamSection function checks whether the number of input RamSection parameter is within the range of module configurations, if not, the function report DET error to DET module with the error code MCU_E_PARAM_RAMSECTION.

[SWDESG_MCU_024] Mcu_InitRamSection function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_025] Mcu_InitRamSection function return function called status with the type Std_ReturnType which is different with the DET error type or DEM error type.

[SWDESG_MCU_026] Mcu_InitRamSection function report DET error by the function Det_ReportError. In this case, the function return E_NOT_OK.

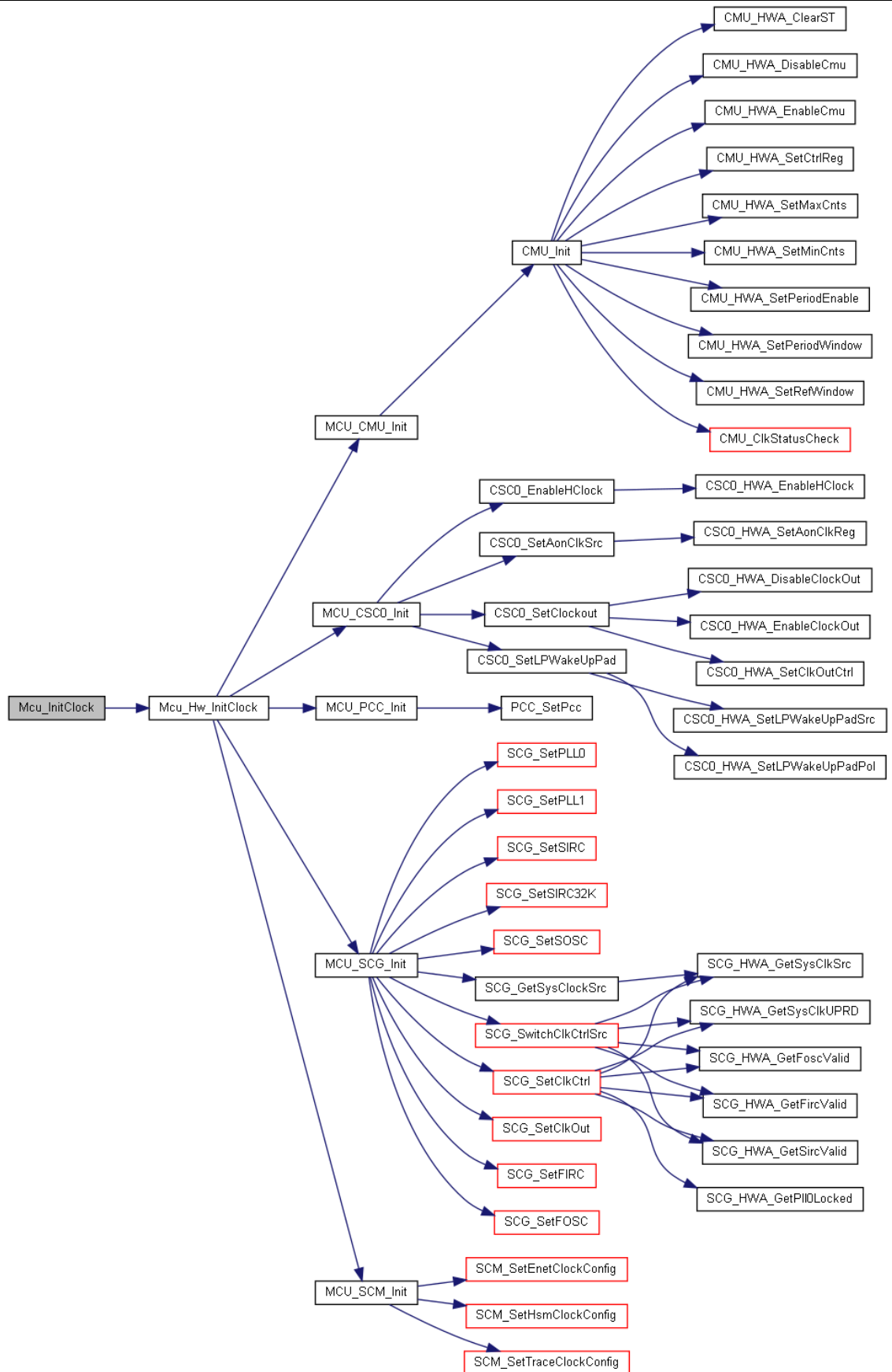
[SWDESG_MCU_027] Mcu_InitRamSection function checks whether the number of input RamSection parameter is within the range of module configurations, if not, the function report DET error to DET module with the error code MCU_E_PARAM_RAMSECTION.

[SWDESG_MCU_184] This type specifies the identification (ID) for a RAM section, which is configured in the configuration structure.

2.7.1.3 Mcu_InitClock

Function	Std_ReturnType Mcu_InitClock (Mcu_ClockType ClockSetting)
Description	This service initializes the PLL and other MCU specific clock options.

Diagram



Parameters

Parameter

Description

ClockSetting

Clock setting ID from config structure to be used.

Register Accessed

SCG_CSR(r), SCG_CCR(rw), SCG_CLKOUTCFG(rw), SCG_FOSCCSR(rw), SCG_FOSCDIV(rw),
 SCG_FOSCCFG(rw), SCG_SIRCCSR(rw), SCG_SIRCDIV(rw), SCG_SIRCTCFG(rw), SCG_SIRCSTAT(rw),
 SCG_FIRCCSR(rw), SCG_FIRCDIV(rw), SCG_FIRCCFG(rw), SCG_FIRCTCFG(rw), SCG_FIRCSTAT(rw),

	SCG_SOSCCSR(rw), SCG_SOSCCFG(rw), SCG_PLL0CSR(rw), SCG_PLL0DIV(rw), SCG_PLL0CFG(rw), SCG_PLL1CSR(rw), SCG_PLL1DIV(rw), SCG_PLL1CFG(rw), SCG_SIRC32KCSR(rw), PCC_XXX(rw)('XXX' represents specific peripherals), CSC0_CCM0_CFG (rw),CSC0_CLKOUT_CTRL (rw),CSC0_AONCLKSR (rw), CMU_CTRL(rw), SCM_DEBUG_TRACE(rw),SCM_ENET(rw),SCM_HSM_PCC(rw)CMU_MIN(rw), CMU_MAX(rw), CMU_REF_WINDOW(rw), CMU_MON_CNT(r), CMU_ST(rw), CMU_PERIOD(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	Std_ReturnType
Architecture Requirements	SWR_ARCHI_001, SWR_ARCHI_003, SWR_ARCHI_004, SWR_ARCHI_033, SWR_ARCHI_051, FMR_MCU_0001, FMR_MCU_0002, FMR_MCU_0003, SWS_MCU_00155, SWS_MCU_00248, SWS_MCU_00137, SWS_MCU_00012, SWS_MCU_00019, SWS_MCU_00125, SWS_MCU_00053, SWS_MCU_00257, SWS_MCU_00258, SWS_MCU_00051, SWS_MCU_00166, SWS_MCU_00226, SWS_MCU_00152, SWS_MCU_00210, SWS_MCU_00017, FUNC_MCU_0030

[SWDESG_MCU_028] Mcu_InitClock provide the service to enable and set the MCU clock, and the clock configuration is specified by the input parameter.

[SWDESG_MCU_029] Mcu_InitClock function checks whether the number of input ClockSetting parameter is within the range of module configurations, if not, the function report DET error to DET module with the error code MCU_E_PARAM_CLOCK.

[SWDESG_MCU_030] Mcu_InitClock function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_031] Mcu_InitClock function checks whether clock source initialize failed, if failed and clock notification is enabled in configuration set, the function report DEM error to DEM module with the error code MCU_E_CLOCK_FAILURE and DEM event status is failed which means a clock source failure occurs.

If clock source initialize success and clock notification is enabled in configuration set, the function will also report DEM error to DEM module with the error code MCU_E_CLOCK_FAILURE and DEM event status is success which means no clock source failure occurs.

[SWDESG_MCU_032] Mcu_InitClock function return function called status with the type Std_ReturnType which is different with the DET error type or DEM error type.

[SWDESG_MCU_033] Mcu_InitClock function use Dem_EventIdType and Dem_EventStatusType to report DEM error to DEM module which is import from Dem module.

[SWDESG_MCU_034] Mcu_InitClock function not reached if the value of macro MCU_INIT_CLOCK is STD_OFF. Instead, the function reached if the value of macro MCU_INIT_CLOCK is STD_ON.

[SWDESG_MCU_035] Mcu_InitClock function report DET error by the function Det_ReportError. In this case, the function return E_NOT_OK.

[SWDESG_MCU_036] PCC should be configured before peripheral actual initialize. PCC setting is in Mcu_InitClock function, Mcu_Init called before Mcu_InitClock. So mcu initialize should not contain PCC setting related peripherals. Such as WKU, CMU.

[SWDESG_MCU_181] A pointer to such a structure specifies the identification (ID) for a clock setting, which is configured in the configuration structure.

[SWDESG_MCU_207] PCC should be configured before peripheral actual initialize. PCC setting is in Mcu_InitClock function, Mcu_Init is called before Mcu_InitClock. It should not contain PCC setting related peripherals.

2.7.1.4 Mcu_DistributePllClock

Function	Std_ReturnType Mcu_DistributePllClock (void)
Description	This function activates the PLL clock to the MCU clock distribution.

Diagram	<pre> graph LR A[Mcu_DistributePllClock] --> B[Mcu_GetPllStatus] B --> C[Mcu_Hw_ScgGetPllStatus] C --> D[SCG_GetClockStatus] D --> E[SCG_HWA_GetFircValid] D --> F[SCG_HWA_GetFoscDivLValid] D --> G[SCG_HWA_GetFoscValid] D --> H[SCG_HWA_GetPll0Locked] D --> I[SCG_HWA_GetPll1Locked] D --> J[SCG_HWA_GetSirc32kValid] D --> K[SCG_HWA_GetSoscValid] </pre>
Parameters	N/A
Register Accessed	SCG_PLL0CSR(rw), SCG_PLL0DIV(rw), SCG_PLL0CFG(rw), SCG_PLL1CSR(rw), SCG_PLL1DIV(rw), SCG_PLL1CFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	Std_ReturnType
Architecture Requirements	SWR_ARCHI_003, SWR_ARCHI_004, SWR_ARCHI_047, FMR_MCU_0001, FMR_MCU_0002, SWS_MCU_00156, SWS_MCU_00056, SWS_MCU_00142, SWS_MCU_00012, SWS_MCU_00122, SWS_MCU_00125, SWS_MCU_00226, SWS_MCU_00205, SWS_MCU_00163, SWS_MCU_00017

[SWDESG_MCU_037] Mcu_DistributePllClock function activate the PLL clock to the MCU clock distribution.

[SWDESG_MCU_038] Mcu_DistributePllClock function check the PLL clock is valid or not without affecting the MCU hardware.

[SWDESG_MCU_039] Mcu_DistributePllClock function checks whether PLL is locked, if not, the function report DET error to DET module with the error code MCU_E_PLL_NOT_LOCKED.

[SWDESG_MCU_040] Mcu_DistributePllClock function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_041] Mcu_DistributePllClock function return function called status with the type Std_ReturnType which is different with the DET error type or DEM error type.

[SWDESG_MCU_042] Mcu_DistributePllClock function not reached if the value of macro MCU_NO_PLL is STD_ON. Instead, the function reached if the value of macro MCU_NO_PLL is STD_OFF.

[SWDESG_MCU_043] Mcu_DistributePllClock function report DET error by the function Det_ReportError. In this case, the function return E_NOT_OK.

2.7.1.5 Mcu_GetPllStatus

Function	Mcu_PllStatusType Mcu_GetPllStatus (void)
Description	This service provides the lock status of the PLL.

Diagram	<pre> graph LR A[Mcu_GetPllStatus] --> B[Mcu_Hw_ScgGetPllStatus] B --> C[SCG_GetClockStatus] C --> D[SCG_HWA_GetFircValid] C --> E[SCG_HWA_GetFoscDivLValid] C --> F[SCG_HWA_GetFoscValid] C --> G[SCG_HWA_GetPll0Locked] C --> H[SCG_HWA_GetPll1Locked] C --> I[SCG_HWA_GetSirc32kValid] C --> J[SCG_HWA_GetSoscValid] </pre>
Parameters	N/A
Register Accessed	SCG_PLL0CSR(rw), SCG_PLL1CSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	Mcu_PllStatusType PLL Status
Architecture Requirements	SWR_ARCHI_003, FMR_MCU_0002, SWS_MCU_00157, SWS_MCU_00008, SWS_MCU_00012, SWS_MCU_00132], SWS_MCU_00125, SWS_MCU_00230, SWS_MCU_00206, SWS_MCU_00163

[SWDESG_MCU_044] Mcu_GetPllStatus function get PLL clock status.

[SWDESG_MCU_045] Mcu_GetPllStatus function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT and return MCU_PLL_STATUS_UNDEFINED.

[SWDESG_MCU_046] Mcu_GetPllStatus function return PLL status with type Mcu_PllStatusType.

[SWDESG_MCU_047] Mcu_GetPllStatus function return MCU_PLL_STATUS_UNDEFINED if the value of macro MCU_NO_PLL is STD_ON.

[SWDESG_MCU_048] Mcu_GetPllStatus function report DET error by the function Det_ReportError.

[SWDESG_MCU_180] The type of Mcu_PllStatusType is an enumeration with the following values: MCU_PLL_LOCKED, MCU_PLL_UNLOCKED, MCU_PLL_STATUS_UNDEFINED.

2.7.1.6 Mcu_GetResetReason

Function	Mcu_ResetType Mcu_GetResetReason (void)
Description	This function returns the Reset reason.
Diagram	<pre> graph LR A[Mcu_GetResetReason] --> B[Mcu_Hw_RgmGetResetReason] B --> C[RGM_GetLastResetFlag] C --> D[RGM_HWA_ReadCore0SRS] C --> E[RGM_HWA_ReadCore1SRS] C --> F[RGM_HWA_ReadCore2SRS] C --> G[RGM_HWA_ReadSysSRS] </pre>
Parameters	N/A
Register Accessed	RGM_SRS(r), RGM_CO_SSRS(rw), RGM_C1_SSRS(rw), RGM_C2_SSRS(rw)

	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	AUTOSAR reset reason
Architecture Requirements	SWR_ARCHI_001, SWS_MCU_00158, SWS_MCU_00052, SWS_MCU_00005, SWS_MCU_00012], SWS_MCU_00125, FUNC_MCU_0013

[SWDESG_MCU_049] Mcu_GetResetReason function get MCU raw reset reason and revert to the reset reason defined by the configuration.

[SWDESG_MCU_050] Mcu_GetResetReason function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT and return MCU_RESET_UNDEFINED.

[SWDESG_MCU_051] Mcu_GetResetReason function return the same value if called multiple times.

[SWDESG_MCU_182] This is the type of the reset enumerator containing the subset of reset types. It is not required that all reset types are supported by hardware.

2.7.1.7 Mcu_GetResetRawValue

Function	Mcu_RawResetType Mcu_GetResetRawValue (void)
Description	This function returns the Raw Reset value.
Diagram	<pre> graph LR A[Mcu_GetResetRawValue] --> B[Mcu_Hw_RgmGetResetRawValue] B --> C[RGM_GetLastResetFlag] C --> D[RGM_HWA_ReadCore0SRS] C --> E[RGM_HWA_ReadCore1SRS] C --> F[RGM_HWA_ReadCore2SRS] C --> G[RGM_HWA_ReadSysSRS] </pre>
Parameters	N/A
Register Accessed	RGM_SRS(r), RGM_CO_SSRS(rw), RGM_C1_SSRS(rw), RGM_C2_SSRS(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	Raw reset reason
Architecture Requirements	SWR_ARCHI_001, SWR_ARCHI_047, SWS_MCU_00159, SWS_MCU_00012, SWS_MCU_00135, SWS_MCU_00125, SWS_MCU_00163, FUNC_MCU_0013

[SWDESG_MCU_052] Mcu_GetResetRawValue function return the MCU raw reset value.

[SWDESG_MCU_053] Mcu_GetResetRawValue function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT. If yes, the function returns the valid value and not equal to 0 which read from register.

[SWDESG_MCU_054] Mcu_GetResetRawValue function report DET error by the function Det_ReportError.

[SWDESG_MCU_055] Mcu_GetResetRawValue function return the same value if called multiple times.

2.7.1.8 Mcu_PerformReset

Function	void Mcu_PerformReset (void)
Description	Generate software reset.
Diagram	<pre> graph LR A[Mcu_PerformReset] --> B[Mcu_Hw_Reset] B --> C[CM7_GenerateSystemReset] C --> D[CM7_HWA_SystemReset] </pre>
Parameters	N/A

Register Accessed	SCB-> AIRCR (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Architecture Requirements	SWR_ARCHI_001, SWR_ARCHI_050, FMR_MCU_0001, SWS_MCU_00160, SWS_MCU_00055, SWS_MCU_00143, SWS_MCU_00144, SWS_MCU_00012, SWS_MCU_00145, SWS_MCU_00125, SWS_MCU_00146, SWS_MCU_00163, FUNC_MCU_0001

[SWS_MCU_00052] Mcu_PerformReset function generate MCU software reset, and the reset type is corresponding to the value which configured in the configuration set.

[SWDESG_MCU_057] Mcu_PerformReset function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_058] Mcu_PerformReset function not reached if the value of macro MCU_PERFORM_RESET_API is STD_OFF. Instead, the function reached if the value of macro MCU_PERFORM_RESET_API is STD_ON.

[SWDESG_MCU_059] Mcu_PerformReset function report DET error by the function Det_ReportError.

[SWDESG_MCU_060] Mcu_PerformReset function provide a vender specific, configurable callback. If this callback function is active, the callback function will be called.

2.7.1.9 Mcu_SetMode

Function	void Mcu_SetMode (Mcu_ModeType McuMode)
Description	This function sets the MCU power mode.
Diagram	<pre> graph LR Mcu_SetMode --> Mcu_Hw_CmuDisableNoLp Mcu_SetMode --> Mcu_Hw_CmuDisableNoStop Mcu_SetMode --> Mcu_Hw_WkuEnable Mcu_SetMode --> Mcu_Hw_SmcModeConfig Mcu_Hw_CmuDisableNoLp --> MCU_CMU_Disable MCU_CMU_Disable --> CMU_Disable MCU_CMU_Disable --> CMU_DisableInterrupt CMU_Disable --> CMU_HWA_DisableCmu CMU_DisableInterrupt --> CMU_HWA_DisableInt Mcu_Hw_CmuDisableNoStop --> PCC_GetPccDwp PCC_GetPccDwp --> WKU_HWA_EnableDelayCounter WKU_HWA_EnableDelayCounter --> WKU_HWA_SetDelayTime WKU_HWA_SetDelayTime --> WKU_HWA_DisableDelayCounter WKU_HWA_DisableDelayCounter --> WKU_HWA_SetWakeupSource WKU_HWA_SetWakeupSource --> SMC_HWA_SetStandbyMode SMC_HWA_SetStandbyMode --> SMC_HWA_SetStopModeCtrl SMC_HWA_SetStopModeCtrl --> CM7_HWA_DisableDeepSleep CM7_HWA_DisableDeepSleep --> CM7_HWA_EnableDeepSleep Mcu_Hw_WkuEnable --> WKU_SetWakeupDelay WKU_SetWakeupDelay --> WKU_HWA_SetDelayTime WKU_HWA_SetDelayTime --> WKU_HWA_DisableDelayCounter WKU_HWA_DisableDelayCounter --> WKU_HWA_SetWakeupSource WKU_HWA_SetWakeupSource --> SMC_HWA_SetStandbyMode SMC_HWA_SetStandbyMode --> SMC_HWA_SetStopModeCtrl SMC_HWA_SetStopModeCtrl --> CM7_HWA_DisableDeepSleep CM7_HWA_DisableDeepSleep --> CM7_HWA_EnableDeepSleep Mcu_Hw_SmcModeConfig --> SMC_SetSystemMode SMC_SetSystemMode --> SMC_HWA_SetStandbyMode SMC_SetSystemMode --> SMC_HWA_SetStopModeCtrl SMC_SetSystemMode --> CM7_HWA_DisableDeepSleep SMC_SetSystemMode --> CM7_HWA_EnableDeepSleep </pre>
Parameters	N/A
Register Accessed	WKU_MWER0(rw), WKU_MWER1(rw), WKU_MWER_2(rw), CMU_CTRL(rw), CMU_ST(rw), SMC_PMCTRL(rw), SMC_STANDBY_CFG(rw), SCB->SCR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Architecture Requirements	SWR_ARCHI_001, SWR_ARCHI_002, SWR_ARCHI_044, SWR_ARCHI_046, FMR_MCU_0001, SWS_MCU_00161, SWS_MCU_00164, FUNC_MCU_0004, FUNC_MCU_0003, FUNC_MCU_0045, SWS_MCU_00012, SWS_MCU_00148, SWS_MCU_00125, SWS_MCU_00147, SWS_MCU_00163, FUNC_MCU_0040, SWS_MCU_00020

[SWDESG_MCU_061] Mcu_SetMode function provide a service to activate MCU reduced power modes.

[SWDESG_MCU_062] Mcu_SetMode function initialize WKU peripheral if input parameter is low power mode(STANDBY mode).

[SWDESG_MCU_063] Mcu_SetMode function disable CMU peripheral which does not enable the low power function when the input parameter is low power mode(STOP/STANDBY mode).

[SWDESG_MCU_064] Mcu_SetMode function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_065] Mcu_SetMode function enter low power mode by input parameter of STOP mode, the function returns after it has performed a wake-up.

[SWDESG_MCU_066] Mcu_SetMode function report DET error by the function Det_ReportError.

[SWDESG_MCU_067] The Macro of MCU_ENABLE_SMC defined as STD_OFF, the chip will not enter the excepted state after the function Mcu_SetMode called.

[SWDESG_MCU_068] Mcu_SetMode function checks whether the number of input McuMode parameter is within the range of module configurations, if not, the function report DET error to DET module with the error code MCU_E_PARAM_CLOCK.

[SWDESG_MCU_183] This type specifies the identification (ID) for a MCU mode, which is configured in the configuration structure.

2.7.1.10 Mcu_GetVersionInfo

Function	void Mcu_GetVersionInfo (Std_VersionInfoType *pVersionInfo)
Description	This function returns the Version Information for the MCU module.
Diagram	N/A
Parameters	N/A
Register Accessed	N/A
Returns	This function returns the vendor id, module id, major, minor and patch version
Architecture Requirements	SWR_ARCHI_001, SWS_MCU_00162, SWS_MCU_00012, SWS_MCU_00152, SWS_MCU_00163

[SWDESG_MCU_069] Mcu_GetVersionInfo function return module version value.

[SWDESG_MCU_070] Mcu_GetVersionInfo function checks whether input parameter pVersionInfo is not empty, if not, the function report DET error to DET module with the error code MCU_E_PARAM_POINTER.

[SWDESG_MCU_071] Mcu_GetVersionInfo function use Std_VersionInfoType to return MCU module version information which is import from Std_Types file.

[SWDESG_MCU_072] Mcu_GetVersionInfo function report DET error by the function Det_ReportError.

2.7.1.11 Mcu_GetRamState

Function	Mcu_RamStateType Mcu_GetRamState (void)
Description	Get Ram State of the micro controller.
Diagram	N/A
Parameters	N/A
Register Accessed	N/A
Returns	Mcu_RamStateType: MCU_RAM_STATE_INVALID or MCU_RAM_STATE_VALID
Architecture Requirements	SWR_ARCHI_001, SWS_MCU_00207, SWS_MCU_00125, SWS_MCU_00209, SWS_MCU_00163

[SWDESG_MCU_073] Mcu_GetRamState function return ram status, it always valid.

[SWDESG_MCU_074] Mcu_GetRamState function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_075] Mcu_GetRamState function not reached if the value of macro MCU_GET_RAM_STATE_API is STD_OFF. Instead, the function reached if the value of macro MCU_GET_RAM_STATE_API is STD_ON.

[SWDESG_MCU_076] Mcu_GetRamState function report DET error by the function Det_ReportError.

[SWDESG_MCU_185] This is the Ram State data type returned by the function Mcu_GetRamState of the Mcu module. It is not required that all RAM state types are supported by the hardware.

2.7.1.12 Mcu_EnableCmu

Function	void Mcu_EnableCmu (void)
Description	Enable CMU.
Diagram	<pre> graph LR Mcu_EnableCmu[Mcu_EnableCmu] --> Mcu_Hw_CmuEnable[Mcu_Hw_CmuEnable] Mcu_Hw_CmuEnable --> MCU_CMU_Enable[MCU_CMU_Enable] MCU_CMU_Enable --> CMU_Enable[CMU_Enable] MCU_CMU_Enable --> CMU_EnableInterrupt[CMU_EnableInterrupt] CMU_Enable --> CMU_HWA_EnableCmu[CMU_HWA_EnableCmu] CMU_EnableInterrupt --> CMU_HWA_EnableInt[CMU_HWA_EnableInt] </pre>
Parameters	N/A
Register Accessed	CMU_CTRL(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Architecture Requirements	SWR_ARCHI_001, SWR_ARCHI_003, SWR_ARCHI_004, FMR_MCU_0002, SWS_MCU_00012, FUNC_MCU_0038

[SWDESG_MCU_077] Mcu_EnableCmu function enable CMU peripheral.

[SWDESG_MCU_078] Mcu_EnableCmu function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_079] Mcu_EnableCmu function can access when the macro of MCU_ENABLE_CMU defined as STD_ON.

2.7.1.13 Mcu_DisableCmu

Function	void Mcu_DisableCmu (void)
Description	Disable CMU.
Diagram	<pre> graph LR Mcu_DisableCmu[Mcu_DisableCmu] --> Mcu_Hw_CmuDisable[Mcu_Hw_CmuDisable] Mcu_Hw_CmuDisable --> MCU_CMU_Disable[MCU_CMU_Disable] MCU_CMU_Disable --> CMU_Disable[CMU_Disable] MCU_CMU_Disable --> CMU_DisableInterrupt[CMU_DisableInterrupt] CMU_Disable --> CMU_HWA_DisableCmu[CMU_HWA_DisableCmu] CMU_DisableInterrupt --> CMU_HWA_DisableInt[CMU_HWA_DisableInt] CMU_DisableInterrupt --> CMU_HWA_ClsST[CMU_HWA_ClsST] </pre>
Parameters	N/A
Register Accessed	CMU_CTRL(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Architecture Requirements	SWR_ARCHI_001, SWR_ARCHI_003, SWR_ARCHI_004, FMR_MCU_0002, FUNC_MCU_0010, SWS_MCU_00012, FUNC_MCU_0038

[SWDESG_MCU_080] Mcu_DisableCmu function disable CMU peripheral.

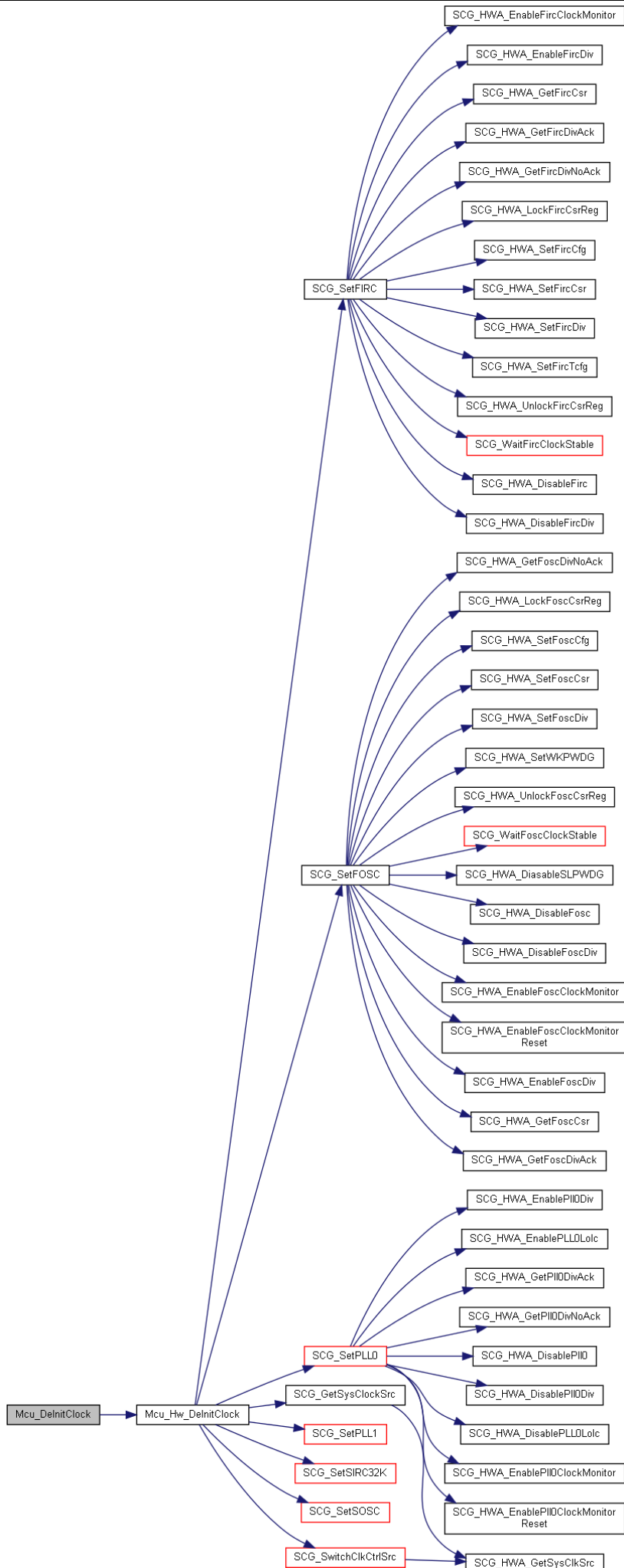
[SWDESG_MCU_081] Mcu_DisableCmu function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

[SWDESG_MCU_082] Mcu_DisableCmu function can access when the macro of MCU_ENABLE_CMU defined as STD_ON.

2.7.1.14 Mcu_DeInitClock

Function	Std_ReturnType Mcu_DeInitClock(const Mcu_DeinitClockType* pDeInitClockPtr)
Description	This service de-initializes the MCU specific clock options.

Diagram



Parameters	Parameter	Description
	pDeInitClockPtr	SCG de-initialize clock config structure to be used.
Register Accessed	SCG_CSR(r), SCG_CCR(rw), SCG_CLKOUTCFG(rw), SCG_FOSCCSR(rw), SCG_FOSCDIV(rw), SCG_FOSCCFG(rw), SCG_SIRCCSR(rw), SCG_SIRCDIV(rw), SCG_SIRCTCFG(rw), SCG_SIRCSTAT(rw), SCG_FIRCCSR(rw), SCG_FIRCDIV(rw), SCG_FIRCCFG(rw), SCG_FIRCTCFG(rw), SCG_FIRCSTAT(rw), SCG_SOSCCSR(rw), SCG_SOSCCFG(rw), SCG_PLL0CSR(rw), SCG_PLL0DIV(rw), SCG_PLL0CFG(rw), SCG_PLL1CSR(rw), SCG_PLL1DIV(rw), SCG_PLL1CFG(rw), SCG_SIRC32KCSR(rw)	
	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	Std_ReturnType	
Architecture Requirements	FUNC_MCU_0011, SWS_MCU_00012	

[SWDESG_MCU_218] Mcu_DeInitClock provide the service to de-initialize the MCU clock, and the clock configuration is specified by the input parameter.

[SWDESG_MCU_219] Mcu_DeInitClock function checks whether the pointer of input Mcu_DeinitClockType parameter not empty, if not, the function report DET error to DET module with the error code MCU_E_PARAM_POINTER.

2.7.1.15 Mcu_DisablePmc

Function	void Mcu_DisablePmc(void)
Description	Disable PMC.
Diagram	<pre> graph LR A[Mcu_DisablePmc] --> B[Mcu_Hw_PmcDisable] B --> C[Pmc_Disable] C --> D[PMC_HWA_SetLVCSRRegister] C --> E[PMC_HWA_SetPmcConfiguration] </pre>
Parameters	N/A
Register Accessed	PMC_LVSCR(rw), PMC_CONFIG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Architecture Requirements	FUNC_MCU_0052

[SWDESG_MCU_220] Mcu_DisablePmc provide the service to disable power manage module.

[SWDESG_MCU_221] Mcu_DisablePmc function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

2.7.1.16 Mcu_DisableRgm

Function	void Mcu_DisableRgm (void)
Description	Disable RGM.

Diagram	<pre> graph LR A[Mcu_DisableRgm] --> B[Mcu_Hw_RgmDisable] B --> C[RGM_Disable] C --> D[RGM_HWA_WriteC0CfgReg] C --> E[RGM_HWA_WriteC1CfgReg] C --> F[RGM_HWA_WriteC2CfgReg] C --> G[RGM_HWA_WriteRstfltReg] C --> H[RGM_HWA_WriteSrieReg] </pre>
Parameters	N/A
Register Accessed	RGM_RSTFLT(rw), RGM_SRIE(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Architecture Requirements	FUNC_MCU_0053

[SWDESG_MCU_222] Mcu_DisableRgm provide the service to disable reset generate module.

[SWDESG_MCU_223] Mcu_DisableRgm function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

2.7.1.17 Mcu_GetSoscValidStatus

Function	Std_ReturnType Mcu_GetSoscValidStatus(void)
Description	Get SOSC clock valid status.
Diagram	N/A
Parameters	N/A
Register Accessed	SCG_SOSCCSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	Std_ReturnType
Architecture Requirements	FUNC_MCU_0054

[SWDESG_MCU_224] Mcu_GetSoscValidStatus provide the service to get SOSC clock valid status

[SWDESG_MCU_225] Mcu_GetSoscValidStatus function checks whether MCU module is initialized, if not, the function report DET error to DET module with the error code MCU_E_UNINIT.

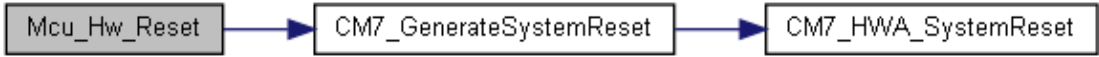
2.8 Hardware Functions

[SWDESG_MCU_011] The MCU peripheral property names shall match the names in MCU datasheet when possible.

2.8.1 Functions in Mcu_Hw.h

2.8.1.1 Mcu_Hw_Reset

Function	void Mcu_Hw_Reset(void)
Description	The function initiates a system reset request to reset the SoC

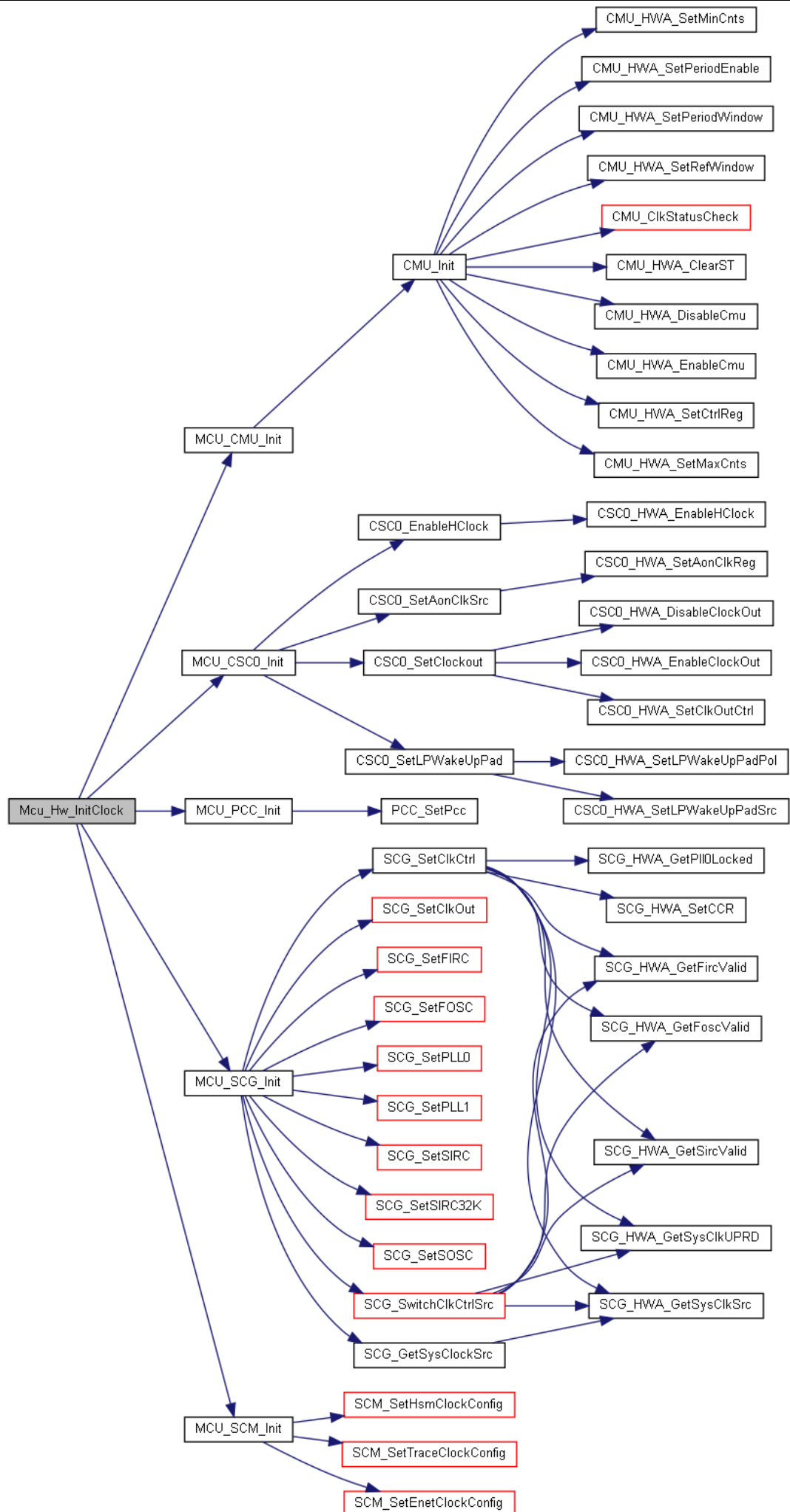
Diagram	 <pre> graph LR A[Mcu_Hw_Reset] --> B[CM7_GenerateSystemReset] B --> C[CM7_HWA_SystemReset] </pre>
Parameters	N/A
Register Accessed	SCB-> AIRCR (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	void Mcu_PerformReset(void)

[SWDESG_MCU_083] Mcu_Hw_Reset function generate software reset to reset MCU.

2.8.1.2 Mcu_Hw_InitClock

Function	Std_ReturnType Mcu_Hw_InitClock (const Mcu_ClockConfigType *const sMcu_pClockConfig)
Description	HW functions for MCU_InitClock, this function would call peripheral clock initialize functions like SCG clock initialization, PCC initialization...

Diagram



Parameters	Parameter	Description
	Mcu_pClockConfig	MCU module configuration, contains SCG,PCC,SMISC,CMU.. configuration
Register Accessed	SCG_CSR(r), SCG_CCR(rw), SCG_CLKOUTCFG(rw), SCG_FOSCCSR(rw), SCG_FOSCDIV(rw), SCG_FOSCCFG(rw), SCG_SIRCCSR(rw), SCG_SIRCDIV(rw), SCG_SIRCTCFG(rw), SCG_SIRCSTAT(rw), SCG_FIRCCSR(rw), SCG_FIRCDIV(rw), SCG_FIRCCFG(rw), SCG_FIRCTCFG(rw), SCG_FIRCSTAT(rw), SCG_SOSCCSR(rw), SCG_SOSCCFG(rw), SCG_PLL0CSR(rw), SCG_PLL0DIV(rw), SCG_PLL0CFG(rw), SCG_PLL1CSR(rw), SCG_PLL1DIV(rw), SCG_PLL1CFG(rw), SCG_SIRC32KCSR(rw), PCC_XXX(rw)('XXX' represents specific peripherals), CSC0_CCM0_CFG (rw),CSC0_CLKOUT_CTRL (rw),CSC0_AONCLKSR (rw), CMU_CTRL(rw), SCM_DEBUG_TRACE(rw),SCM_ENET(rw),SCM_HSM_PCC(rw)CMU_MIN(rw), CMU_MAX(rw), CMU_REF_WINDOW(rw), CMU_MON_CNT(r), CMU_ST(rw), CMU_PERIOD(rw)	
	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	Function status	
Referenced By	Std_ReturnType Mcu_InitClock(Mcu_ClockType ClockSetting)	

[SWDESG_MCU_084] Mcu_Hw_InitClock function initialize SCG peripheral.

[SWDESG_MCU_085] Mcu_InitClock initialize CMU peripheral.

[SWDESG_MCU_086] Mcu_InitClock initialize PCC peripheral.

[SWDESG_MCU_087] Mcu_InitClock initialize clock source which include in CSC0 and SCM peripheral.

2.8.1.3 Mcu_Hw_ScgGetPLLStatus

Function	Mcu_PllStatusType Mcu_Hw_ScgGetPLLStatus (void)
Description	Get status of PLL.
Diagram	<pre> graph LR A[Mcu_Hw_ScgGetPLLStatus] --> B[SCG_GetClockStatus] B --> C[SCG_HWA_GetFircValid] B --> D[SCG_HWA_GetFoscDivLValid] B --> E[SCG_HWA_GetFoscValid] B --> F[SCG_HWA_GetPIIOLocked] B --> G[SCG_HWA_GetPII1Locked] B --> H[SCG_HWA_GetSirc32kValid] B --> I[SCG_HWA_GetSoscValid] </pre>
Parameters	N/A
Register Accessed	SCG_PLL0CSR(rw), SCG_PLL1CSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>

Returns	PLL status
Referenced By	Mcu_PllStatusType Mcu_GetPllStatus(void)

[SWDESG_MCU_088] Mcu_Hw_ScgGetPLLStatus function get PLL status.

2.8.1.4 Mcu_Hw_RgmGetResetReason

Function	Mcu_ResetType Mcu_Hw_RgmGetResetReason (void)
Description	Get reset reason.
Diagram	<pre> graph LR A[Mcu_Hw_RgmGetResetReason] --> B[RGM_GetLastResetFlag] B --> C[RGM_HWA_ReadCore0SRS] B --> D[RGM_HWA_ReadCore1SRS] B --> E[RGM_HWA_ReadCore2SRS] B --> F[RGM_HWA_ReadSysSRS] </pre>
Parameters	N/A
Register Accessed	RGM_SRS(r), RGM_C0_SSRS(rw), RGM_C1_SSRS(rw), RGM_C2_SSRS(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	Reset reason
Referenced By	Mcu_ResetType Mcu_GetResetReason(void)

[SWDESG_MCU_089] Mcu_Hw_RgmGetResetReason function get last raw reset reason and revert to the defined reset reason.

2.8.1.5 Mcu_Hw_RgmGetResetRawValue

Function	Mcu_RawResetType Mcu_Hw_RgmGetResetRawValue (void)
Description	This function returns the Raw Reset value.
Diagram	<pre> graph LR A[Mcu_Hw_RgmGetResetRawValue] --> B[RGM_GetLastResetFlag] B --> C[RGM_HWA_ReadCore0SRS] B --> D[RGM_HWA_ReadCore1SRS] B --> E[RGM_HWA_ReadCore2SRS] B --> F[RGM_HWA_ReadSysSRS] </pre>
Parameters	N/A
Register Accessed	RGM_SRS(r), RGM_C0_SSRS(rw), RGM_C1_SSRS(rw), RGM_C2_SSRS(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	Register value with the Reset status.
Referenced By	Mcu_RawResetType Mcu_GetResetRawValue(void)

[SWDESG_MCU_090] Mcu_Hw_RgmGetResetRawValue function return last raw reset reason.

2.8.1.6 Mcu_Hw_Rgmlnit

Function	Void Mcu_Hw_Rgmlnit (const Mcu_RGM_ConfigType *const pConfigPtr)	
Description	Initialize RGM (reset related functions)	
Diagram	<pre> graph LR Mcu_Hw_Rgmlnit[Mcu_Hw_Rgmlnit] --> RGM_EnableInt[RGM_EnableInt] Mcu_Hw_Rgmlnit --> RGM_EnableResetFilter[RGM_EnableResetFilter] RGM_EnableInt --> RGM_HWA_EnableCore0ResetInterrupt[RGM_HWA_EnableCore0ResetInterrupt] RGM_EnableInt --> RGM_HWA_EnableCore1ResetInterrupt[RGM_HWA_EnableCore1ResetInterrupt] RGM_EnableInt --> RGM_HWA_EnableCore2ResetInterrupt[RGM_HWA_EnableCore2ResetInterrupt] RGM_EnableInt --> RGM_HWA_EnableGlobalResetInterrupt[RGM_HWA_EnableGlobalResetInterrupt] RGM_EnableInt --> RGM_HWA_EnableSysResetInterrupt[RGM_HWA_EnableSysResetInterrupt] RGM_EnableInt --> RGM_HWA_SetResetDelay[RGM_HWA_SetResetDelay] RGM_EnableInt --> RGM_HWA_WriteRstfltReg[RGM_HWA_WriteRstfltReg] RGM_EnableResetFilter --> RGM_HWA_WriteRstfltReg </pre>	
Parameters	Parameter	Description
	pConfigPtr	Initialize RGM (reset related functions)
Register Accessed	RGM_RSTFLT(rw), RGM_SRIE(rw), <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_Init(const Mcu_ConfigType * pConfigPtr)	

[SWDESG_MCU_091] Mcu_Hw_Rgmlnit function initialize RGM peripheral.

[SWDESG_MCU_092] Mcu_Hw_Rgmlnit function can access when the macro of MCU_ENABLE_RGM defined as STD_ON.

2.8.1.7 Mcu_Hw_PmcPowerInit

Function	Void Mcu_Hw_PmcPowerInit (const PMC_InterruptType *const pConfigPtr)	
Description	This function configure the Power Management Controller.	
Diagram	<pre> graph LR Mcu_Hw_PmcPowerInit[Mcu_Hw_PmcPowerInit] --> Pmc_Init[Pmc_Init] Pmc_Init --> PMC_HWA_SetLVCSRRegister[PMC_HWA_SetLVCSRRegister] Pmc_Init --> PMC_HWA_SetPmcConfiguration[PMC_HWA_SetPmcConfiguration] </pre>	
Parameters	Parameter	Description
	pConfigPtr	Pointer to PMC configuration structure.
Register Accessed	PMC_LVSCR(rw), PMC_CONFIG(rw)	

	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	void Mcu_Init(const Mcu_ConfigType * pConfigPtr)

[SWDESG_MCU_093] Mcu_Hw_PmcPowerInit function initialize PMC peripheral.

[SWDESG_MCU_094] Mcu_Hw_PmcPowerInit function can access when the macro of MCU_ENABLE_WKU defined as STD_ON.

2.8.1.8 Mcu_Hw_SmcModeConfig

Function	void Mcu_Hw_SmcModeConfig (const Mcu_ModeConfigType *const pModeConfigPtr)	
Description	This function switches the mode by writing SMC_PMCTRL and SMC_STOPCTRL.	
Diagram	<pre> graph LR A[Mcu_Hw_SmcModeConfig] --> B[SMC_SetSystemMode] B --> C[CM7_HWA_DisableDeepSleep] B --> D[CM7_HWA_EnableDeepSleep] B --> E[SMC_HWA_SetStandbyMode] B --> F[SMC_HWA_SetStopModeCtrl] </pre>	
Parameters	Parameter	Description
	pModeConfigPtr	Pointer to mode configuration structure.
Register Accessed	SMC_PMCTRL(rw), SMC_STANDBY_CFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_SetMode(Mcu_ModeType McuMode)	

[SWDESG_MCU_095] Mcu_Hw_SmcModeConfig function configure MCU power mode and enter the specified power mode.

2.8.1.9 Mcu_Hw_WkuEnable

Function	void Mcu_Hw_WkuEnable (const uint32 u32Input)	
Description	Configure wake-up source.	
Diagram	<pre> graph LR A[Mcu_Hw_WkuEnable] --> B[PCC_GetPccDwp] A --> C[WKU_EnableWakeupSource] A --> D[WKU_SetWakeupDelay] C --> E[WKU_HWA_SetWakeupSource] D --> F[WKU_HWA_DisableDelayCounter] D --> G[WKU_HWA_EnableDelayCounter] D --> H[WKU_HWA_SetDelayTime] </pre>	
Parameters	Parameter	Description
	u32Input	input wake-up source variable

Register Accessed	WKU_MWER0(rw), WKU_MWER1(rw), WKU_MWER2(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	Enable wake-up source status
Referenced By	void Mcu_SetMode(Mcu_ModeType McuMode)

[SWDESG_MCU_096] Mcu_Hw_WkuEnable function enable WKU peripheral.

[SWDESG_MCU_097] Mcu_Hw_WkuEnable function can access when the macro of MCU_ENABLE_WKU defined as STD_ON.

2.8.1.10 Mcu_Hw_CmuEnable

Function	void Mcu_Hw_CmuEnable (const CLOCK_CMU_ConfigType *const CLOCK_CMU_Config)	
Description	Enable CMU.	
Diagram	<pre> graph LR Mcu_Hw_CmuEnable --> MCU_CMU_Enable MCU_CMU_Enable --> CMU_Enable MCU_CMU_Enable --> CMU_EnableInterrupt CMU_Enable --> CMU_HWA_EnableCmu CMU_EnableInterrupt --> CMU_HWA_EnableInt </pre>	
Parameters	Parameter	Description
	CLOCK_CMU_Config	CMU configuration pointer
Register Accessed	CMU_CTRL(rw), CMU_ST(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_EnableCmu(void)	

[SWDESG_MCU_098] Mcu_Hw_CmuEnable function enable CMU peripherals.

[SWDESG_MCU_099] Mcu_Hw_CmuEnable function can access when the macro of MCU_ENABLE_CMU defined as STD_ON.

[SWDESG_MCU_166] Mcu_Hw_CmuEnable function enable single CMU peripheral by calling the function MCU_CMU_ENABLE().

2.8.1.11 Mcu_Hw_CmuDisable

Function	void Mcu_Hw_CmuDisable (const CLOCK_CMU_ConfigType *const CLOCK_CMU_Config)	
Description	Disable CMU.	
Diagram	<pre> graph LR Mcu_Hw_CmuDisableNoLp --> MCU_CMU_Disable MCU_CMU_Disable --> CMU_Disable MCU_CMU_Disable --> CMU_DisableInterrupt MCU_CMU_Disable --> PCC_GetPccDwp CMU_Disable --> CMU_HWA_DisableCmu CMU_DisableInterrupt --> CMU_HWA_ClearST CMU_DisableInterrupt --> CMU_HWA_DisableInt </pre>	
Parameters	Parameter	Description
	CLOCK_CMU_Config	CMU configuration pointer
Register Accessed	CMU_CTRL(rw), CMU_ST(rw)	

	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	void Mcu_DisableCmu(void)

[SWDESG_MCU_100] Mcu_Hw_CmuDisable function disable CMU peripheral.

[SWDESG_MCU_167] Mcu_Hw_CmuDisable function disable single CMU peripheral by calling the function MCU_CMU_DISABLE().

2.8.1.12 Mcu_Hw_CmuDisableNoLp

Function	void Mcu_Hw_CmuDisableNoLp (const CLOCK_CMU_ConfigType *const CLOCK_CMU_Config)	
Description	Disable CMU which not support low power.	
Diagram	<pre> graph LR A[Mcu_Hw_CmuDisableNoLp] --> B[MCU_CMU_Disable] A --> C[PCC_GetPccDwp] B --> D[CMU_Disable] B --> E[CMU_DisableInterrupt] D --> F[CMU_HWA_DisableCmu] E --> G[CMU_HWA_ClearST] E --> H[CMU_HWA_DisableInt] </pre>	
Parameters	Parameter	Description
	CLOCK_CMU_Config	CMU configuration pointer
Register Accessed	CMU_CTRL(rw), CMU_ST(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_SetMode(Mcu_ModeType McuMode)	

[SWDESG_MCU_101] Mcu_Hw_CmuDisableNoLp function disable CMU which is not enable under low power mode(STANDBY mode).

2.8.1.13 Mcu_Hw_CmuDisableNoStop

Function	void Mcu_Hw_Cmu_DisableNoStop (const CLOCK_CMU_ConfigType *const CLOCK_CMU_Config)	
Description	Disable CMU which not support stop mode	
Diagram	<pre> graph LR A[Mcu_Hw_CmuDisableNoStop] --> B[MCU_CMU_Disable] A --> C[PCC_GetPccDwp] B --> D[CMU_Disable] B --> E[CMU_DisableInterrupt] D --> F[CMU_HWA_DisableCmu] E --> G[CMU_HWA_ClearST] E --> H[CMU_HWA_DisableInt] </pre>	
Parameters	Parameter	Description
	CLOCK_CMU_Config	CMU configuration pointer
Register Accessed	CMU_CTRL(rw), CMU_ST(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	

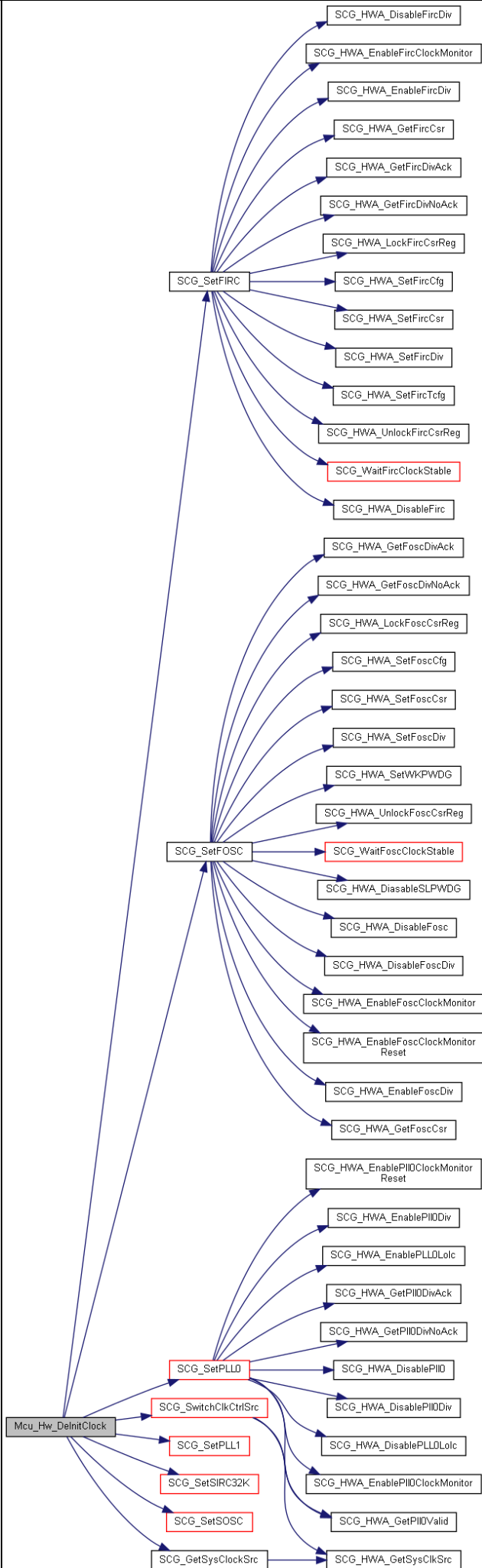
Referenced By	void Mcu_SetMode(Mcu_ModeType McuMode)
----------------------	--

[SWDESG_MCU_102] Mcu_Hw_CmuDisableNoStop function disable CMU which is not enable under low power mode(STOP mode).

2.8.1.14 Mcu_Hw_DeInitClock

Function	Std_ReturnType Mcu_Hw_DeInitClock(const SCG_DeInitClockType* pConfigPtr)
Description	Deinitialize clock in SCG.

Diagram



Parameters	Parameter	Description
	SCG_DeInitClockType	DeInit configuration pointer
Register Accessed	SCG_CSR(r), SCG_CCR(rw), SCG_CLKOUTCFG(rw), SCG_FOSCCSR(rw), SCG_FOSCDIV(rw), SCG_FOSCCFG(rw), SCG_SIRCCSR(rw), SCG_SIRCDIV(rw), SCG_SIRCTCFG(rw), SCG_SIRCSTAT(rw), SCG_FIRCCSR(rw), SCG_FIRCDIV(rw), SCG_FIRCCFG(rw), SCG_FIRCTCFG(rw), SCG_FIRCSTAT(rw), SCG_SOSCCSR(rw), SCG_SOSCCFG(rw), SCG_PLL0CSR(rw), SCG_PLL0DIV(rw), SCG_PLL0CFG(rw), SCG_PLL1CSR(rw), SCG_PLL1DIV(rw), SCG_PLL1CFG(rw), SCG_SIRC32KCSR(rw)	
	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	Std_ReturnType	
Referenced By	Std_ReturnType Mcu_DeInitClock(const Mcu_DeinitClockType* pDeInitClockPtr)	

[SWDESG_MCU_218] SWDESG_MCU_218 cover this item.

2.8.1.15 Mcu_Hw_PmcDisable

Function	void Mcu_Hw_PmcDisable(void)	
Description	Disable power manage module.	
Diagram	<pre> graph LR A[Mcu_Hw_PmcDisable] --> B[Pmc_Disable] B --> C[PMC_HWA_SetLVCSRRegister] B --> D[PMC_HWA_SetPmcConfiguration] </pre>	
Parameters	N/A	
Register Accessed	PMC_LVSCR(rw), PMC_CONFIG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_DisablePmc(void)	

[SWDESG_MCU_220] SWDESG_MCU_220 cover this item.

2.8.1.16 Mcu_Hw_RgmDisable

Function	void Mcu_Hw_RgmDisable(void)	
Description	Disable RGM.	
Diagram	<pre> graph LR A[Mcu_Hw_RgmDisable] --> B[RGM_Disable] B --> C[RGM_HWA_WriteC0CfgReg] B --> D[RGM_HWA_WriteC1CfgReg] B --> E[RGM_HWA_WriteC2CfgReg] B --> F[RGM_HWA_WriteRstfltReg] B --> G[RGM_HWA_WriteSrieReg] </pre>	
Parameters	N/A	

Register Accessed	RGM_RSTFLT(rw), RGM_SRIE(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	void Mcu_DisableRgm(void)

[SWDESG_MCU_222] SWDESG_MCU_222 cover this item.

2.8.2 Peripheral Functions

MCU module combines peripheral functions for AUTOSAR MCU module. These peripherals include Clock Monitor Unit (CMU), System Clock Generator (SCG), Peripheral Clock Control (PCC), Wake-up Unit (WKU), CPU System Control (CSC), System Control Module(SCM), System Mode Controller (SMC), Power Management Controller (PMC), Reset Generation Module (RGM).

[SWDESG_MCU_103] The functions which access to the register filter the value read from the data register by clearing any bit values that do not belong to the actual data bitfield. Additionally, the functions take into account the configured data alignment in the register.

[SWDESG_MCU_104] Interrupt service routines (ISR) not run longer than $t \leq 20\mu s$ or equivalent $t \leq 5200$ CPU cycles (core @ 300 MHz, I-Cache off), with empty callback functions.

2.8.2.1 Functions in Mcu_Scg.h

2.8.2.1.1 SCG_SetSOSC

Function	SCG_StatusType SCG_SetSOSC(const SCG_SoscType *pSoscConfig)	
Description	Set SOSC configuration.	
Diagram	<pre> graph LR SCG_SetSOSC[SCG_SetSOSC] --> SCG_HWA_DisableSosc[SCG_HWA_DisableSosc] SCG_SetSOSC --> SCG_HWA_EnableSoscClockMonitor[SCG_HWA_EnableSoscClockMonitor] SCG_SetSOSC --> SCG_HWA_EnableSoscClockMonitor_Reset[SCG_HWA_EnableSoscClockMonitor Reset] SCG_SetSOSC --> SCG_HWA_GetSoscCsr[SCG_HWA_GetSoscCsr] SCG_SetSOSC --> SCG_HWA_LockSoscCsrReg[SCG_HWA_LockSoscCsrReg] SCG_SetSOSC --> SCG_HWA_SetSoscCfg[SCG_HWA_SetSoscCfg] SCG_SetSOSC --> SCG_HWA_SetSoscCsr[SCG_HWA_SetSoscCsr] SCG_SetSOSC --> SCG_HWA_UnlockSoscCsrReg[SCG_HWA_UnlockSoscCsrReg] SCG_SetSOSC --> SCG_WaitSoscClockStable[SCG_WaitSoscClockStable] SCG_WaitSoscClockStable --> SCG_HWA_GetSoscValid[SCG_HWA_GetSoscValid] </pre>	
Parameters	Parameter	Description
	pSoscConfig	pointer to the SoscType structure variable, which defined SOSC initial information.

Register Accessed	SCG_SOSCCSR(rw), SCG_SOSCCFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	SOSC status. SOSC would wait for SOSC valid in while loop within limited time to check the SOSC valid or not. If SOSC still not valid(this may happen if external Slow OSC not placed), it would return fail
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)

[SWDESG_MCU_105] SCG_SetSOSC function used to initialize SOSC clock.

[SWDESG_MCU_163] SCG_SetSOSC function call SCG_WaitSoscClockStable to wait SOSC stable and SCG_WaitSoscClockStable use while loop with dedicated escape mechanism to avoid infinite loop.

2.8.2.1.2 SCG_SetSIRC

Function	SCG_StatusType SCG_SetSIRC(const SCG_SircType *pSircConfig)	
Description	Set SIRC configuration.	
Diagram	<pre> graph LR SCG_SetSIRC[SCG_SetSIRC] --> SCG_HWA_DiableSircDiv[SCG_HWA_DiableSircDiv] SCG_SetSIRC --> SCG_HWA_EnableSircDiv[SCG_HWA_EnableSircDiv] SCG_SetSIRC --> SCG_HWA_GetSircCsr[SCG_HWA_GetSircCsr] SCG_SetSIRC --> SCG_HWA_GetSircDivAck[SCG_HWA_GetSircDivAck] SCG_SetSIRC --> SCG_HWA_GetSircDivNoAck[SCG_HWA_GetSircDivNoAck] SCG_SetSIRC --> SCG_HWA_LockSircCsrReg[SCG_HWA_LockSircCsrReg] SCG_SetSIRC --> SCG_HWA_SetSircCsr[SCG_HWA_SetSircCsr] SCG_SetSIRC --> SCG_HWA_SetSircDiv[SCG_HWA_SetSircDiv] SCG_SetSIRC --> SCG_HWA_SetSircTcfg[SCG_HWA_SetSircTcfg] SCG_SetSIRC --> SCG_HWA_UnlockSircCsrReg[SCG_HWA_UnlockSircCsrReg] </pre>	
Parameters	Parameter	Description
	pSircConfig	Pointer to the SIRC Type structure data instance, which defined SIRC initial information.
Register Accessed	SCG_SIRCCSR(rw), SCG_SIRCDIV(rw), SCG_SIRCTCFG(rw), SCG_SIRCSTAT(rw)	

	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	SIRC status. SIRC would wait for SIRC valid in while loop within limited time to check the SIRC valid or not. If SOSC still not valid, it would return fail.
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)

[SWDESG_MCU_106] SCG_SetSIRC function used to initialize SIRC clock.

2.8.2.1.3 SCG_SetSIRC32K

Function	SCG_StatusType SCG_SetSIRC32K(const SCG_Sirc32kType *pSirc32kConfig)	
Description	Set SIRC32K configuration.	
Diagram	<pre> graph LR A[SCG_SetSIRC32K] --> B[SCG_HWA_DisableSirc32k] A --> C[SCG_HWA_GetSirc32kValid] A --> D[SCG_HWA_LockSirc32kCsrReg] A --> E[SCG_HWA_SetSirc32kCsr] A --> F[SCG_HWA_UnlockSirc32kCsrReg] </pre>	
Parameters	Parameter	Description
	pSirc32kConfig	Pointer to the Sirc32k Type structure data instance, which defined SIRC32K initial information.
Register Accessed	SCG_SIRC32KCSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	SIRC32K status	
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)	

[SWDESG_MCU_107] SCG_SetSIRC32K used to initialize SIRC32K clock.

2.8.2.1.4 SCG_SetFIRC

Function	SCG_StatusType SCG_SetFIRC(const SCG_FircType *pFircConfig)
Description	Set FIRC configuration.

Diagram	<pre>graph LR; SCG_SetFIRC[SCG_SetFIRC] --> SCG_HWA_DisableFirc[SCG_HWA_DisableFirc]; SCG_SetFIRC --> SCG_HWA_EnableFircClockMonitor[SCG_HWA_EnableFircClockMonitor]; SCG_SetFIRC --> SCG_HWA_EnableFircDiv[SCG_HWA_EnableFircDiv]; SCG_SetFIRC --> SCG_HWA_GetFircCsr[SCG_HWA_GetFircCsr]; SCG_SetFIRC --> SCG_HWA_GetFircDivAck[SCG_HWA_GetFircDivAck]; SCG_SetFIRC --> SCG_HWA_LockFircCsrReg[SCG_HWA_LockFircCsrReg]; SCG_SetFIRC --> SCG_HWA_SetFircCfg[SCG_HWA_SetFircCfg]; SCG_SetFIRC --> SCG_HWA_SetFircCsr[SCG_HWA_SetFircCsr]; SCG_SetFIRC --> SCG_HWA_SetFircDiv[SCG_HWA_SetFircDiv]; SCG_SetFIRC --> SCG_HWA_SetFircTcfg[SCG_HWA_SetFircTcfg]; SCG_SetFIRC --> SCG_HWA_UnlockFircCsrReg[SCG_HWA_UnlockFircCsrReg]; SCG_SetFIRC --> SCG_WaitFircClockStable[SCG_WaitFircClockStable]; SCG_WaitFircClockStable --> SCG_HWA_GetFircValid[SCG_HWA_GetFircValid];</pre>					
Parameters	<table><tr><th>Parameter</th><th>Description</th></tr><tr><td>pFircConfig</td><td>Pointer to the FIRC Type structure data instance, which defined FIRC initial information.</td></tr></table>	Parameter	Description	pFircConfig	Pointer to the FIRC Type structure data instance, which defined FIRC initial information.	
Parameter	Description					
pFircConfig	Pointer to the FIRC Type structure data instance, which defined FIRC initial information.					
Register Accessed	SCG_FIRCCSR(rw), SCG_FIRCDIV(rw), SCG_FIRCCFG(rw), SCG_FIRCTCFG(rw), SCG_FIRCSTAT(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>					
Returns	FIRC status. FIRC would wait for FIRC valid in while loop within limited time to check the FIRC valid or not. If FIRC still not valid, it would return fail.					
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)					

[SWDESG_MCU_108] SCG_SetFIRC function used to initialize FIRC clock.

[SWDESG_MCU_164] SCG_SetFIRC function call SCG_WaitFircClockStable to wait FIRC stable and SCG_WaitFircClockStable use while loop with dedicated escape mechanism to avoid infinite loop.

2.8.2.1.5 SCG_SetFOSC

Function	SCG_StatusType SCG_SetFOSC(const SCG_FoscType* pFoscConfig)
Description	Set FOSC configurations.

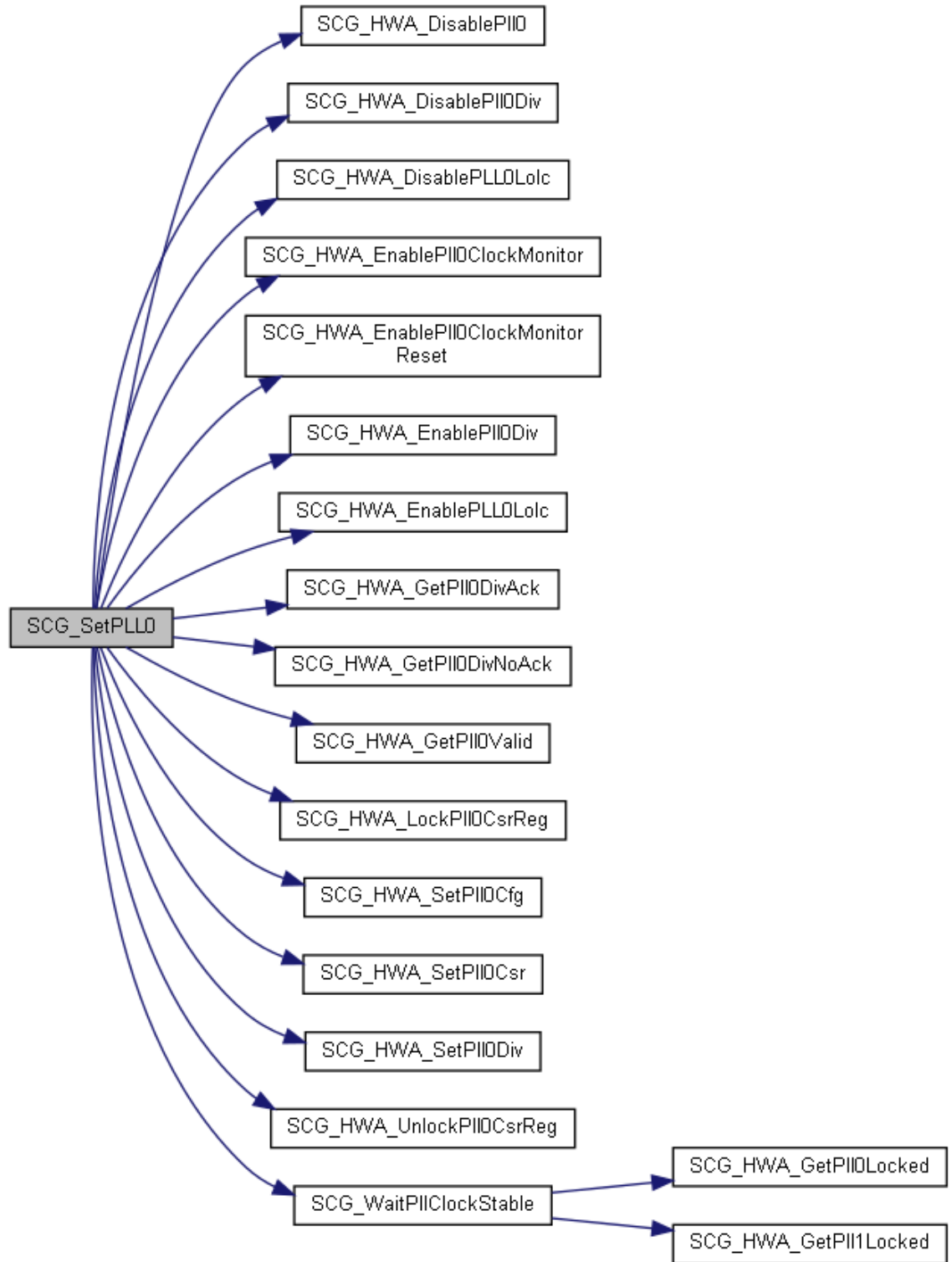
Diagram	<pre>graph LR SCG_SetFOSC[SCG_SetFOSC] --> SCG_HWA_DisableFosc[SCG_HWA_DisableFosc] SCG_SetFOSC --> SCG_HWA_EnableFoscClockMonitor[SCG_HWA_EnableFoscClockMonitor] SCG_SetFOSC --> SCG_HWA_EnableFoscClockMonitor_Reset[SCG_HWA_EnableFoscClockMonitor Reset] SCG_SetFOSC --> SCG_HWA_EnableFoscDiv[SCG_HWA_EnableFoscDiv] SCG_SetFOSC --> SCG_HWA_GetFoscCsr[SCG_HWA_GetFoscCsr] SCG_SetFOSC --> SCG_HWA_GetFoscDivAck[SCG_HWA_GetFoscDivAck] SCG_SetFOSC --> SCG_HWA_LockFoscCsrReg[SCG_HWA_LockFoscCsrReg] SCG_SetFOSC --> SCG_HWA_SetFoscCfg[SCG_HWA_SetFoscCfg] SCG_SetFOSC --> SCG_HWA_SetFoscCsr[SCG_HWA_SetFoscCsr] SCG_SetFOSC --> SCG_HWA_SetFoscDiv[SCG_HWA_SetFoscDiv] SCG_SetFOSC --> SCG_HWA_UnlockFoscCsrReg[SCG_HWA_UnlockFoscCsrReg] SCG_SetFOSC --> SCG_WaitFoscClockStable[SCG_WaitFoscClockStable] SCG_WaitFoscClockStable --> SCG_HWA_GetFoscValid[SCG_HWA_GetFoscValid]</pre>					
Parameters	<table><tr><th>Parameter</th><th>Description</th></tr><tr><td>pFoscConfig</td><td>Pointer to the FOSC Type structure data instance, which defined FOSC initial information.</td></tr></table>	Parameter	Description	pFoscConfig	Pointer to the FOSC Type structure data instance, which defined FOSC initial information.	
Parameter	Description					
pFoscConfig	Pointer to the FOSC Type structure data instance, which defined FOSC initial information.					
Register Accessed	SCG_FOSCCSR(rw), SCG_FOSCDIV(rw), SCG_FOSCCFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>					
Returns	FOSC status. FOSC would wait for FOSC valid in while loop within limited time to check the FOSC valid or not. If FOSC still not valid(this may happen if external Fast OSC not placed), it would return fail.					
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)					

[SWDESG_MCU_109] SCG_SetFOSC function used to initialize FOSC clock.

[SWDESG_MCU_165] SCG_SetFOSC function call SCG_WaitFoscClockStable to wait FOSC stable and SCG_WaitFoscClockStable use while loop with dedicated escape mechanism to avoid infinite loop.

2.8.2.1.6 SCG_SetPLL0

Function	SCG_StatusType SCG_SetPLL0(const SCG_PllType *pPll0Config)
Description	Set PLL0 configuration.

Diagram**Parameters****Parameter****Description**

pPII0Config

Pointer to the PLL0 Type structure data instance, which defined PLL0 initial information.

Register Accessed

SCG_PLL0CSR(rw), SCG_PLL0DIV(rw), SCG_PLL0CFG(rw)

Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.

Returns

PLL0 status. PLL0 would wait for PLL0 valid in while loop within limited time to check the PLL0 valid or not. If PLL0 still not valid(this may happen if external Fast OSC not placed), it would return fail. PLL0 would check the status of input clock source(FIRC or FOSC), if it is not valid, it would return FALSE.

Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)
----------------------	--

[SWDESG_MCU_110] SCG_SetPll0 function used to initialize PLL0 clock.

[SWDESG_MCU_111] SCG_SetPll0 function call SCG_WaitPll0ClockStable to wait PLL lock and SCG_WaitPll0ClockStable use while loop with dedicated escape mechanism to avoid infinite loop.

[SWDESG_MCU_112] SCG_SetPll0 function configure PLL parameters like percale value is adjustable in the configuration set.

2.8.2.1.7 SCG_SetPLL1

Function	SCG_StatusType SCG_SetPLL1(const SCG_PllType * pPll1Config)	
Description	Set PLL1 configuration.	
Diagram	<pre> graph LR SCG_SetPLL1[SCG_SetPLL1] --> SCG_HWA_DisablePll1[SCG_HWA_DisablePll1] SCG_SetPLL1 --> SCG_HWA_DisablePll1Div[SCG_HWA_DisablePll1Div] SCG_SetPLL1 --> SCG_HWA_DisablePll1Lolc[SCG_HWA_DisablePll1Lolc] SCG_SetPLL1 --> SCG_HWA_EnablePll1ClockMonitor[SCG_HWA_EnablePll1ClockMonitor] SCG_SetPLL1 --> SCG_HWA_EnablePll1ClockMonitor_Reset[SCG_HWA_EnablePll1ClockMonitor Reset] SCG_SetPLL1 --> SCG_HWA_EnablePll1Div[SCG_HWA_EnablePll1Div] SCG_SetPLL1 --> SCG_HWA_EnablePll1Lolc[SCG_HWA_EnablePll1Lolc] SCG_SetPLL1 --> SCG_HWA_GetPll1DivAck[SCG_HWA_GetPll1DivAck] SCG_SetPLL1 --> SCG_HWA_GetPll1DivNoAck[SCG_HWA_GetPll1DivNoAck] SCG_SetPLL1 --> SCG_HWA_GetPll1Valid[SCG_HWA_GetPll1Valid] SCG_SetPLL1 --> SCG_HWA_LockPll1CsrReg[SCG_HWA_LockPll1CsrReg] SCG_SetPLL1 --> SCG_HWA_SetPll1Cfg[SCG_HWA_SetPll1Cfg] SCG_SetPLL1 --> SCG_HWA_SetPll1Csr[SCG_HWA_SetPll1Csr] SCG_SetPLL1 --> SCG_HWA_SetPll1Div[SCG_HWA_SetPll1Div] SCG_SetPLL1 --> SCG_HWA_UnlockPll1CsrReg[SCG_HWA_UnlockPll1CsrReg] SCG_SetPLL1 --> SCG_WaitPll1ClockStable[SCG_WaitPll1ClockStable] SCG_WaitPll1ClockStable --> SCG_HWA_GetPll1Locked_1[SCG_HWA_GetPll1Locked] SCG_WaitPll1ClockStable --> SCG_HWA_GetPll1Locked_2[SCG_HWA_GetPll1Locked] </pre>	
Parameters	Parameter	Description

	pPll1Config	Pointer to the PLL1 Type structure data instance, which defined PLL0 initial information.
Register Accessed	SCG_PLL1CSR(rw), SCG_PLL1DIV(rw), SCG_PLL1CFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	PLL1 status. PLL1 would wait for PLL1 valid in while loop within limited time to check the PLL1 valid or not. If PLL0 still not valid(this may happen if external Fast OSC not placed), it would return fail. PLL1 would check the status of input clock source(FIRC or FOSC), if it is not valid, it would return FALSE.	
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)	

[SWDESG_MCU_226] SCG_SetPll1 function used to initialize PLL1 clock.

[SWDESG_MCU_227] SCG_SetPll1 function call SCG_WaitPll1ClockStable to wait PLL lock and SCG_WaitPll1ClockStable use while loop with dedicated escape mechanism to avoid infinite loop.

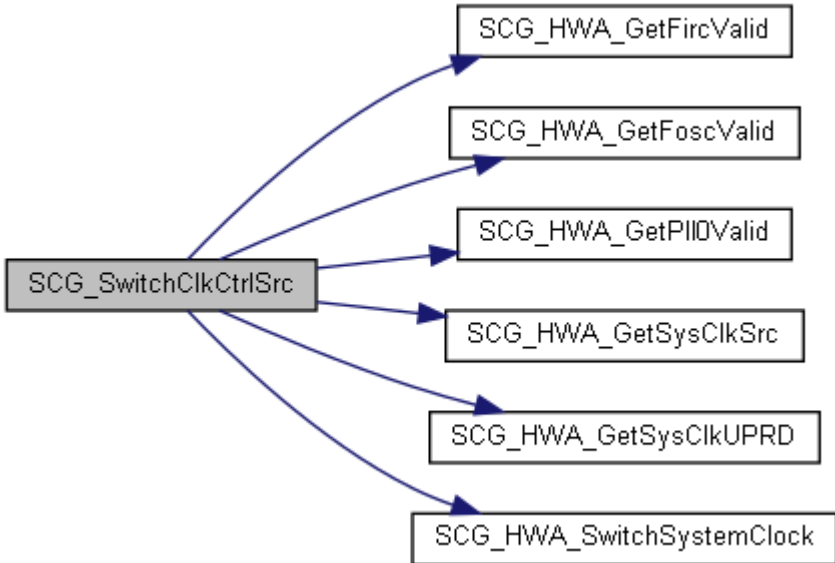
[SWDESG_MCU_228] SCG_SetPll1 function configure PLL parameters like percale value is adjustable in the configuration set.

2.8.2.1.8 SCG_SetClkCtrl

Function	SCG_StatusType SCG_SetClkCtrl(const SCG_ClockCtrlType *const pSysClkConfig)	
Description	Set system run time clock and related CORE/BUS/SLOW clock.	
Diagram	<pre> graph LR A[SCG_SetClkCtrl] --> B[SCG_HWA_GetFircValid] A --> C[SCG_HWA_GetFoscValid] A --> D[SCG_HWA_GetPll0Valid] A --> E[SCG_HWA_GetSysClkSrc] A --> F[SCG_HWA_GetSysClkUPRD] A --> G[SCG_HWA_SetCCR] </pre>	
Parameters	Parameter	Description
	pSysClkConfig	pointer to the clockCtrlType structure data instance, which defined for system clock selection.
Register Accessed	SCG_CSR(r), SCG_CCR(rw), SCG_CLKOUTCFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	System clock status. This function would check the clock source status before set it system clock, if the chosen clock source is invalid, it would return FALSE.	
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)	

[SWDESG_MCU_113] SCG_SetClkCtrl function used to initialize system clock.

2.8.2.1.9 SCG_SwitchClkCtrlSrc

Function	SCG_StatusType SCG_SwitchClkCtrlSrc(SCG_ClockSrcType eClock)	
Description	Set system run time clock and related CORE/BUS/SLOW clock.	
Diagram	 <pre> graph LR A[SCG_SwitchClkCtrlSrc] --> B[SCG_HWA_GetFircValid] A --> C[SCG_HWA_GetFoscValid] A --> D[SCG_HWA_GetPIIOValid] A --> E[SCG_HWA_GetSysClkSrc] A --> F[SCG_HWA_GetSysClkUPRD] A --> G[SCG_HWA_SwitchSystemClock] </pre>	
Parameters	Parameter	Description
	eClock	Target clock source user wants to switch.
Register Accessed	SCG_CSR(r), SCG_CCR(rw), SCG_CLKOUTCFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	Switch clock source status. This function check if the clock source is in proper range.	
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)	

[SWDESG_MCU_114] SCG_SwitchClkCtrlSrc function used to switch system clock.

2.8.2.1.10 SCG_SetClkOut

Function	void SCG_SetClkOut(const SCG_ClockCtrlType *pSysClkConfig);	
Description	Set clock out source in SCG. it set SCG_CLKOUTCFG [CLKOUTSEL].	
Diagram	 <pre> graph LR A[SCG_SetClkOut] --> B[SCG_HWA_SetClkOutCfg] </pre>	
Parameters	Parameter	Description
	pSysClkConfig	Pointer to the clockCtrlType structure data instance, which defined for system clock selection.
Register Accessed	SCG_CLKOUTCFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)	

[SWDESG_MCU_117] SCG_SetClkOut function used to set clock out configurations.

2.8.2.1.11 SCG_GetClockStatus

Function	boolean SCG_GetClockStatus (SCG_ClkSrcType eClock)
-----------------	--

Description	Get SCG clock status.	
Diagram	<pre> graph LR SCG_GetClockStatus[SCG_GetClockStatus] --> SCG_HWA_GetFircValid[SCG_HWA_GetFircValid] SCG_GetClockStatus --> SCG_HWA_GetFoscDivLValid[SCG_HWA_GetFoscDivLValid] SCG_GetClockStatus --> SCG_HWA_GetFoscValid[SCG_HWA_GetFoscValid] SCG_GetClockStatus --> SCG_HWA_GetPll0Valid[SCG_HWA_GetPll0Valid] SCG_GetClockStatus --> SCG_HWA_GetSirc32kValid[SCG_HWA_GetSirc32kValid] SCG_GetClockStatus --> SCG_HWA_GetSoscValid[SCG_HWA_GetSoscValid] </pre>	
Parameters	Parameter	Description
	eClock	SCG clock
Register Accessed	SCG_CSR(r), SCG_FOSCCSR(rw), SCG_FOSCDIV(rw), SCG_SIRCCSR(rw), SCG_FIRCCSR(rw), SCG_SOSCCSR(rw), SCG_PLL0CSR(rw), SCG_SIRC32KCSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	true: SCG clock is valid, false: SCG clock is invalid or parameter invalid	
Referenced By	boolean SMISC_GetSmiscClockStatus(SMISC_ClkSrcType eSmiscClockName)	

[SWDESG_MCU_115] SCG_GetClockStatus function used to get clock status like FIRC, FOSC, etc.

2.8.2.1.12 SCG_GetSysClockSrc

Function	SCG_SystemClkType SCG_GetSysClockSrc (void)	
Description	Get system clock source.	
Diagram	<pre> graph LR SCG_GetSysClockSrc[SCG_GetSysClockSrc] --> SCG_HWA_GetSysClkSrc[SCG_HWA_GetSysClkSrc] </pre>	
Parameters	Parameter	Description
	N/A	N/A
Register Accessed	SCG_CSR(r) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	System clock source type	
Referenced By	static SCG_StatusType MCU_SCG_Init(const SCG_ClockType *const pScgCfg)	

[SWDESG_MCU_116] SCG_GetSysClockSrc function used to get system clock source.

2.8.2.1.13 SCG_GetSysClockValid

Function	boolean SCG_GetSysClockValid(void)
Description	Get system clock valid.

Diagram	<pre> graph LR SCG_GetSysClockValid --> SCG_GetSysClockSrc SCG_GetSysClockValid --> SCG_HWA_GetFircValid SCG_GetSysClockValid --> SCG_HWA_GetFoscValid SCG_GetSysClockValid --> SCG_HWA_GetPIIOValid SCG_GetSysClockSrc --> SCG_HWA_GetSysClkSrc </pre>
Parameters	N/A
Register Accessed	SCG_CSR(r), SCG_FIRCCSR(rw), SCG_FOSCCSR(rw), SCG_PLL0CSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	True/False
Referenced By	CMU_StatusType CMU_Init(CMU_InstanceType eInstance, const CMU_CfgType *const pInitCfg)

[SWDESG_MCU_229] SCG_GetSysClockValid function used to get system clock valid status.

2.8.2.1.14 SCG_GetCMU4ClockSrc

Function	SCG_Cmu4ClkSrcType SCG_GetCMU4ClockSrc (void)
Description	Get system clock valid.
Diagram	<pre> graph LR SCG_GetCMU4ClockSrc --> SCG_HWA_GetClkOutCfg </pre>
Parameters	N/A
Register Accessed	SCG_CLKOUTCFG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	SCG_Cmu4ClkSrcType
Referenced By	CMU_StatusType CMU_Init(CMU_InstanceType eInstance, const CMU_CfgType *const pInitCfg)

[SWDESG_MCU_230] SCG_GetCMU4ClockSrc function used to get CMU4 clock source.

2.8.2.2 Functions in Mcu_Pcc.h

2.8.2.2.1 PCC_SetPcc

Function	void PCC_SetPcc(const PCC_ClkCtrlType *const pPccConfig)	
Description	set PCC one peripheral clock configuration.	
Diagram	N/A	
Parameters	Parameter	Description
	pPccConfig	the PCC initialize value point set by user.
Register Accessed	PCC_XXX(rw) ('XXX' represents specific peripherals)	

	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	static void MCU_PCC_Init(const PCC_GrpConfigType *const pccGrpConfig)

[SWDESG_MCU_118] PCC_SetPcc function used to configure peripheral clock controls.

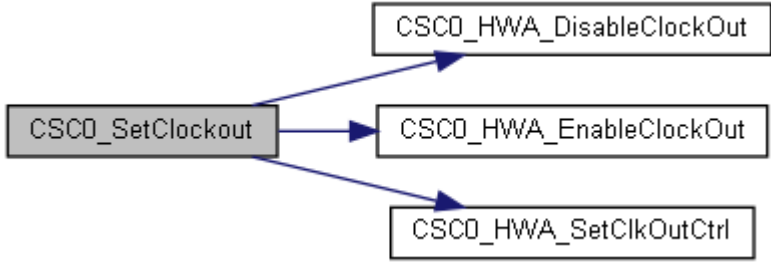
2.8.2.2.2 PCC_GetPccDwp

Function	uint8 PCC_GetPccDwp(uint32 u32OffSet)	
Description	Get PCC DWP configuration.	
Diagram	N/A	
Parameters	Parameter	Description
	pPccConfig	the PCC initialize value point set by user.
Register Accessed	PCC_XXX(rw) ('XXX' represents specific peripherals) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_Hw_CmuDisableNoStop(const CLOCK_CMU_ConfigType *const CLOCK_CMU_Config)	

[SWDESG_MCU_231] PCC_GetPccDwp function used to get PCC DWP configuration.

2.8.2.3 Functions in Mcu_Csc.h

2.8.2.3.1 CSC0_SetClockout

Function	void CSC0_SetClockout(const CSC0_ClkoutType *pCsc0ClkOut)	
Description	Set clock out. with clock out pin configure, the clock would be monitored. need to call SCG_SetClkOut, if clock out source set to SCG CLKOUT.	
Diagram	 <pre> graph LR A[CSC0_SetClockout] --> B[CSC0_HWA_DisableClockOut] A --> C[CSC0_HWA_EnableClockOut] A --> D[CSC0_HWA_SetClkOutCtrl] </pre>	
Parameters	Parameter	Description
	pCsc0ClkOut	Pointer to CSC0_ClkOut instance for clock out configuration.
Register Accessed	CSC0_CLKOUT_CTRL (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_CSC0_Init(const CLOCK_CSC0_ConfigType *const CLOCK_CSC0_pConfig)	

[SWDESG_MCU_119] CSC0_SetClockout function used to set clock out configuration which the control bit is include in the CSC peripheral register.

2.8.2.3.2 CSC0_SetAonClkSrc

Function	void CSC0_SetAonClkSrc(const CSC0_AONCLKSRType *pAonclkSrcType)	
Description	Set always on clock source configuration include AON32K, RTC, AONCLK clock.	
Diagram	 <pre> graph LR A[CSC0_SetAonClkSrc] --> B[CSC0_HWA_SetAonClkReg] </pre>	
Parameters	Parameter	Description
	pAonclkSrcType	Pointer to AONCLKSR instance for AON clock source configuration
Register Accessed	CSC0_AONCLKSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_CSC0_Init(const CLOCK_CSC0_ConfigType *const CLOCK_CSC0_pConfig)	

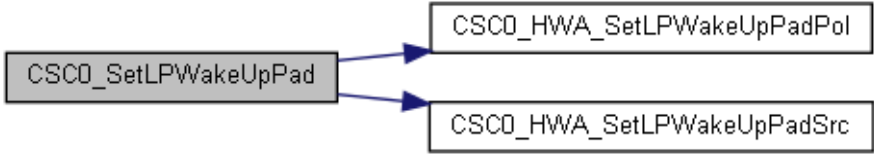
[SWDESG_MCU_121] CSC0_SetAonClkSrc function used to set AONCLK clock source configurations.

2.8.2.3.3 CSC0_EnableHClock

Function	void CSC0_EnableHClock(void)	
Description	Enable HClock to keep clock of ram in TCM active, prevent DMA access to TCM memory form some error when low power request.	
Diagram	N/A	
Parameters	N/A	
Register Accessed	CSC0_CCM0_CFG (rw), <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_CSC0_Init(const CLOCK_CSC0_ConfigType *const CLOCK_CSC0_pConfig)	

[SWDESG_MCU_122] CSC0_EnableHClock function used to enable HClock to keep clock of ram in TCM active, prevent DMA access to TCM memory form some error when low power request.

2.8.2.3.4 CSC0_SetLPWakeUpPad

Function	void CSC0_SetLPWakeUpPad(const CSC0_LPWakeUpPadType* pLPWakeUpPad)	
Description	Configure the low power wakeup PAD output in CSC0_LP_WAKEUP register	
Diagram	 <pre> graph LR A[CSC0_SetLPWakeUpPad] --> B[CSC0_HWA_SetLPWakeUpPadPol] A --> C[CSC0_HWA_SetLPWakeUpPadSrc] </pre>	
Parameters	Parameter	Description
	pLPWakeUpPad	Data structure point of low power wakeup PAD configuration.
Register Accessed	CSC0->LP_WAKEUP (rw)	

	<i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	static void MCU_CSC0_Init(const CLOCK_CSC0_ConfigType *const CLOCK_CSC0_pConfig)

[SWDESG_MCU_123] CSC0_SetLPWakeUpPad function used to configure the low power wakeup PAD output.

2.8.2.3.5 CSC0_GetCsc0ClockStatus

Function	boolean CSC0_GetCsc0ClockStatus(CSC0_ClkSrcType eCsc0ClockName)	
Description	Get CSC0 clock status	
Diagram	<pre> graph LR CSC0_GetCsc0ClockStatus --> CSC0_GetAon32kClockStatus CSC0_GetCsc0ClockStatus --> CSC0_GetRtcClockStatus CSC0_GetCsc0ClockStatus --> CSC0_GetAonClockStatus CSC0_GetAon32kClockStatus --> CSC0_HWA_GetAon32kClk CSC0_GetRtcClockStatus --> CSC0_HWA_GetSircDiv32KClk CSC0_GetAonClockStatus --> CSC0_HWA_GetRtcClk CSC0_HWA_GetAon32kClk --> SCG_GetClockStatus CSC0_HWA_GetSircDiv32KClk --> SCG_GetClockStatus CSC0_HWA_GetRtcClk --> SCG_GetClockStatus SCG_GetClockStatus --> SCG_HWA_GetFircValid SCG_GetClockStatus --> SCG_HWA_GetFoscDivLValid SCG_GetClockStatus --> SCG_HWA_GetFoscValid SCG_GetClockStatus --> SCG_HWA_GetPI0Locked SCG_GetClockStatus --> SCG_HWA_GetPI1Locked SCG_GetClockStatus --> SCG_HWA_GetSirc32kValid SCG_GetClockStatus --> SCG_HWA_GetSoscValid </pre>	
Parameters	Parameter	Description
	eCsc0ClockName	CSC0 AONCLKSR register clock source select.
Register Accessed	CSC0_AONCLKSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static CMU_StatusType CMU_ClkStatusCheck(CMU_InstanceType eInstance)	

[SWDESG_MCU_242] CSC0_GetCsc0ClockStatus function used to get CSC0 clock status.

2.8.2.3.6 CSC0_GetAon32kClockStatus

Function	static boolean CSC0_GetAon32kClockStatus(void)	
Description	Get CSC0 Aon32k clock status	
Diagram	<pre> graph LR CSC0_GetAon32kClockStatus --> CSC0_HWA_GetAon32kClk CSC0_GetAon32kClockStatus --> CSC0_HWA_GetSircDiv32KClk CSC0_HWA_GetAon32kClk --> SCG_GetClockStatus CSC0_HWA_GetSircDiv32KClk --> SCG_GetClockStatus SCG_GetClockStatus --> SCG_HWA_GetFircValid SCG_GetClockStatus --> SCG_HWA_GetFoscDivLValid SCG_GetClockStatus --> SCG_HWA_GetFoscValid SCG_GetClockStatus --> SCG_HWA_GetPI0Locked SCG_GetClockStatus --> SCG_HWA_GetPI1Locked SCG_GetClockStatus --> SCG_HWA_GetSirc32kValid SCG_GetClockStatus --> SCG_HWA_GetSoscValid </pre>	

Parameters	N/A
Register Accessed	CSC0_AONCLKSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	boolean CSC0_GetCsc0ClockStatus(CSC0_ClkSrcType eCsc0ClockName)

[SWDESG_MCU_243] CSC0_GetAon32kClockStatus function used to get CSC0 Aon32k clock status.

2.8.2.3.7 CSC0_GetRtcClockStatus

Function	static boolean CSC0_GetRtcClockStatus(void)
Description	Get CSC0 Rtc clock status
Diagram	<pre> graph LR CSC0_GetRtcClockStatus --> CSC0_HWA_GetRtcClk CSC0_GetRtcClockStatus --> CSC0_HWA_GetSircDiv32KClk CSC0_GetRtcClockStatus --> SCG_GetClockStatus SCG_GetClockStatus --> SCG_HWA_GetFircValid SCG_GetClockStatus --> SCG_HWA_GetFoscDivLValid SCG_GetClockStatus --> SCG_HWA_GetFoscValid SCG_GetClockStatus --> SCG_HWA_GetPII0Locked SCG_GetClockStatus --> SCG_HWA_GetPII1Locked SCG_GetClockStatus --> SCG_HWA_GetSirc32kValid SCG_GetClockStatus --> SCG_HWA_GetSoscValid </pre>
Parameters	N/A
Register Accessed	CSC0_AONCLKSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	boolean CSC0_GetCsc0ClockStatus(CSC0_ClkSrcType eCsc0ClockName)

[SWDESG_MCU_244] CSC0_GetRtcClockStatus function used to get CSC0 Rtc clock status.

2.8.2.3.8 CSC0_GetAonClockStatus

Function	static boolean CSC0_GetAonClockStatus(void)
Description	Get CSC0 Aon clock status

Diagram	<pre> graph LR CSC0_GetAonClockStatus --> CSC0_HWA_GetAonClk CSC0_GetAonClockStatus --> CSC0_HWA_GetSirc32KDiv1KClk CSC0_GetAonClockStatus --> CSC0_HWA_GetSircDiv32KClk CSC0_GetAonClockStatus --> SCG_GetClockStatus SCG_GetClockStatus --> SCG_HWA_GetFircValid SCG_GetClockStatus --> SCG_HWA_GetFoscDivLValid SCG_GetClockStatus --> SCG_HWA_GetFoscValid SCG_GetClockStatus --> SCG_HWA_GetPIIOLocked SCG_GetClockStatus --> SCG_HWA_GetPII1Locked SCG_GetClockStatus --> SCG_HWA_GetSirc32kValid SCG_GetClockStatus --> SCG_HWA_GetSoscValid </pre>
Parameters	N/A
Register Accessed	CSC0_AONCLKSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	boolean CSC0_GetCsc0ClockStatus(CSC0_ClkSrcType eCsc0ClockName)

[SWDESG_MCU_244] CSC0_GetRtcClockStatus function used to get CSC0 Aon clock status.

2.8.2.4 Functions in Mcu_Scm.h

2.8.2.4.1 SCM_SetTraceClockConfig

Function	void SCM_SetTraceClockConfig(const SCM_TraceClkDivType *pTraceClk)	
Description	Enable to set trace clock.	
Diagram	<pre> graph LR SCM_SetTraceClockConfig --> SCM_HWA_EnableDebugATClk SCM_SetTraceClockConfig --> SCM_HWA_SetDebugTrace </pre>	
Parameters	Parameter	Description
	pTraceClk	Data structure point of debug trace configuration.
Register Accessed	SCM_DEBUG_TRACE (rw), <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_SCM_Init(const CLOCK_SCM_ConfigType *const CLOCK_SCM_pConfig)	

[SWDESG_MCU_232] SCM_SetTraceClockConfig function used to set trace clock.

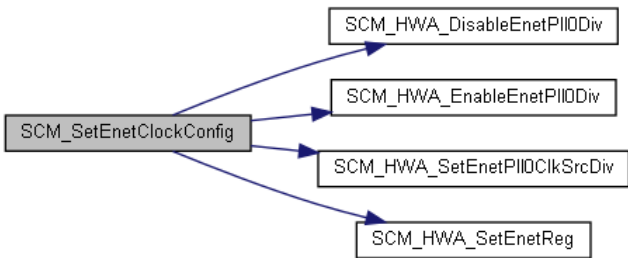
2.8.2.4.2 SCM_SetHsmClockConfig

Function	void SCM_SetHsmClockConfig(const SCM_HsmClkSelType *pHsmClk)	
Description	Set Hsm clock.	
Diagram	<pre> graph LR SCM_SetHsmClockConfig --> SCM_HWA_SetHsmReg </pre>	

Parameters	Parameter	Description
	pHsmClk	Data structure point of SCM Hsm clock configuration
Register Accessed	SCM_HSM_PCC (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_SCM_Init(const CLOCK_SCM_ConfigType *const CLOCK_SCM_pConfig)	

[SWDESG_MCU_233] SCM_SetHsmClockConfig function used to set Hsm clock.

2.8.2.4.3 SCM_SetEnetClockConfig

Function	void SCM_SetEnetClockConfig(const SCM_EnetClkSelType *pEnetClk);	
Description	Set Enet clock.	
Diagram	 <pre> graph LR A[SCM_SetEnetClockConfig] --> B[SCM_HWA_DisableEnetPIODiv] A --> C[SCM_HWA_EnableEnetPIODiv] A --> D[SCM_HWA_SetEnetPIODivSrcDiv] A --> E[SCM_HWA_SetEnetReg] </pre>	
Parameters	Parameter	Description
	pEnetClk	Data structure point of SCM Enet clock configuration.
Register Accessed	SCM_PERI_CLKDIV (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_SCM_Init(const CLOCK_SCM_ConfigType *const CLOCK_SCM_pConfig)	

[SWDESG_MCU_234] SCM_SetEnetClockConfig function used to set Enet clock.

2.8.2.4.4 SCM_SetSubSysClockConfig

Function	void SCM_SetSubSysClockConfig(const SCM_SubSysClkType *pSubSysClk)	
Description	Set SubSystem clock.	
Diagram		
Parameters	Parameter	Description
	pSubSysClk	Data structure point of SCM subsystem clock configuration
Register Accessed	SCM_SUBSYS_PCC (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	

Referenced By	static void MCU_SCM_Init(const CLOCK_SCM_ConfigType *const CLOCK_SCM_pConfig)
----------------------	---

[SWDESG_MCU_235] SCM_SetSubSysClockConfig function used to set subsystem clock.

2.8.2.5 Functions in Mcu_Cmu.h

2.8.2.5.1 CMU_Init

Function	CMU_StatusType CMU_Init(CMU_InstanceType eInstance, const CMU_CfgType*const pInitCfg)	
Description	Initialize the CMU instance.	
Diagram	<pre> graph LR CMU_Init[CMU_Init] --> CMU_HWA_DisableCmu[CMU_HWA_DisableCmu] CMU_Init --> CMU_HWA_EnableCmu[CMU_HWA_EnableCmu] CMU_Init --> CMU_HWA_SetCtrlReg[CMU_HWA_SetCtrlReg] CMU_Init --> CMU_HWA_SetMaxCnts[CMU_HWA_SetMaxCnts] CMU_Init --> CMU_HWA_SetMinCnts[CMU_HWA_SetMinCnts] CMU_Init --> CMU_HWA_SetPeriodEnable[CMU_HWA_SetPeriodEnable] CMU_Init --> CMU_HWA_SetPeriodWindow[CMU_HWA_SetPeriodWindow] CMU_Init --> CMU_HWA_SetRefWindow[CMU_HWA_SetRefWindow] CMU_Init --> CMU_ClkStatusCheck[CMU_ClkStatusCheck] CMU_Init --> CMU_HWA_ClsST[CMU_HWA_ClsST] CMU_ClkStatusCheck --> SMISC_GetSmiscClockStatus[SMISC_GetSmiscClockStatus] CMU_ClkStatusCheck --> SCG_GetClockStatus[SCG_GetClockStatus] SMISC_GetSmiscClockStatus --> SMISC_HWA_GetAon32kClk[SMISC_HWA_GetAon32kClk] SMISC_GetSmiscClockStatus --> SMISC_HWA_GetRtcClk[SMISC_HWA_GetRtcClk] SMISC_GetSmiscClockStatus --> SMISC_HWA_GetSircDiv32KClk[SMISC_HWA_GetSircDiv32KClk] SCG_GetClockStatus --> SCG_HWA_GetFircValid[SCG_HWA_GetFircValid] SCG_GetClockStatus --> SCG_HWA_GetFoscDivLValid[SCG_HWA_GetFoscDivLValid] SCG_GetClockStatus --> SCG_HWA_GetFoscValid[SCG_HWA_GetFoscValid] SCG_GetClockStatus --> SCG_HWA_GetPIIOValid[SCG_HWA_GetPIIOValid] SCG_GetClockStatus --> SCG_HWA_GetSirc32kValid[SCG_HWA_GetSirc32kValid] SCG_GetClockStatus --> SCG_HWA_GetSoscValid[SCG_HWA_GetSoscValid] </pre>	
Parameters	Parameter	Description
	pInitCfg	CMU configuration pointer
	eInstance	CMU module index.
Register Accessed	CMU_CTRL(rw), CMU_MIN(rw), CMU_MAX(rw), CMU_REF_WINDOW(rw), CMU_MON_CNT(r), CMU_ST(rw), CMU_PERIOD(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_CMU_Init(const CLOCK_CMU_ConfigType *const CLOCK_CMU_Config)	

[SWDESG_MCU_124] CMU_Init function used to initialize CMU peripheral.

[SWDESG_MCU_168] CMU_Init function check the relate clock status before enable CMU.

2.8.2.5.2 CMU_Enable

Function	void CMU_Enable(CMU_InstanceType eInstance)	
Description	Enable CMU instance.	
Diagram	<pre> graph LR CMU_Enable[CMU_Enable] --> CMU_HWA_EnableCmu[CMU_HWA_EnableCmu] </pre>	
Parameters	Parameter	Description

	eInstance	CMU instance
Register Accessed	CMU_CTRL(rw), CMU_ST(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_CMU_Enable(const CMU_InstanceType eInstance, const CMU_CfgType *const CMU_Config)	

[SWDESG_MCU_125] CMU_Enable function used to enable CMU peripheral.

2.8.2.5.3 CMU_Disable

Function	void CMU_Disable(CMU_InstanceType eInstance)	
Description	Disable CMU instance.	
Diagram	 <pre> graph LR A[CMU_Disable] --> B[CMU_HWA_DisableCmu] </pre>	
Parameters	Parameter	Description
	eInstance	CMU instance
Register Accessed	CMU_CTRL(rw), CMU_ST(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_CMU_Disable(const CMU_InstanceType eInstance, const CMU_CfgType *const CMU_Config)	

[SWDESG_MCU_126] CMU_Disable function used to disable CMU peripheral.

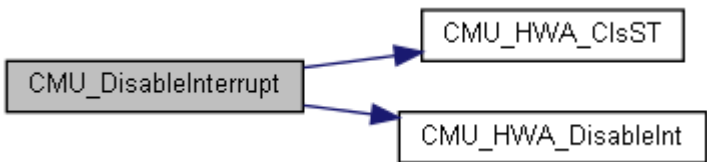
2.8.2.5.4 CMU_EnableInterrupt

Function	void CMU_EnableInterrupt(CMU_InstanceType eInstance)	
Description	Enable CMU interrupt	
Diagram	 <pre> graph LR A[CMU_EnableInterrupt] --> B[CMU_HWA_EnableInt] </pre>	
Parameters	Parameter	Description
	eInstance	CMU instance
Register Accessed	CMU_CTRL(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_CMU_Enable(const CMU_InstanceType eInstance, const CMU_CfgType *const CMU_Config)	

[SWDESG_MCU_127] CMU_EnableInterrupt function used to enable CMU interrupt functions.

2.8.2.5.5 CMU_DisableInterrupt

Function	void CMU_DisableInterrupt (CMU_InstanceType eInstance)
-----------------	--

Description	Enable CMU interrupt	
Diagram	 <pre> graph LR A[CMU_DisableInterrupt] --> B[CMU_HWA_ClsST] A --> C[CMU_HWA_DisableInt] </pre>	
Parameters	Parameter	Description
	eInstance	CMU instance
Register Accessed	CMU_CTRL(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_CMU_Disable(const CMU_InstanceType eInstance, const CMU_CfgType *const CMU_Config)	

[SWDESG_MCU_128] CMU_DisableInterrupt function used to disable CMU interrupt functions.

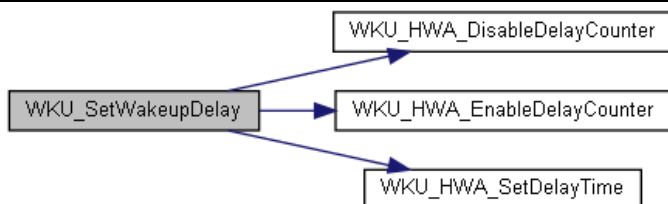
2.8.2.6 Functions in Mcu_Wku.h

2.8.2.6.1 WKU_EnableWakeupSource

Function	WKU_StatusType WKU_EnableWakeupSource(const uint32 u32Input)	
Description	WKU Enable wakeup source for single input	
Diagram	 <pre> graph LR A[WKU_EnableWakeupSource] --> B[WKU_HWA_EnableWakeupSource] </pre>	
Parameters	Parameter	Description
	u32Input	Number of inputs, WKU_WakeupInputType type is wakeup source definition.
Register Accessed	WKU_MWER0(rw), WKU_MWER1(rw), WKU_MWER2(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	Wake up interrupt source inputs	
Referenced By	void Mcu_Hw_WkuEnable(const uint32 u32Input)	

[SWDESG_MCU_129] WKU_EnableWakeupSource function used to set wakeup configurations.

2.8.2.6.2 WKU_SetWakeupDelay

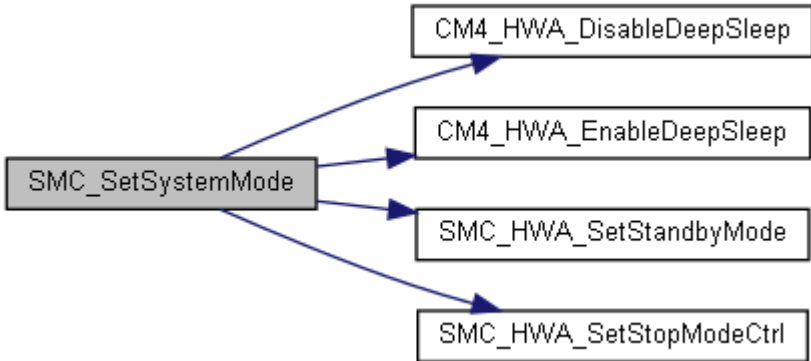
Function	void WKU_SetWakeupDelay(boolean bDelayEn, uint8 u8Delaytime);	
Description	WKU set wake up delay time	
Diagram	 <pre> graph LR A[WKU_SetWakeupDelay] --> B[WKU_HWA_DisableDelayCounter] A --> C[WKU_HWA_EnableDelayCounter] A --> D[WKU_HWA_SetDelayTime] </pre>	
Parameters	Parameter	Description
	bDelayEn	Enable/Disable wakeup delay
	u8Delaytime	The delay time is 2^(u8Delaytime+3) AON_CLK cycles

Register Accessed	WKU_MDC (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>
Returns	N/A
Referenced By	void Mcu_Hw_WkuEnable(const uint32 u32Input)

[SWDESG_MCU_236] WKU_SetWakeupDelay function used to set wakeup delay time.

2.8.2.7 Functions in Mcu_Smc.h

2.8.2.7.1 SMC_SetSystemMode

Function	SMC_StatusType SMC_SetSystemMode(SMC_ModeType eMode)	
Description	Set system mode.	
Diagram	 <pre> graph LR SMC_SetSystemMode[SMC_SetSystemMode] --> CM4_HWA_DisableDeepSleep[CM4_HWA_DisableDeepSleep] SMC_SetSystemMode --> CM4_HWA_EnableDeepSleep[CM4_HWA_EnableDeepSleep] SMC_SetSystemMode --> SMC_HWA_SetStandbyMode[SMC_HWA_SetStandbyMode] SMC_SetSystemMode --> SMC_HWA_SetStopModeCtrl[SMC_HWA_SetStopModeCtrl] </pre>	
Parameters	Parameter	Description
	eMode	Smc configuration mode.
Register Accessed	SMC_PMCTRL(rw), SMC_STANDBY_CFG(rw), SCB->SCR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	Success or param_invalid	
Referenced By	void Mcu_Hw_SmcModeConfig(const Mcu_ModeConfigType *const pModeConfigPtr)	

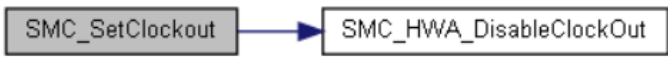
[SWDESG_MCU_130] SMC_SetSystemMode function used to set power mode configurations.

2.8.2.7.2 CM7_GenerateSystemReset

Function	void CM7_GenerateSystemReset (void)	
Description	Generate software reset.	
Diagram	 <pre> graph LR CM7_GenerateSystemReset[CM7_GenerateSystemReset] --> CM7_HWA_SystemReset[CM7_HWA_SystemReset] </pre>	
Parameters	N/A	
Register Accessed	SCB-> AIRCR (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_Hw_Reset(void)	

[SWDESG_MCU_131] CM7_GenerateSystemReset function used to generate software reset.

2.8.2.7.3 SMC_SetClockout

Function	void SMC_SetClockout(const SMC_ClkoutType *pCsc0ClkOut)	
Description	Set clock out. with clock out pin configure, the clock would be monitored. need to call SCG_SetClkOut, if clock out source set to SCG CLKOUT.(Just available in FC7300DQ)	
Diagram	 <pre> graph LR A[SMC_SetClockout] --> B[SMC_HWA_DisableClockOut] </pre>	
Parameters	Parameter	Description
	pSmcClkOut	Pointer to Smc_ClkOut instance for clock out configuration.
Register Accessed	SMC_CLKOUT_CTRL (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_SMC_Init(const CLOCK_SMC_ConfigType *const CLOCK_SMC_pConfig)	

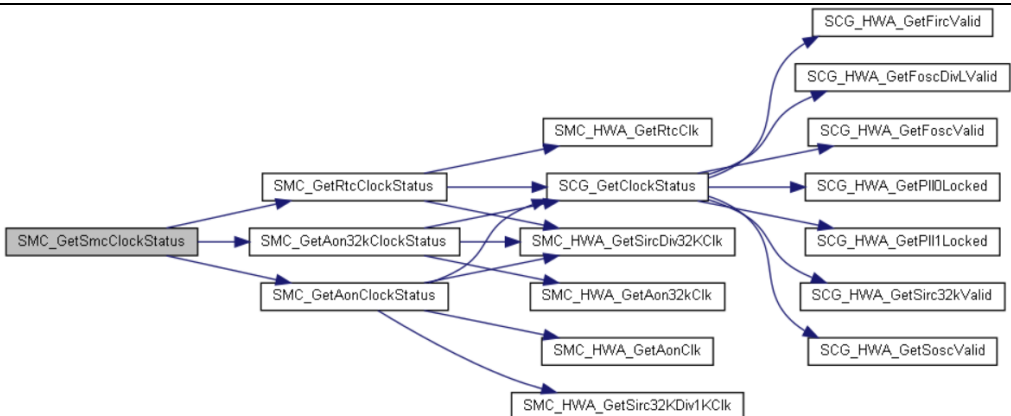
[SWDESG_MCU_119] SWDESG_MCU_119 cover this.

2.8.2.7.4 SMC_SetAonClkSrc

Function	void SMC_SetClockout(const SMC_ClkoutType *pSmcClkOut)	
Description	Set always on clock source configuration include AON32K, RTC, AONCLK clock. (Just available in FC7300DQ)	
Diagram	 <pre> graph LR A[SMC_SetAonClkSrc] --> B[SMC_HWA_SetAonClkReg] </pre>	
Parameters	Parameter	Description
	pSmcClkOut	Pointer to AONCLKSR instance for AON clock source configuration
Register Accessed	SMC_AONCLKSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_SMC_Init(const CLOCK_SMC_ConfigType *const CLOCK_SMC_pConfig)	

[SWDESG_MCU_119] SWDESG_MCU_119 cover this.

2.8.2.7.5 SMC_GetSmcClockStatus

Function	boolean SMC_GetSmcClockStatus(SMC_ClkSrcType eSmcClockName)	
Description	Get Smc clock status(Just available in FC7300DQ)	
Diagram	 <pre> graph LR A[SMC_GetSmcClockStatus] --> B[SMC_GetRtcClockStatus] A --> C[SMC_GetAon32kClockStatus] A --> D[SMC_GetAonClockStatus] B --> E[SMC_HWA_GetRtcClk] C --> F[SMC_HWA_GetSircDiv32kClk] D --> G[SMC_HWA_GetAon32kClk] D --> H[SMC_HWA_GetAonClk] D --> I[SMC_HWA_GetSirc32kDiv1KClk] E --> J[SCG_GetClockStatus] F --> J G --> J H --> J I --> J J --> K[SCG_HWA_GetFircValid] J --> L[SCG_HWA_GetFoscDivLValid] J --> M[SCG_HWA_GetFoscValid] J --> N[SCG_HWA_GetPII0Locked] J --> O[SCG_HWA_GetPII1Locked] J --> P[SCG_HWA_GetSirc32kValid] J --> Q[SCG_HWA_GetSoscValid] </pre>	

Parameters	Parameter	Description
	eSmcClockName	SMC clock source select.
Register Accessed	SMC_AONCLKSR(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static CMU_StatusType CMU_ClkStatusCheck(CMU_InstanceType elnstance)	

[SWDESG_MCU_242] SWDESG_MCU_242 cover this.

2.8.2.7.6 SMC_SetLPWakeUpPad

Function	void SMC_SetLPWakeUpPad(const SMC_LPWakeUpPadType* pLPWakeUpPad)	
Description	Configure the low power wakeup PAD output in CSC0_LP_WAKEUP register(Just available in FC7300DQ)	
Diagram	 <pre> graph LR A[SMC_SetLPWakeUpPad] --> B[SMC_HWA_SetLPWakeUpPadSrc] </pre>	
Parameters	Parameter	Description
	pLPWakeUpPad	Data structure point of low power wakeup PAD configuration.
Register Accessed	SMC->LP_WAKEUP (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_SMC_Init(const CLOCK_SMC_ConfigType *const CLOCK_SMC_pConfig)	

[SWDESG_MCU_123] SWDESG_MCU_123 cover this.

2.8.2.7.7 SMC_SetMixedUsedConfiguration

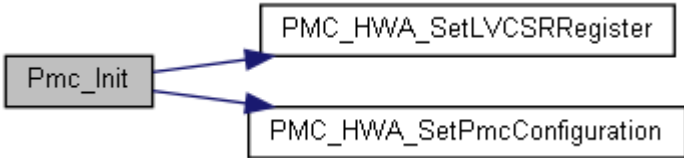
Function	void SMC_SetMixedUsedConfiguration(const SMC_MixedUsedType* pMixedUsedConfig)	
Description	Configure SMC mixed used control (Just available in FC7300DQ)	
Diagram	 <pre> graph LR A[SMC_SetMixedUsedConfiguration] --> B[SMC_HWA_SetRamretReg] </pre>	
Parameters	Parameter	Description
	MixedUsedType	SMC mixed used control type.
Register Accessed	SMC-> PGCTRL (rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	static void MCU_SMC_Init(const CLOCK_SMC_ConfigType *const CLOCK_SMC_pConfig)	

[SWDESG_MCU_123] SWDESG_MCU_123 cover this.

2.8.2.8 Functions in Mcu_Pmc.h

2.8.2.8.1 Pmc_Init

Function	void Pmc_Init (const PMC_InterruptType *const pInterruptCfg)
Description	Initialize PMC function.

Diagram	 <pre> graph LR Pmc_Init[Pmc_Init] --> PMC_HWA_SetLVCSRRegister[PMC_HWA_SetLVCSRRegister] Pmc_Init --> PMC_HWA_SetPmcConfiguration[PMC_HWA_SetPmcConfiguration] </pre>	
Parameters	Parameter	Description
	pInterruptCfg	Initialization structure of PMC
Register Accessed	PMC_LVSCR(rw), PMC_CONFIG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_Hw_PmcPowerInit(const PMC_InterruPtType *const pConfigPtr)	

[SWDESG_MCU_132] Pmc_Init function used to initialize PMC peripheral.

2.8.2.8.2 Pmc_Disable

Function	void Pmc_Disable(void);	
Description	Disable power manage module.	
Diagram	 <pre> graph LR Pmc_Disable[Pmc_Disable] --> PMC_HWA_SetLVCSRRegister[PMC_HWA_SetLVCSRRegister] Pmc_Disable --> PMC_HWA_SetPmcConfiguration[PMC_HWA_SetPmcConfiguration] </pre>	
Parameters	N/A	
Register Accessed	PMC_LVSCR(rw), PMC_CONFIG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_Hw_PmcDisable(void)	

[SWDESG_MCU_220] SWDESG_MCU_220 cover this item.

2.8.2.9 Functions in Mcu_Rgm.h

2.8.2.9.1 RGM_EnableInt

Function	void RGM_EnableInt(RGM_ResetDelayType eDelay, uint16 u16SysReset,const uint16* pCoreReset, \ RGM_ISRCallbackType* plsrNotify);
Description	This API can enable interrupt before a reset appears.

Diagram	<pre>graph LR; RGM_EnableInt --> RGM_HWA_EnableCore0ResetInterrupt; RGM_EnableInt --> RGM_HWA_EnableCore1ResetInterrupt; RGM_EnableInt --> RGM_HWA_EnableCore2ResetInterrupt; RGM_EnableInt --> RGM_HWA_EnableGlobalResetInterrupt; RGM_EnableInt --> RGM_HWA_EnableSysResetInterrupt; RGM_EnableInt --> RGM_HWA_SetResetDelay;</pre>											
Parameters	<table><tr><th>Parameter</th><th>Description</th></tr><tr><td>eDelay</td><td>Enumeration of delay cycles</td></tr><tr><td>u16SysReset</td><td>System reset event source</td></tr><tr><td>u16CoreReset</td><td>Core reset event source.</td></tr><tr><td>plsrNotify</td><td>Interrupt function.</td></tr></table>	Parameter	Description	eDelay	Enumeration of delay cycles	u16SysReset	System reset event source	u16CoreReset	Core reset event source.	plsrNotify	Interrupt function.	
Parameter	Description											
eDelay	Enumeration of delay cycles											
u16SysReset	System reset event source											
u16CoreReset	Core reset event source.											
plsrNotify	Interrupt function.											
Register Accessed	RGM_SRIE(rw), RGM_C0_CFG(rw), RGM_C1_CFG(rw), RGM_C2_CFG(rw), <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>											
Returns	N/A											
Referenced By	void Mcu_Hw_RgmInit(const Mcu_RGM_ConfigType *const pConfigPtr)											

[SWDESG_MCU_133] RGM_EnableInt function used to enable RGM interrupt functions.

2.8.2.9.2 RGM_GetLastResetFlag

Function	uint32 RGM_GetLastResetFlag (void)	
Description	This api can get RGM register that indicate the source of the most recent reset.	
Diagram	<pre> graph LR RGM_GetLastResetFlag --> RGM_HWA_ReadCore0SRS[RGM_HWA_ReadCore0SRS] RGM_GetLastResetFlag --> RGM_HWA_ReadCore1SRS[RGM_HWA_ReadCore1SRS] RGM_GetLastResetFlag --> RGM_HWA_ReadCore2SRS[RGM_HWA_ReadCore2SRS] RGM_GetLastResetFlag --> RGM_HWA_ReadSysSRS[RGM_HWA_ReadSysSRS] </pre>	
Parameters	N/A	
Register Accessed	RGM_SRS(r), RGM_C0_SSRS(rw), RGM_C1_SSRS(rw), RGM_C2_SSRS(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	Reset reason	
Referenced By	Mcu_ResetType Mcu_Hw_RgmGetResetReason(void)	

[SWDESG_MCU_134] RGM_GetLastResetFlag function used to get the recent reset flag.

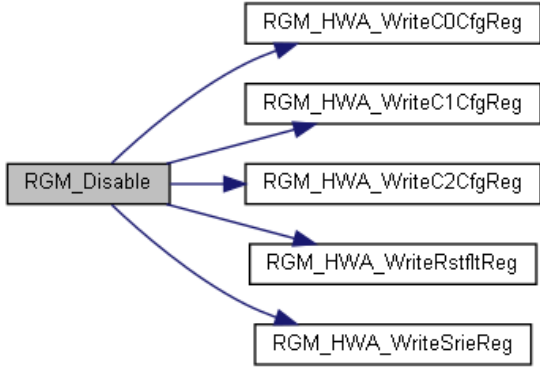
2.8.2.9.3 RGM_EnableResetFilter

Function	void RGM_EnableResetFilter(RGM_FilterClkSrc eClk,uint8 u8BusClockFilterWidth,Boolean bLpClkEn)
-----------------	--

Description	Enable reset pin filter.	
Diagram	 <pre> graph LR A[RGM_EnableResetFilter] --> B[RGM_HWA_WriteRstfltReg] </pre>	
Parameters	Parameter	Description
	eClk	Reset pin filter clock source
	u8BusClockFilterWidth	Bus clock filter width
	bLpClkEn	select AON clock at low power mode
Register Accessed	RGM_RSTFLT(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_Hw_RgmInit(const Mcu_RGM_ConfigType *const pConfigPtr)	

[SWDESG_MCU_135] RGM_EneableResetFilter function used to set reset pin filter configurations.

2.8.2.9.4 Rgm Disable

Function	void Pmc_Disable(void);	
Description	Disable reset generation module.	
Diagram	 <pre> graph LR A[RGM_Disable] --> B[RGM_HWA_WriteC0CfgReg] A --> C[RGM_HWA_WriteC1CfgReg] A --> D[RGM_HWA_WriteC2CfgReg] A --> E[RGM_HWA_WriteRstfltReg] A --> F[RGM_HWA_WriteSrieReg] </pre>	
Parameters	N/A	
Register Accessed	PMC_LVSCR(rw), PMC_CONFIG(rw) <i>Note : The list includes all the register accessed in the context of the API. It lists the register accessed by the driver and called interfaces from other drivers. During runtime, the register accessed from this list may vary based on configuration and execution context.</i>	
Returns	N/A	
Referenced By	void Mcu_Hw_RgmDisable(void)	

[SWDESG_MCU_222] SWDESG_MCU_222 cover this item.

2.8.3 Interrupt Functions

[SWDESG_MCU_170] MCU provides service to handle interrupt and user can config callback function which functions will be called during the routine of handle interrupt.

[SWDESG_MCU_136] User defined notification functions will always be called as function pointers.

2.8.3.1 ISR(Mcu_PMC_LVD_LVW_ISR)

Function	ISR(Mcu_PMC_LVD_LVW_ISR)
Description	PMC interrupt handler
Parameters	N/A

Returns	N/A
Referenced By	N/A
Requirements	FUNC_MCU_0012, FUNC_MCU_0013

[SWDESG_MCU_171] Mcu_PMC_LVD_LVW_ISR function used to handle PMC low power detection interrupt by call the function PMC_UserIRQHandler().

2.8.3.2 ISR(Mcu_CMU0_ISR)

Function	ISR(Mcu_CMU0_ISR)
Description	CMU0 interrupt handler
Parameters	N/A
Returns	N/A
Referenced By	N/A
Requirements	FUNC_MCU_0012, FUNC_MCU_0013

[SWDESG_MCU_172] Mcu_CMU0_ISR function used to handle CMU0 interrupt by call the function CMU_UserIRQHandler().

2.8.3.3 ISR(Mcu_CMU1_ISR)

Function	ISR(Mcu_CMU1_ISR)
Description	CMU1 interrupt handler
Parameters	N/A
Returns	N/A
Referenced By	N/A
Requirements	FUNC_MCU_0012, FUNC_MCU_0013

[SWDESG_MCU_173] Mcu_CMU1_ISR function used to handle CMU1 interrupt by call the function CMU_UserIRQHandler().

2.8.3.4 ISR(Mcu_CMU2_ISR)

Function	ISR(Mcu_CMU2_ISR)
Description	CMU2 interrupt handler
Parameters	N/A
Returns	N/A
Referenced By	N/A
Requirements	FUNC_MCU_0012, FUNC_MCU_0013

[SWDESG_MCU_174] Mcu_CMU2_ISR function used to handle CMU2 interrupt by call the function CMU_UserIRQHandler().

2.8.3.5 ISR(Mcu_CMU3_ISR)

Function	ISR(Mcu_CMU3_ISR)
Description	CMU3 interrupt handler
Parameters	N/A
Returns	N/A
Referenced By	N/A
Requirements	FUNC_MCU_0012, FUNC_MCU_0013

[SWDESG_MCU_175] Mcu_CMU3_ISR function used to handle CMU3 interrupt by call the function CMU_UserIRQHandler().

2.8.3.6 ISR(Mcu_CMU4_ISR)

Function	ISR(Mcu_CMU4_ISR)
Description	CMU4 interrupt handler
Parameters	N/A
Returns	N/A
Referenced By	N/A
Requirements	FUNC_MCU_0012, FUNC_MCU_0013

[SWDESG_MCU_237] Mcu_CMU4_ISR function used to handle CMU4 interrupt by call the function CMU_UserIRQHandler().

2.8.3.7 ISR(MCU_RGM_ISR)

Function	ISR(MCU_RGM_ISR)
Description	RGM interrupt handler
Parameters	N/A
Returns	N/A
Referenced By	N/A
Requirements	FUNC_MCU_0012, FUNC_MCU_0013

[SWDESG_MCU_176] Mcu_RGM_ISR function used to handle RGM interrupt by call the function RGM_UserIRQHandler().

2.8.3.8 ISR(MCU_SCG_ISR)

Function	ISR(MCU_SCG_ISR)
Description	SCG interrupt handler
Parameters	N/A
Returns	N/A
Referenced By	N/A
Requirements	FUNC_MCU_0012, FUNC_MCU_0013

[SWDESG_MCU_177] Mcu_SCG_ISR function used to handle SCG interrupt by call the function SCG_UserIRQHandler().

2.8.3.9 RGM_pInterruptFunc

Function	void (*RGM_pInterruptFunc)(uint32 u32SRS)	
Description	RGM user defined interrupt notification function.	
Parameters	Parameter	Description
	u32SRS	MCU last reset flag.
Returns	N/A	
Referenced By	RGM_UserIRQHandler	
Requirements	FUNC_MCU_0012, FUNC_MCU_0013	

[SWDESG_MCU_137] RGM_pInterruptFunc function pointer used to store the RGM callback function.

2.8.3.10 PMC_pInterruptNotify

Function	void (*PMC_pInterruptNotify)(uint32 u32Flag)	
Description	PMC user defined interrupt notification function.	
Parameters	Parameter	Description
	u32Flag	MCU LVD and HVD flag
Returns	N/A	
Referenced By	RGM_UserIRQHandler	
Requirements	FUNC_MCU_0012, FUNC_MCU_0013	

[SWDESG_MCU_138] PMC_pInterruptNotify function pointer used to store the PMC callback function.

2.8.3.11 CMU_pInterruptNotify

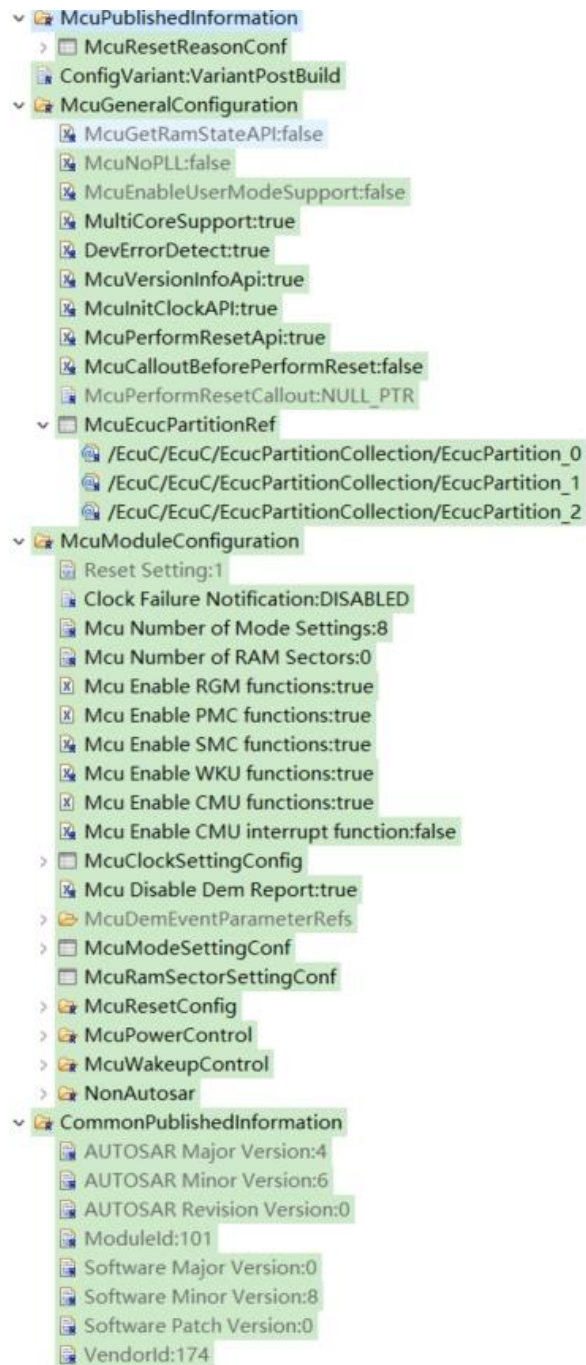
Function	void (*CMU_ISRCallbackType)(const CMU_InstanceType eInstance, const CMU_InterruptType eStatus);	
Description	CMU user defined interrupt notification function.	
Parameters	Parameter	Description
	eInstance	CMU module index.
	eStatus	CMU interrupt reason
Returns	N/A	
Referenced By	Mcu_CMU0_ISR/ Mcu_CMU1_ISR/ Mcu_CMU2_ISR/ Mcu_CMU3_ISR	
Requirements	FUNC_MCU_0012, FUNC_MCU_0013	

[SWDESG_MCU_139] CMU_ISRCallbackType function pointer used to store the CMU callback function.

Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation

The container inclusion relation is shown below:



[SWDESG_MCU_140] The variables predefined by the code generator (such as \$date, \$target, etc) shall not be overwritten.

3.2 Containers and Variables

[SWDESG_MCU_141] MCU module support all possible implementation configuration variants (pre-compile, link-time, post-build) specified in modules SWS document.

[SWDESG_MCU_142] If configuration influences data types or size of data fields, such shall be configurable in pre-compile variants only.

[SWDESG_MCU_143] If constraints checked are necessary for a successful code generation, a failed check shall raise an error. All other tests shall raise a warning.

Rationale: Modules may be stubbed or exist outside of the tool chain, so only produce an error, if the configuration is absolutely necessary.

[SWDESG_MCU_144] Every definition of configuration parameter holding a constant value shall have type INTEGER_LABEL, STRING_LABEL or FLOAT_LABEL.

Rationale: Values of constant configuration parameters must not be overwritten by a configuration.

[SWDESG_MCU_145] Every parameter that is used as user notification function name input shall allowed to enter only:

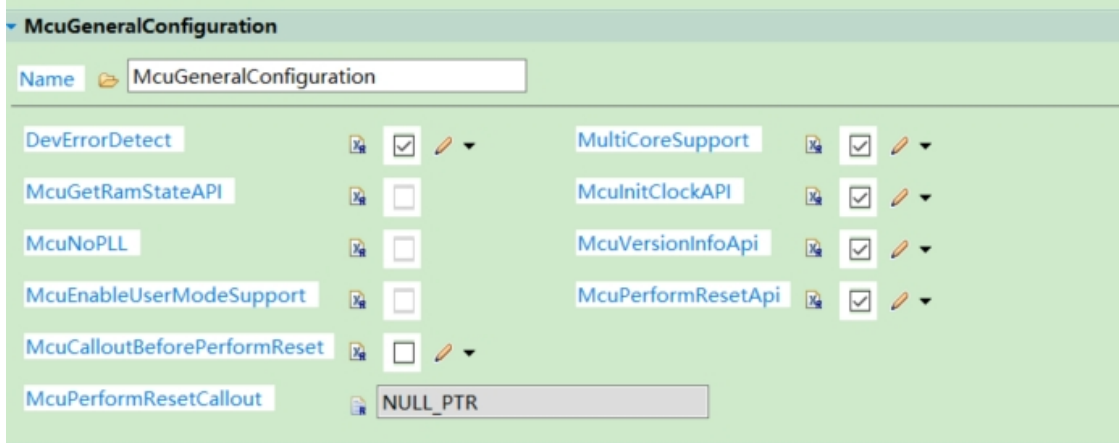
-NULL_PTR, NULL for no user notification function configured

-or c-function valid identifier only for user notification function configured

Default value shall be set to: NULL_PTR

[SWDESG_MCU_146] Every container and parameter shall contain a detailed description (the XML attribute called 'DESC').

3.2.1 McuGeneralConfiguration

Container	McuGeneralConfiguration	
Description	MCU General configuration container.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	SWR_ARCHI_049, ECUC_MCU_00118, FUNC_MCU_0021, FUNC_MCU_0025, FUNC_MCU_0026, FUNC_MCU_0027	

[SWDESG_MCU_186] This container contains the configuration (parameters) of the MCU driver.

3.2.1.1 DevErrorDetect

Variable	McuDevErrorDetect	
Description	Pre-processor switch for enabling the default error detection and reporting to the DET. The switch McuDevErrorDetect shall switch the Default Error Tracer (Det) detection and notification ON or OFF. The detection of default errors is configurable (ON/OFF) at precompile time. #define MCU_DEV_ERROR_DETECT (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00166	

[SWDESG_MCU_187] This Switches the Default Error Tracer (Det) detection and notification ON or OFF.

3.2.1.2 MultiCoreSupport

Variable	MultiCoreSupport	
Description	This parameter globally enables the possibility to support multicore.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC
Requirements	FUNC_MCU_0055	

[SWDESG_MCU_238] This Switches the MultiCoreSupport ON or OFF.

3.2.1.3 McuGetRamStateAPI

Variable	McuGetRamStateApi	
Description	Pre-processor switch to enable/disable the API Mcu_GetRamState. #define MCU_GET_RAM_STATE_API (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00181	

[SWDESG_MCU_147] Configuration variable McuGetRamStateApi is pre-compile only and generate macro to disable the access to the function.

3.2.1.4 McuInitClockAPI

Variable	McuInitClock	
Description	If this parameter is set to FALSE, the clock initialization has to be disabled from the MCU driver. This concept applies when there are some write once clock registers and a bootloader is present. If this parameter is set to TRUE, the MCU driver is responsible of the clock initialization #define MCU_INIT_CLOCK (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00182	

[SWDESG_MCU_148] Configuration variable McuInitClock is pre-compile only and generate macro to disable the access to the function.

3.2.1.5 McuNoPLL

Variable	McuNoPll	
Description	This parameter shall be set True, if the H/W does not have a PLL or the PLL circuitry is enabled after the power on without S/W intervention. In this case MCU_DistributePllClock has to be disabled and MCU_GetPllStatus has to return MCU_PLL_STATUS_UNDEFINED. Otherwise, this parameters has to be set False. #define MCU_NO_PLL (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	

Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00180	

[SWDESG_MCU_188] This parameter shall be set True, if the H/W does not have a PLL or the PLL circuitry is enabled after the power on without S/W intervention. In this case MCU_DistributePllClock has to be disabled and MCU_GetPllStatus has to return MCU_PLL_STATUS_UNDEFINED. Otherwise this parameters has to be set False.

3.2.1.6 McuVersionInfoApi

Variable	McuVersionInfoApi	
Description	Pre-processor switch to enable/disable the API to read out the modules version information. #define MCU_VERSION_INFO_API (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00168	

[SWDESG_MCU_149] Configuration variable McuVersionInfoApi is pre-compile only and generate macro to disable the access to the function.

3.2.1.7 McuEnableUserModeSupport

Variable	McuEnableUserModeSupport	
Description	When this parameter is enabled, the MDL module will adapt to run from User Mode, with the following measures: a) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode. b) other module specific measures for more information, please see chapter 5.7 User Mode Support in IM Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0009	

[SWDESG_MCU_206] Any new functionality, that has been introduced due to a feature request, shall be optionally active. Per default this functionality shall be switched off and an integrator is able to enable that functionality.

3.2.1.8 McuPerformResetApi

Variable	McuPerformResetApi	
Description	Pre-processor switch to enable/disable the use the Mcu_PerformReset() API. OFF - Mcu_PerformReset() API is not used. ON - Mcu_PerformReset() API is used. #define MCU_PERFORM_RESET_API (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot		
Properties	Property	Value

	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00167	

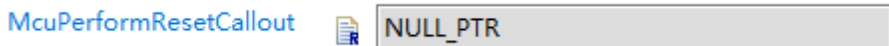
[SWDESG_MCU_150] Configuration variable McuPerformResetApi is pre-compile only and generate macro to disable the access to the function.

3.2.1.9 McuCalloutBeforePerformReset

Variable	McuCalloutBeforePerformReset	
Description	Check this if you want a callout function, called by MCU right before Mcu_PerformReset(). This parameter is available for configuration only if "McuPerformResetApi" is ON. #define MCU_RESET_CALLOUT_USED (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0050	

[SWDESG_MCU_217] This container contains the configuration (parameters) for reset reason settings.

3.2.1.10 McuPerformResetCallout

Variable	McuPerformResetCallout	
Description	Function name of callout. The field is editable only if "McuCalloutBeforePerformReset" is ON. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip
Requirements	FUNC_MCU_0050	

[SWDESG_MCU_150] SWDESG_MCU_150 cover this item.

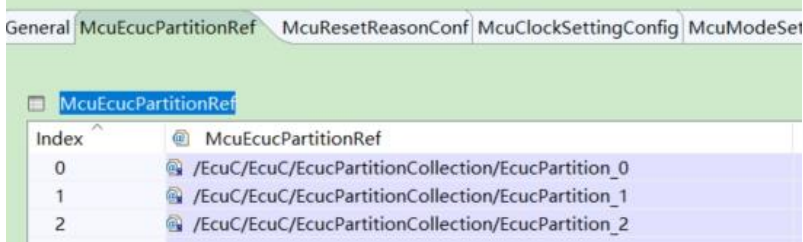
3.2.2 McuPublishedInformation

Container	McuPublishedInformation	
Description	Container holding all MCU specific published information parameters.	
Screenshot	N/A	
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	ECUC_MCU_00184	

[SWDESG_MCU_203] Container holding all MCU specific published information parameters.

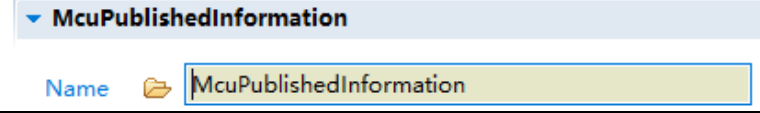
3.2.3 McuEcucPartitionRef

Container	McuEcucPartitionRef	
Description	Container maps the MCU driver to zero or multiple ECUC partition to make the driver API available in this partition.	

Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	ECUC_MCU_00191	

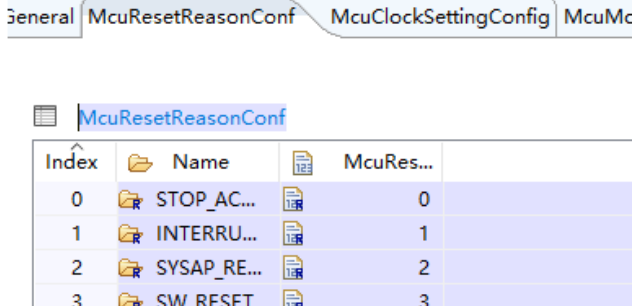
[SWDESG_MCU_239] Container maps the MCU driver to zero or multiple ECUC partition to make the driver API available in this partition.

3.2.4 McuResetReasonConf

Container	McuResetReasonConf	
Description	This container contains the configuration for the different type of reset reason that can be retrieved from Mcu_GetResetReason Api.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	ECUC_MCU_00185	

[SWDESG_MCU_204] This container contains the configuration for the different type of reset reason that can be retrieved from Mcu_GetResetReason Api.

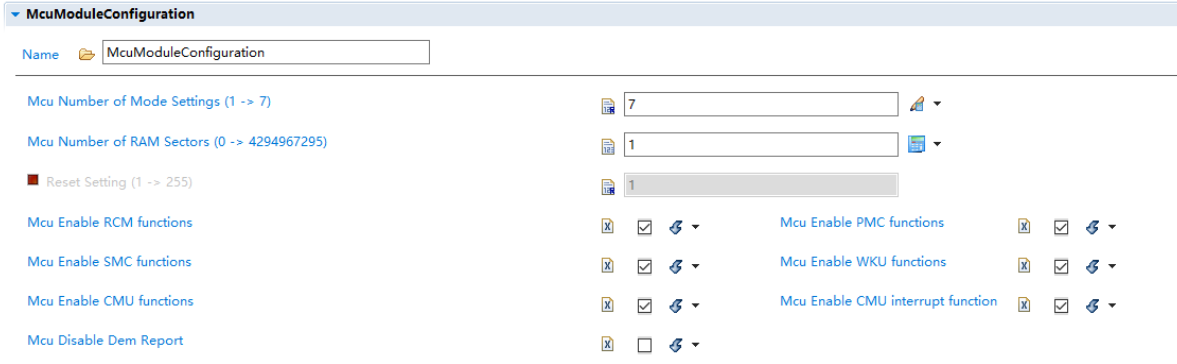
3.2.4.1 McuResetReason

Variable	McuResetReason	
Description	The parameter represents the different type of reset that a Micro supports. This parameter is referenced by the parameter EcuMResetReason in the ECU State manager module.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	true
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00186	

[SWDESG_MCU_205] The parameter represents the different type of reset that a Micro supports. This parameter is referenced by the parameter EcuMResetReason in the ECU State manager module.

3.2.5 McuModuleConfiguration

Container	McuModuleConfiguration	
Description	This container contains the configuration for the MCU driver.	

Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	ECUC_MCU_00119	

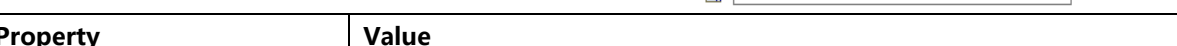
[SWDESG_MCU_151] The new functionality of MCU module like CMU function is enabled only after the Mcu Enable CMU functions is enable, then the related API is able to access. [FUNC_MCU_0009]

3.2.5.1 Clock Failure Notification

Variable	McuClockSrcFailureNotification	
Description	Enables/Disables clock failure notification.	
	In case this feature is not supported by HW the setting should be disabled.	
	Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00170	

[SWDESG_MCU_152] McuClockSrcFailureNotificaiton provide a configurable user callback for clock error is detected.

3.2.5.2 Mcu Number of Mode Settings

Variable	McuNumberOfMcuModes	
Description	This parameter shall represent the number of Modes available for the MCU (from "McuModeSettingConf" list). CalculationFormula = Number of configured "McuModeSettingConf".	
	This parameter is not used.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00171	

[SWDESG_MCU_189] This parameter shall represent the number of Modes available for the MCU.

calculationFormula = Number of configured McuModeSettingConf.

3.2.5.3 Mcu Number of RAM Sectors

Variable	McuRamSectors	
Description	This parameter shall represent the number of RAM sectors available for the MCU (from "McuRamSectorSettingConf" list). CalculationFormula = Number of configured "McuRamSectorSettingConf". This parameter is not used.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00172	

[SWDESG_MCU_190] This parameter shall represent the number of RAM sectors available for the MCU.

calculationFormula = Number of configured McuRamSectorSettingConf.

3.2.5.4 Reset Setting

Variable	McuResetSetting	
Description	This parameter applies to the function Mcu_PerformReset(), which performs a microcontroller reset using the hardware feature of the microcontroller. Note: This parameter is not used by the current Implementation. Software Reset occurs when Mcu_PerformReset() function is called. This parameter is not used.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00173	

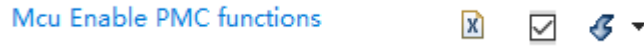
[SWDESG_MCU_191] This parameter relates to the MCU specific reset configuration. This applies to the function Mcu_PerformReset, which performs a microcontroller reset using the hardware feature of the microcontroller.

3.2.5.5 Mcu Enable RGM functions

Variable	McuEnableRcmInit	
Description	If this parameter is set to FALSE, the Reset Control Module (RGM) initialization has to be disabled in the MCU driver. If this parameter is set to TRUE, the MCU driver is responsible for the Reset Control Module (RGM) initialization. #define MCU_ENABLE_RGM (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0043, FUNC_MCU_0044	

[SWDESG_MCU_212] If this parameter is set to FALSE, the Reset Control Module (RGM) initialization has to be disabled in the MCU driver.

3.2.5.6 Mcu Enable PMC functions

Variable	McuDisablePmcInit	
Description	If this parameter is set to FALSE, the Power Management Controller (PMC) initialization has to be disabled in the MCU driver. If this parameter is set to TRUE, the MCU driver is responsible for the Power Management Controller (PMC) initialization. #define MCU_ENABLE_PMC (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0041, FUNC_MCU_0042	

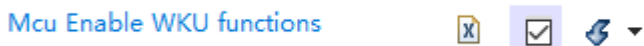
[SWDESG_MCU_211] If this parameter is set to FALSE, the Power Management Controller (PMC) initialization has to be disabled in the MCU driver.

3.2.5.7 Mcu Enable SMC functions

Variable	McuEnableSmcInit	
Description	If this parameter is set to FALSE, the System Mode Controller (SMC) initialization has to be disabled in the MCU driver. If this parameter is set to TRUE, the MCU driver is responsible for the System Mode Controller (SMC) initialization. #define MCU_ENABLE_SMC (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0039	

[SWDESG_MCU_210] If this parameter is set to FALSE, the System Mode Controller (SMC) initialization has to be disabled in the MCU driver.

3.2.5.8 Mcu Enable WKU functions

Variable	McuEnableWkuInit	
Description	If this parameter is set to FALSE, the Wakeup Unit (WKU) initialization has to be disabled in the MCU driver. If this parameter is set to TRUE, the MCU driver is responsible for the Wakeup Unit (WKU) initialization. #define MCU_ENABLE_WKU (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file. note: Must enable WKU PCC in McuClockSettingConfig/PCC.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false

	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0035, FUNC_MCU_0036	

[SWDESG_MCU_208] If this parameter is set to FALSE, the Wakeup Unit (WKU) initialization has to be disabled in the MCU driver.

3.2.5.9 Mcu Enable CMU functions

Variable	McuEnableCmulnit	
Description	If this parameter is set to FALSE, the Clock Monitor Unit (CMU) initialization has to be disabled in the MCU driver. If this parameter is set to TRUE, the MCU driver is responsible for the Clock Monitor Unit (CMU) initialization. #define MCU_ENABLE_CMU (STD_ON)/(STD_OFF) will be generated in Mcu_Cfg.h file.	
Screenshot	Mcu Enable CMU functions  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0037, FUNC_MCU_0038	

[SWDESG_MCU_209] If this parameter is set to FALSE, the Clock Monitor Unit (CMU) initialization has to be disabled in the MCU driver.

3.2.5.10 Mcu Enable CMU interrupt function

Variable	McuEnableCmulInterrupt	
Description	If this parameter is set to FALSE, disable CMU interrupt function If this parameter is set to TRUE, enable CMU interrupt function	
Screenshot	Mcu Enable CMU interrupt function   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0045	

[SWDESG_MCU_209] SWDESG_MCU_209 covers this item.

3.2.5.11 Mcu Disable Dem Report

Variable	McuDisableDemReportErrorStatus	
Description	Enable/Disable the API for Dem Error Report.	
Screenshot	Mcu Disable Dem Report   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0046	

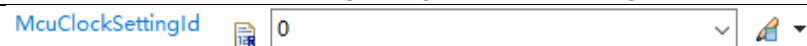
[SWDESG_MCU_213] The Macro of Mcu Disable Dem Report related to DEM module.

3.2.5.12 McuClockSettingConfig

Container	McuClockSettingConfig	
Description	This container contains the configuration for the Clock settings of the MCU.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	ECUC_MCU_00124	

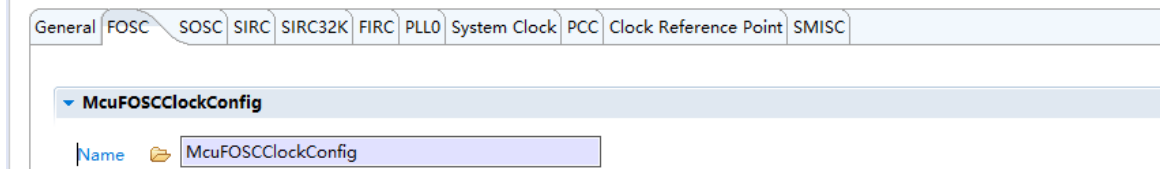
[SWDESG_MCU_192] This container contains the configuration (parameters) for the Clock settings of the MCU.

3.2.5.12.1 McuClockSettingId

Variable	McuClockSettingId	
Description	The Id of this McuClockSettingConfig to be used as argument for the API call Mcu_InitClock().	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	true
	Origin	AUTOSAR_ECUC
	Range	N/A
Requirements	ECUC_MCU_00183	

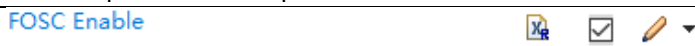
[SWDESG_MCU_193] The ID of this McuClockSettingConfig to be used as argument for the API call "Mcu_InitClock", and it contains all clock configuration.

3.2.5.12.2 McuFOSCClockConfig

Container	McuFOSCClockConfig	
Description	Configures Fast OSC registers. Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.1 FOSC Enable

Variable	McuFOSCEnable	
Description	SCG_FOSCCSR[FOSCEN] - Fast OSC Enable 0 - FAST OSC is disabled. 1 - FAST OSC is enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.2 FOSC Clock Monitor Enable

Variable	McuFOSCClockMonitorEnable	
Description	SCG_FOSCCSR[FOSCCM] - System OSC Clock Monitor Enables the clock monitor, if the clock source is disabled in a low power mode then the clock monitor is also disabled in the low power mode. When the clock monitor is disabled in a low power mode, it remains disabled until the clock valid flag is set following exit from the low power mode. 0 - System OSC Clock Monitor is disabled. 1 - System OSC Clock Monitor is enabled. Note: Implementation Specific Parameter.	
Screenshot	FOSC Clock Monitor Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.3 FOSC clock monitor reset Enable

Variable	McuFOSCClockMonitorResetEnable	
Description	SCG_FOSCCSR[FOSCCMRE] - System OSC Clock Monitor Reset Enable 0 - Clock Monitor generates interrupt when error detected. 1 - Clock Monitor generates reset when error detected. Note: Implementation Specific Parameter.	
Screenshot	FOSC clock monitor reset Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.4 FOSC Stop Enable

Variable	McuFOSCStopEnable	
Description	SCG_FOSCCSR[STEN] bit, Clock enable in Stop modes unchecked - FOSC disable in stop mode. checked - FOSC enable in stop mode Note: Implementation Specific Parameter.	
Screenshot	FOSC Stop Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.5 FOSC Bypass Enable

Variable	McuFOSCBypass	
Description	SCG_FOSCCFG[BYPASS] bit, Configures FOSC for bypassing the internal oscillator Fields COMP_EN and BYPASS specify the oscillator configuration Selects the source for the external reference clock. unchecked - Internal oscillator not bypassed. checked - Internal oscillator bypassed Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

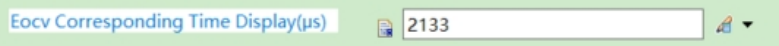
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.6 FOSC EOCV

Variable	FOSC EOCV	
Description	End of Count Value: Specifies the end of the count value. The oscillator stabilization counter uses EOCV for comparison when turning the clock on. This counting period ensures that the external oscillator clock signal is stable before the system selects the clock as a source. The counter runs on the crystal clock divided by 4 and counts up to $EOCV \times 256$. After the counting is finished, the FOSCCSR[VLD] bit will be set. EOCV can only be configured when the clock is disabled	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	50
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

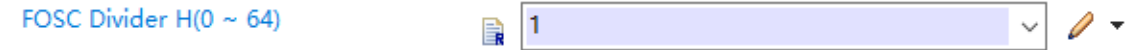
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.7 Eocv Corresponding Time

Variable	Eocv Corresponding Time	
Description	Display time that the oscillator stabilization calculated according to EOCV configuration. $T = 1/(f(\text{osc})/4) * EOCV \times 256$. After the counting is finished, the FOSCCSR[VLD] bit will be set. EOCV can only be configured when the clock is disabled.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2133
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

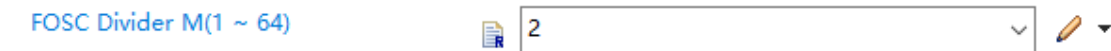
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.8 FOSC Divider H(0 ~ 64)

Variable	McuFOSCDIVH	
Description	Configure FOSC DIVH clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

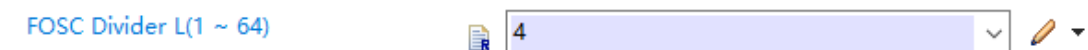
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.9 FOSC Divider M(1 ~ 64)

Variable	McuFOSCDIVM	
Description	Configure FOSC DIVM clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

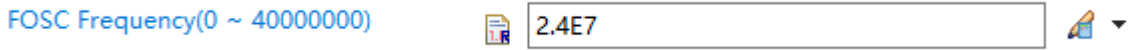
3.2.5.12.2.10 FOSC Divider L(1 ~ 64)

Variable	McuFOSCDIVL	
Description	Configure FOSC DIVL clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

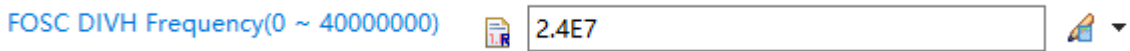
3.2.5.12.2.11 FOSC Frequency(0 ~ 40000000)

Variable	McuFOSCFrequency
Description	This is the FOSC frequency and it is expressed in Hz. Note: Implementation Specific Parameter.

Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

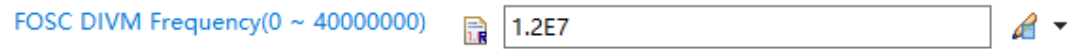
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.12 FOSC DIVH Frequency(0 ~ 40000000)

Variable	McuFOSCDIVHFrequency	
Description	This is the FOSC Divider H frequency and it is expressed in Hz. This parameter should be configured less than 40MHZ Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

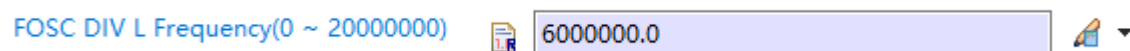
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.13 FOSC DIVM Frequency(0 ~ 40000000)

Variable	McuFOSCDIVMFrequency	
Description	This is the FOSC Divider M frequency and it is expressed in Hz. This parameter should be configured less than 40MHZ Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

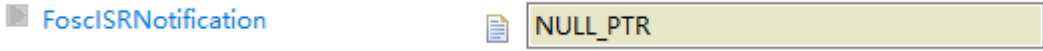
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.14 FOSC DIV L Frequency(0 ~ 20000000)

Variable	McuFOSCDIVLFrequency	
Description	This is the FOSC Divider L frequency and it is expressed in Hz. This parameter should be configured less than 20MHZ Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

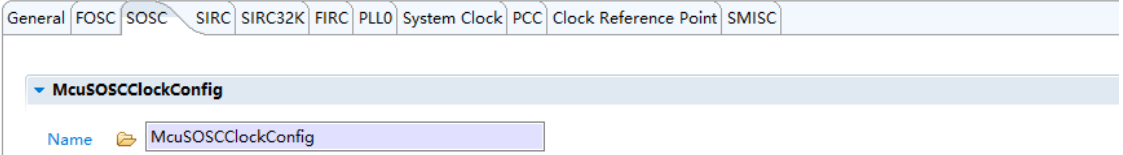
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.2.15 FosclSRNotification

Variable	FosclSRNotification	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3 McuSOSCClockConfig

Container	McuSOSCClockConfig	
Description	Configures Slow OSC registers. Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3.1 SOSC Enable

Variable	McuSOSCEnable	
Description	SCG_SOSCCSR[EN] bit, SOSC clock enable 0 - Slow OSC clock is disabled. 1 - Slow OSC 32k clock is enabled. Note: Implementation Specific Parameter. This is needed for switching system clock.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3.2 SOSC Bypass

Variable	McuSOSCBypass	
Description	SCG_SOSCCSR[BYPASS] bit, Configures SOSC to bypass mode. 0 - Internal oscillator not bypassed. 1 - Internal oscillator bypassed. Note: Implementation Specific Parameter.	

	This is needed for switching system clock.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

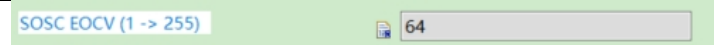
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3.3 SOSC Init No Wait

Variable	SOSC Init No Wait	
Description	Configures waiting valid flag set or not when SOSC initialize. 0 - Wait until valid. 1 - Do not wait. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3.4 SOSC EOCV

Variable	SOSC EOCV	
Description	End of Count Value: Specify the end of the count value. This bit field can only be reset by POR. This value is used by the oscillator Stabilization counter for comparison whenever it is switched on. The oscillator counter runs on a crystal clock and counts values up to $EOCV \times 256$. After the counting is finished, the SOSCCSR[VLD] bit will be set. EOCV can only be configured when the clock is disabled.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	64
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3.5 Eocv Corresponding Time Display

Variable	Eocv Corresponding Time Display	
Description	Display time that the oscillator stabilization calculated according to EOCV configuration. $T = 1/f(\text{osc}) * EOCV \times 256$. After the counting is finished, the SOSCCSR[VLD] bit will be set. EOCV can only be configured when the clock is disabled.	

Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	500000
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

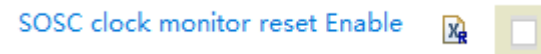
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3.6 Clock Monitor Enable

Variable	McuSOSCClockMonitorEnable	
Description	SCG_SOSCCSR[CM] bit,Clock Monitor enable. 0 - Slow OSC clock monitor is disabled. 1 - Slow OSC clock monitor is enabled. Note: Implementation Specific Parameter. This is needed for switching system clock.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

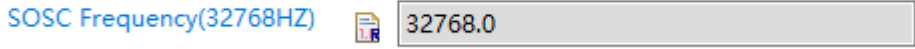
3.2.5.12.3.7 SOSC clock monitor reset Enable

Variable	McuSOSCClockMonitorResetEnable	
Description	SCG_SOSCCSR[CMRE] bit, 1:Clock Monitor Reset, 0: interrupt 0 - Trigger interrupt if clock monitor issue occur. 1 - Trigger Clock Monitor Reset. Note: Implementation Specific Parameter. This is needed for switching system clock.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3.8 SOSC Frequency

Variable	McuSOSCFrequency	
Description	This is the SOSC frequency and it is expressed in Hz. This parameter should be configured to 32768HZ Note: Implementation Specific Parameter.	

Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

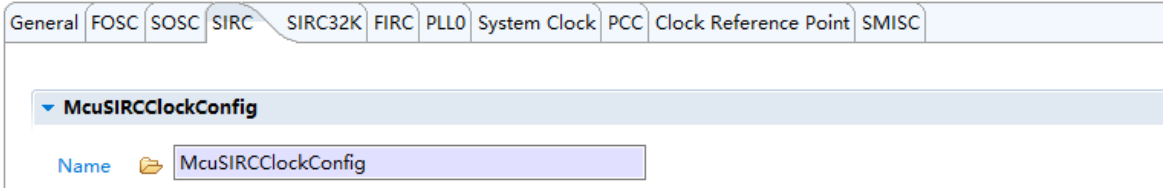
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.3.9 SoscISRNotification

Variable	SoscISRNotification	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4 McuSIRCClockConfig

Container	McuSIRCClockConfig	
Description	Configures Slow IRC (SIRC) registers. Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.1 SIRC CLock Monitor

Variable	McuSIRCCLOCKMONITOR	
Description	SCG_SIRCCSR[CM] bit, SIRC Clock Monitor Enable 0 - Slow IRC clock monitor is disabled. 1 - Slow IRC clock monitor is enabled. Note: Implementation Specific Parameter. This is needed for switching system clock.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.2 SIRC CLock Trim Enable

Variable	McuSIRCLKTRIMENABLE	
Description	SCG_SIRCCSR[TREN] bit, IRC software trim enable (auto trim) 0 - Slow IRC clock trim is disabled. 1 - Slow IRC clock trim is enabled. Note: Implementation Specific Parameter. . This is needed for switching system clock.	
Screenshot	SIRC CLock Trim Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.3 SIRC Low Power Enable

Variable	McuSIRCLowPowerEnable	
Description	SCG_SIRCCSR[SIRCLPEN] - Clock Standby Enable 0 - Slow IRC is disabled in standby modes. 1 - Slow IRC is enabled in standby modes. Note: Implementation Specific Parameter.	
Screenshot	SIRC Low Power Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

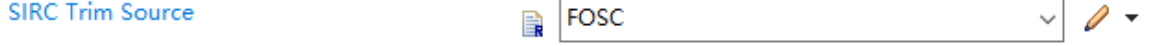
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.4 SIRC Stop Enable

Variable	McuSIRCStopEnable	
Description	SCG_SIRCCSR[SIRCASTEN] - Slow IRC Enable 0 - Slow IRC is disabled. 1 - Slow IRC is enabled. Note: Implementation Specific Parameter.	
Screenshot	SIRC Stop Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.5 SIRC Trim Source

Variable	McuSIRCTrimSrc	
Description	SCG_SIRCTCFG[TRIMSRC] IRC clock auto trim reference clock source select. IRC clock auto trim reference clock source select. FOSC SOSC Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	FOSC
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.6 SIRC Divider H

Variable	McuSIRCDIVH	
Description	Configure SIRC DIVH clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.7 SIRC Divider M

Variable	McuSIRCDIVM	
Description	Configure SIRC DIVM clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

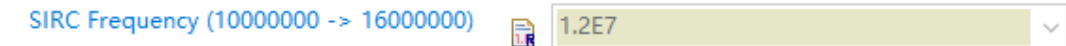
3.2.5.12.4.8 SIRC Divider L

Variable	McuSIRCDIVL
-----------------	-------------

Description	Configure SIRC DIVL clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

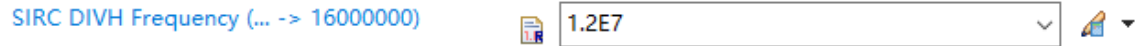
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.9 SIRC Frequency (10000000 -> 16000000)

Variable	McuSIRCFrequency	
Description	This is the SIRC frequency and it is expressed in Hz. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.10 SIRC DIVH Frequency

Variable	McuSIRCDIVHFrequency	
Description	This is the SIRC Divider H frequency and it is expressed in Hz. This parameter should be configured less than 12MHZ	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.11 SIRC DIVM Frequency

Variable	McuSIRCDIVMFrequency	
Description	This is the SIRC Divider M frequency and it is expressed in Hz. This parameter should be configured less than 12MHZ Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

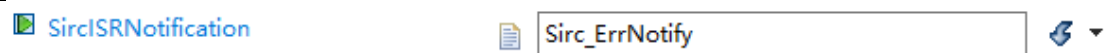
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.12 SIRC DIVL Frequency

Variable	McuSIRCDIVLFrequency	
Description	This is the SIRC Divider L frequency and it is expressed in Hz. This parameter should be configured less than 12MHZ Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

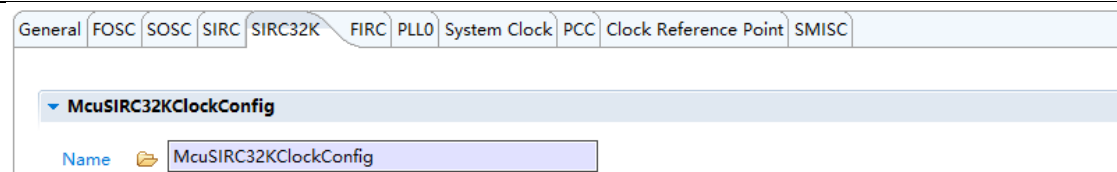
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.4.13 SirclSRNotification

Variable	SirclSRNotification	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.5 McuSIRC32KClockConfig

Container	McuSIRC32KClockConfig	
Description	Configures Slow IRC 32K(SIRC32K) registers. Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.5.1 SIRC32K Enable

Variable	McuSIRC32KENABLE	
Description	SIRC32KCSR[EN] bit, Clock is enabled 0 - Slow IRC 32k clock is disabled. 1 - Slow IRC 32k clock is enabled. Note: Implementation Specific Parameter. This is needed for switching system clock.	
Screenshot		

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

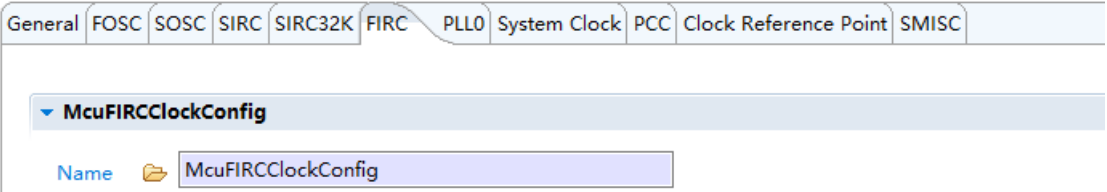
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.5.2 SIRC 32K Frequency

Variable	McuSIRC32KFrequency	
Description	This is the SIRC32K frequency and it is expressed in Hz. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6 McuFIRCClockConfig

Container	McuFIRCClockConfig	
Description	Configures Fast IRC (FIRC) registers. Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.1 FIRC Enable

Variable	McuFIRCEnable	
Description	SCG_FIRCCSR[FIRCEN] - Fast IRC Enable 0 - Fast IRC is disabled. 1 - Fast IRC is enabled. Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.2 FIRC Clock Monitor

Variable	McuFIRCCLOCKMONITOR	
Description	SCG_FIRCCSR[CM] bit, FIRC Clock Monitor Enable 0 - Fast IRC clock monitor is disabled. 1 - Fast IRC clock monitor is enabled. Note: Implementation Specific Parameter. This is needed for switching system clock.	
Screenshot	FIRC Clock Monitor   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.3 FIRC CLock Trim Enable

Variable	McuFIRCTRIMENABLE	
Description	SCG_FIRCCSR[TREN] bit, FIRC software trim enable (auto trim) 0 - Fast IRC clock trim is disabled. 1 - Fast IRC clock trim is enabled. Note: Implementation Specific Parameter. This is needed for switching system clock.	
Screenshot	FIRC CLock Trim Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.4 FIRC Stop Enable

Variable	McuFIRCStopEnable	
Description	SCG_FIRCCSR[STEN] bit, Clock enable in Stop modes unchecked - FIRC disable in stop mode. checked - FIRC enable in stop mode Note: Implementation Specific Parameter.	
Screenshot	FIRC Stop Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.5 FIRC Trim Source

Variable	McuFIRCTrimSrc	
Description	SCG_FIRCTCFG[TRIMSRC] IRC clock auto trim reference clock source select. IRC clock auto trim reference clock source select. FOSC SOSC Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	FOSC
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.6 FIRC Divider H(0 ~ 64)

Variable	McuFIRCDIVH	
Description	Configure FIRC DIVH clock source divider 0 - Output disabled. other valid value: 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.7 FIRC Divider M(0 ~ 64)

Variable	McuFIRCDIVM	
Description	Configure FIRC DIVM clock source divider 0 - Output disabled. other valid value: 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

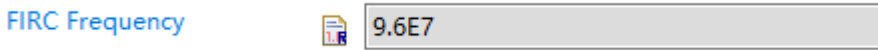
3.2.5.12.6.8 FIRC Divider L(0 ~ 64)

Variable	McuFIRCDIVL
-----------------	-------------

Description	Configure FIRC DIVL clock source divider 0 - Output disabled. other valid value: 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.9 FIRC Frequency

Variable	McuFIRCFrequency	
Description	This is the FIRC frequency and it is expressed in Hz. Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.10 FIRC DIVH Frequency

Variable	McuFIRCDIVHFrequency	
Description	This is the FIRC Divider H frequency and it is expressed in Hz. This parameter should be configured less than 96MHZ Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.11 FIRC DIVM Frequency

Variable	McuFIRCDIVMFrequency	
Description	This is the FIRC Divider M frequency and it is expressed in Hz. This parameter should be configured less than 48MHZ Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	FLOAT

	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

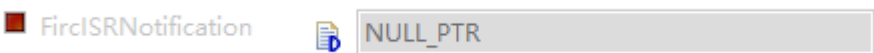
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.12 FIRC DIVL Frequency

Variable	McuFIRCDIVLFrequency	
Description	This is the FIRC Divider L frequency and it is expressed in Hz. This parameter should be configured less than 24MHZ Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

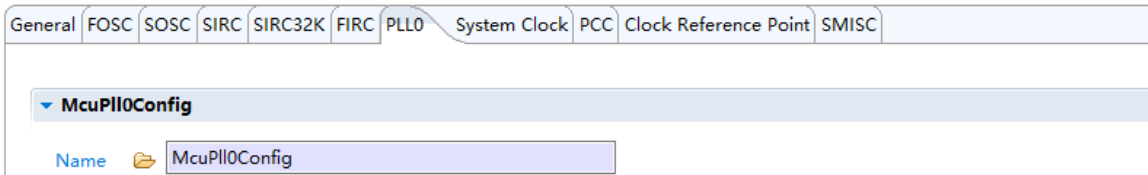
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.6.13 FirclSRNotification

Variable	FirclSRNotification	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7 McuPll0Config

Container	McuPll0Config	
Description	This container provides the specific configuration for the PLL0. Note: Implementation Specific Container.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.1 PLL0 Enable

Variable	McuPLL0Enable	
Description	SCG_PLL0CSR[PLL0EN] - PLL0 Enable 0 - PLL0 is disabled.	

	1 - PLL0 is enabled. Note: Implementation Specific Parameter, If PLL0 is disabled, the following configurations will be generated but not written to the register.	
Screenshot	PLL0 Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.2 PLL0 Clock Monitor Enable

Variable	McuPLL0ClockMonitorEnable	
Description	SCG_PLL0CSR[PLL0CM] - PLL0 Clock Monitor Enables the clock monitor, if the clock source is disabled in a low power mode then the clock monitor is also disabled in the low power mode. When the clock monitor is disabled in a low power mode, it remains disabled until the clock valid flag is set following exit from the low power mode. 0 - RTC OSC Clock Monitor is disabled. 1 - RTC OSC Clock Monitor is enabled. Note: Implementation Specific Parameter.	
Screenshot	PLL0 Clock Monitor Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.3 PLL0 Clock Monitor Reset Enable

Variable	McuPLL0ClockMonitorResetEnable	
Description	SCG_PLL0CSR[PLL0CMRE] - PLL0 Clock Monitor Reset Enable 0 - Clock Monitor generates interrupt when error detected. 1 - Clock Monitor generates reset when error detected. Note: Implementation Specific Parameter.	
Screenshot	PLL0 Clock Monitor Reset Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.4 PLL0 Stop Enable

Variable	McuPLL0StopEnable
Description	SCG_PLL0CSR[PLL0CMRE] - PLL0 Clock Monitor Reset Enable

	0 - Clock Monitor generates interrupt when error detected. 1 - Clock Monitor generates reset when error detected. Note: Implementation Specific Parameter.	
Screenshot	    	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.5 Mcu PLL0 Source Clock

Variable	McuPLL0SelectSourceClock	
Description	PLL0CFG[SOURCE]: PLL0 Clock Source. Configures the input clock source for the PLL0. Note: Implementation Specific Parameter.	
Screenshot	  FOSC  	
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	FOSC
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

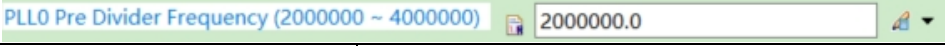
3.2.5.12.7.6 PLL0 Pre Divider

Variable	PLL0PreDivider	
Description	PLL Reference Clock Divider. Set the PLL0: SCG_PLL0CFG[PREDIV] field register. Selects the amount to divide down the reference clock for the PLL0. The resulting frequency must be in the range of 8 MHz to 50 MHz. Note: Implementation Specific Parameter.	
Screenshot	  12  	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	12
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.7 PLL0 Pre Divider Frequency

Variable	McuPLL0PreDivFrequency	
Description	Resulted frequency after predividing. $F = \text{PLL0_ReferenceClock} / \text{PLL0PreDivider.}$ Note: Implementation Specific Parameter.	

Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

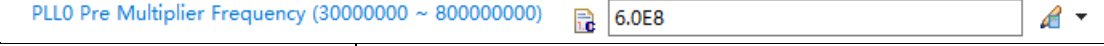
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.8 PLL0 Multiplier

Variable	McuPLL0Multiplier	
Description	PLL0 Multiplier. Set the SCG_PLL0CFG[MULT] field register. Valid range is in [16..47].Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	300
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.9 PLL0 Pre MultiplierFrequency

Variable	McuPLL0PreMultiplierFrequency	
Description	This is the PLL0 Pre Multiplier frequency and it is expressed in Hz. Note: McuPLL0PreDivFrequency = McuPLL0PreDivFrequency * McuPLL0Multiplier Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.10 PLL0 Post Divider

Variable	PLL0PostDivider	
Description	PLL Reference Clock Post Divider. Set the PLL0: SCG_PLL0CFG[PSDIV] field register. Selects the amount to divide down the reference clock for the PLL0. The Divider must be 2/4/8. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip

Requirements	FUNC_MCU_0051
---------------------	---------------

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.11 PLL0 Frequency

Variable	McuPLL0Frequency	
Description	FC7300FXXX:150 ~ 300 MHz FC7240FXXX:120 ~ 320 MHz This is the PLL0 frequency and it is expressed in Hz. Note: $PLL_freq = (McuPLL0Multiplier / McuPLL0PostDivider)$ Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.12 PLL0 Divider H

Variable	McuPLL0DIVH	
Description	Configure PLL0 DIVH clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.13 PLL0 Divider M

Variable	McuPLL0DIVM	
Description	Configure PLL0 DIVM clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

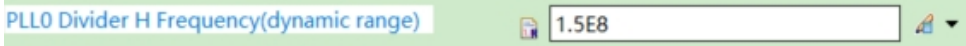
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.14 PLL0 Divider L

Variable	McuPLL0DIVL	
Description	Configure PLL0 DIVL clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

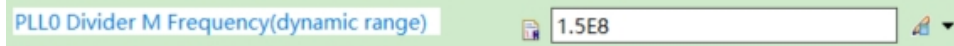
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.15 PLL0 Divider H Frequency

Variable	McuPLL0DIVHFrequency	
Description	FC7300FXXX: Max 150 MHz FC7240FXXX: Max 120 MHz This is the PLL0 Divider H frequency and it is expressed in Hz. This parameter should be configured less than Max value Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.16 PLL0 Divider M Frequency(0 ~ 75000000)

Variable	McuPLL0DIVMFrequency	
Description	FC7300FXXX: Max 150 MHz FC7240FXXX: Max 120 MHz This is the PLL0 Divider H frequency and it is expressed in Hz. This parameter should be configured less than Max value Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.17 PLL0 Divider L Frequency

Variable	McuPLL0DIVLFrequency	
Description	FC7300FXXX: Max 75 MHz FC7240FXXX: Max 60 MHz	

	This is the PLL0 Divider H frequency and it is expressed in Hz. This parameter should be configured less than Max value Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.18 PII0ISRNotification

Variable	PII0ISRNotification	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.19 PLL1 Enable

Variable	McuPLL1Enable	
Description	SCG_PLL1CSR[PLL0EN] - PLL0 Enable 0 – PLL1 is disabled. 1 – PLL1 is enabled. Note: Implementation Specific Parameter, If Pll1 is disabled, the following configurations will be generated but not written to the register.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.20 PLL1 Clock Monitor Enable

Variable	McuPLL1ClockMonitorEnable	
Description	SCG_PLL1CSR[PLL0CM] - PLL0 Clock Monitor Enables the clock monitor, if the clock source is disabled in a low power mode then the clock monitor is also disabled in the low power mode. When the clock monitor is disabled in a low power mode, it remains disabled until the clock valid flag is set following exit from the low power mode. 0 - Clock Monitor is disabled. 1 - Clock Monitor is enabled.	

	Note: Implementation Specific Parameter.	
Screenshot	PLL1 Clock Monitor Enable  ☐ 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.21 PLL1 Clock Monitor Reset Enable

Variable	McuPLL1ClockMonitorResetEnable	
Description	SCG_PLL1CSR[PLL0CMRE] – PLL1 Clock Monitor Reset Enable 0 - Clock Monitor generates interrupt when error detected. 1 - Clock Monitor generates reset when error detected. Note: Implementation Specific Parameter.	
Screenshot	PLL1 Clock Monitor Enable  ☐ 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.22 PLL1 Stop Enable

Variable	McuPLL1StopEnable	
Description	SCG_PLL1CSR[PLL0CMRE] – PLL1 Clock Monitor Reset Enable 0 - Clock Monitor generates interrupt when error detected. 1 - Clock Monitor generates reset when error detected. Note: Implementation Specific Parameter.	
Screenshot	PLL1 Stop Enable  ☐ 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.23 Mcu PLL1 Source Clock

Variable	McuPLL1SelectSourceClock	
Description	PLL1CFG[SOURCE]: PLL1 Clock Source. Configures the input clock source for the PLL1. Note: Implementation Specific Parameter.	
Screenshot	Mcu PLL1 Source Clock  FOSC 	

Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	FOSC
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.24 PLL1 Pre Divider

Variable	PLL1PreDivider	
Description	PLL Reference Clock Divider. Set the PLL1: SCG_PLL1CFG[PREDIV] field register. Selects the amount to divide down the reference clock for the PLL1. The resulting frequency must be in the range of 8 MHz to 50 MHz. Note: Implementation Specific Parameter.	
Screenshot	PLL1 Pre Divider(1 ~ 32) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	12
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.25 PLL1 Pre Divider Frequency

Variable	McuPLL1PreDivFrequency	
Description	Resulted frequency after prediving. $F = \text{PLL1_ReferenceClock} / \text{PLL1PreDivider}$. Note: Implementation Specific Parameter.	
Screenshot	PLL1 Pre Divider Frequency (2000000 ~ 4000000) 	
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.26 PLL1 Multiplier

Variable	McuPLL0Multiplier	
Description	PLL1 Multiplier. Set the SCG_PLL1CFG[MULT] field register. Valid range is in [96.512].Note: Implementation Specific Parameter.	
Screenshot	PLL1 Multiplier(96 ~ 512) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false

	Default	250
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.27 PLL1 Pre MultiplierFrequency

Variable	McuPLL1PreMultiplierFrequency	
Description	FC7300FXXX: 300 ~ 800 MHz FC7240FXXX: 240 ~ 640 MHz Resulted frequency after multiplier. $F = \text{McuPLL1PreDivFrequency} * \text{McuPLL1Multiplier}$. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.28 PLL1 Post Divider

Variable	PLL1PostDivider	
Description	PLL Reference Clock Post Divider. Set the PLL1: SCG_PLL1CFG[PSDIV] field register. Selects the amount to divide down the reference clock for the PLL1. The Divider must be 2/4/8. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.29 PLL1 Frequency

Variable	McuPLL1Frequency	
Description	FC7300FXXX: 150 ~ 300 MHz FC7240FXXX: 120 ~ 320 MHz This is the PLL1 frequency and it is expressed in Hz. Note: $\text{PLL_freq} = (\text{McuPLL1Multiplier} / \text{McuPLL1PostDivider})$ Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip

Requirements	FUNC_MCU_0051
---------------------	---------------

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.30 PLL1 Divider H

Variable	McuPLL1DIVH	
Description	Configure PLL1 DIVH clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot	PLL1 Divider H(0 ~ 64)  2 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.31 PLL1 Divider M

Variable	McuPLL1DIVM	
Description	Configure PLL1 DIVM clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot	PLL1 Divider M(0 ~ 64)  2 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.32 PLL1 Divider L

Variable	McuPLL1DIVL	
Description	Configure PLL1 DIVL clock source divider 0 - Output disabled. valid value: 0, 1, 2, 4, 8, 16, 32 and 64. Note: Implementation specific Parameter.	
Screenshot	PLL1 Divider L(0 ~ 64)  4 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.33 PLL1 Divider H Frequency

Variable	McuPLL1DIVHFrequency
-----------------	----------------------

Description	FC7300FXXX: Max 150 MHz FC7240FXXX: Max 160 MHz This is the PLL1 Divider H frequency and it is expressed in Hz. This parameter should be configured less than Max value Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.34 PLL1 Divider M Frequency

Variable	McuPLL1DIVMFrequency	
Description	FC7300FXXX: Max 150 MHz FC7240FXXX: Max 160 MHz This is the PLL1 Divider H frequency and it is expressed in Hz. This parameter should be configured less than Max value Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.7.35 PLL1 Divider L Frequency

Variable	McuPLL1DIVLFrequency	
Description	FC7300FXXX: Max 75 MHz FC7240FXXX: Max 80 MHz This is the PLL1 Divider H frequency and it is expressed in Hz. This parameter should be configured less than Max value Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

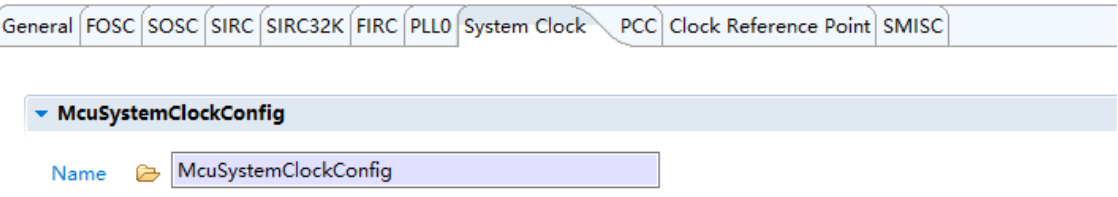
3.2.5.12.7.36 PII1ISRNotification

Variable	PII1ISRNotification
Description	Function pointer to callback function. Note: Implementation Specific Parameter.

Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8 McuSystemClockConfig

Container	McuSystemClockConfig	
Description	This container configures the system clock source and the system clock dividers for the core, platform, external and bus clock domains when in Run mode only. Note: Implementation specific Container.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.1 System Clock Select

Variable	McuSystemClockSwitch	
Description	Run System Clock Select. Switching to a different clock source requires that clock source to be enabled first and be valid before system clocks are allowed to switch to that clock source. Configure the SCG_CCR[SCS] register field. The system clock is either: - FOSC_CLK (SCG_CCR[SCS]=1) - SIRC_CLK (SCG_CCR[SCS]=2)(optional in FC7240FXXX) - FIRC_CLK (SCG_CCR[SCS]=3) - PLL0_CLK (SCG_CCR[SCS]=6)	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.2 Bus Clock Monitor

Variable	McuBusClockMonitor	
Description	SCG_CCR[SYSCLK_CME], System Clock monitor bit this field will generate reset if bus clock lose 0 - disable bus clock monitor. 1 - enable bus clock monitor. Note: Implementation Specific Parameter.	

Screenshot	Bus Clock Monitor 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.3 System Core Clock Divider

Variable	McuCoreClockDivider	
Description	Configure core clock divider.Note: implementation specific parameter.	
Screenshot	System Core Clock Divider (1 -> 16) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.4 System Bus Clock Divider

Variable	McuBusClockDivider	
Description	Configure bus clock divider.Note: implementation specific parameter.	
Screenshot	System Bus Clock Divider (1 -> 16) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.5 System Slow Clock Divider

Variable	McuSlowClockDivider	
Description	Configure slow clock divider.	
Screenshot	System Slow Clock Divider (1 -> 16) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.6 SCG ClkOut Select

Variable	McuScgClkOutSelect
-----------------	--------------------

Description	SCG_CLKOUTCNFG[CLKOUTSEL] - SCG Clkout Select.This register controls which SCG clock source is selected to be ported out to the CLKOUT pin. FOSC SIRC FIRC SOSC PLL0 SIRC32K Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	FOSC
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.7 NVM Clock Source Select

Variable	McuNVMClockSource	
Description	SCG_CLKOUTCFG [NVMCLK_FIRC_SIRC] - nvm program clock source. This register bit field controls nvm program clock frequency. 0 - NVMCLK_FIRC. 1 - NVMCLK_SIRC Note: Implementation Specific Parameter. Recommend use SIRC12M as flash clock	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	SIRC
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.8 CMU4 Clock Source Select

Variable	McuCMU4ClockSource	
Description	SCG_CLKOUTCFG [CMU4CLK_FOSC_SIRC] - nvm program clock source.This register bit field controls nvm program clock frequency. 0 - CMU4CLK_FOSC 1 - CMU4CLK_SIRC Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION

	SymbolicName	false
	Default	SIRC
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.9 System Clock Source Frequency

Variable	SystemClockSourceFrequency	
Description	FC7300FXXX: Max 300 MHz FC7240FXXX: Max 240 MHz Run Core clock - Pre Divide System Clock Frequency. This is the frequency for the specific instance of the "McuClockReferencePoint" container. Value calculated for user info. It is given in Hz. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.10 Core Clock Frequency

Variable	McuCoreClockFrequency	
Description	FC7300FXXX: Max 300 MHz FC7240FXXX: Max 240 MHz Run Core clock - Clocks the ARM core, divided by DIVCORE bits inside SCG This is the frequency for the specific instance of the "McuClockReferencePoint" container. Value calculated for user info. It is given in Hz. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.11 Bus Clock Frequency

Variable	McuBusClockFrequency	
Description	FC7300FXXX: Max 150 MHz FC7240FXXX: Max 120 MHz System Bus clock - BUS_CLK Clocks the Peripherals, divided by DIVBUS bits inside SCG This is the frequency for the specific instance of the "McuClockReferencePoint" container. Value calculated for user info. It is given in Hz. Note: Implementation Specific Parameter.	
Screenshot		

Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.12 Slow Clock Frequency

Variable	McuSlowClockFrequency	
Description	FC7300FXXX: Max 75 MHz FC7240FXXX: Max 60 MHz System Slow clock - Clocks the flash module, divided by DIVSLOW bits inside SCG This is the frequency for the specific instance of the "McuClockReferencePoint" container. Value calculated for user info. It is given in Hz. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.8.13 SCG ClockOut Frequency

Variable	McuScgClkOutFrequency	
Description	FC7300FXXX: Max 300 MHz FC7240FXXX: Max 320 MHz This is frequency of SCG clock out. It is given in Hz. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.9 McuPeripheralClockConfig

Container	McuPeripheralClockConfig
Description	This contains the combination for current peripheral in Run and LowPower Mode. Note: Implementation Specific Container.

Screenshot

GeneralFOSCOSCSIRCSIRC32KFIRCLL0System ClockPCCClock Reference PointSMISC

McuPeripheralClockConfig

Index	Name	Mcu Peri...	Enable p...	Peripher...	Peripher...	Peripher...
0	McuPerip...	DMA_MU...	☒	OFF	1	0.0
1	McuPerip...	FTU0	☒	FOSCDIVH	1	2.4E7
2	McuPerip...	ADC1	☒	FOSCDIVM	1	1.2E7
3	McuPerip...	FCIIC1	☒	FOSCDIVM	1	1.2E7
4	McuPerip...	FTU1	☒	FOSCDIVH	1	2.4E7
5	McuPerip...	FTU2	☒	PLL0DIVH	1	1.5E8
6	McuPerip...	ADC0	☒	FOSCDIVM	1	1.2E7
7	McuPerip...	RTC	☒	OFF	1	0.0
8	McuPerip...	PORTA	☒	OFF	1	0.0
9	McuPerip...	CMP0	☒	OFF	1	0.0
10	McuPerip...	CMP1	☒	OFF	1	0.0
11	McuPerip...	CMP2	☒	OFF	1	0.0
12	McuPerip...	FLEXCAN0	☒	FOSCDIVH	1	2.4E7
13	McuPerip...	FLEXCAN1	☒	SIRCDIVH	1	1.2E7
14	McuPerip...	FLEXCAN2	☒	SIRCDIVH	1	1.2E7
15	McuPerip...	FLEXCAN3	☒	FIRCDIVH	1	9.6E7
16	McuPerip...	FLEXCAN4	☒	FOSCDIVH	1	2.4E7
17	McuPerip...	FLEXCAN5	☒	FOSCDIVH	1	2.4E7
18	McuPerip...	PORTD	☒	OFF	1	0.0

Properties

Property	Value
Type	IDENTIFIABLE

Requirements

FUNC_MCU_0051

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.9.1 Mcu Peripheral Name

Variable	McuPerName	
Description	This is the name of the peripheral. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.9.2 Enable peripheral

Variable	McuPeripheralClockEnable	
Description	Sets PCC_[peripheral][CGC] bit. This read/write bit enables the clock for the peripheral. 0 - Clock disabled. 1 - Clock enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.9.3 Peripheral clock selection

Variable	McuPeripheralClockSelect	
Description	Configures PCC_[peripheral][PCS]. This is used for peripherals that support various clock selections. If the peripheral does not support various clock selections the field won't be editable. This field can only be written when the CGC bit is 0 (clock disabled). Likewise, if the INUSE flag is set, this field is locked. OFF/FTU_TCLK FOSCDIV(H/M) SIRCDIV(H/M) FIRCDIV(H/M) PLL0DIV(H/M) If the field is not editable, it means the bit field is read only and the value will not be used. NOTE: if current peripheral is FTU, clock select OFF means select FTU_TCLK which reference from pin and configured in SMISC register.	
Screenshot	Peripheral clock selection 	
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	OFF
	Origin	Flagchip
	Range	0 - OFF. 1 – FOSCDIVH/FOSCDIVM 2 – SIRCDIVH/SIRCDIVM 3 – FIRCDIVH/FIRCDIVM 6 - PLL0DIVH/PLL0DIVM
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.9.4 Peripheral Clock Divider

Variable	McuPeripheralClockDivider	
Description	Configures PCC_[peripheral][PCD]. This is used for peripherals that require a clock divider. At SOC integration, each peripheral is assigned either a divider or not. This field can only be written when the CGC bit is 0 (clock disabled). Likewise, if the INUSE flag is set, this field is locked. Allowed values are from 1 to 8. Note: Implementation Specific Parameter. If the field is not editable, it means the bit field is read only and the value will not be used.	
Screenshot	Peripheral Clock Divider (1 -> 8) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.9.5 Peripheral Clock Frequency (Hz)

Variable	McuPeripheralClockFrequency	
Description	Divider output clock = Divider input clock / DIV. This is only calculated if the clock source is selectable and if the peripheral is enabled. This is the frequency for the specific instance of the "McuClockReferencePoint" container.Value calculated for user info. It is given in Hz. Note: The maximum frequency of FCUARTx, FCSPIx, FCIICx, AONTIMER and FCPIT are governed by BUS_CLK, FTUx are governed by SYS_CLK, ADCx are 50MHz but always less than BUS_CLK. So please check configuration value is fit for SYS_CLK and BUS_CLK values correspond to MCU mode. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

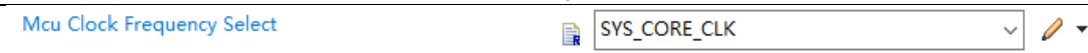
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.10McuClockReferencePoint

Container	McuClockReferencePoint																																																										
Description	This container defines a reference point in the Mcu Clock tree. It defines the frequency which then can be used by other modules as an input value. Lower multiplicity is 1, as even in the simplest case (only one frequency is used), there is one frequency to be defined.																																																										
Screenshot	GeneralFOSC SOSC SIRC SIRC32K FIRC PLL0 System Clock PCC Clock Reference Point SMISC McuClockReferencePoint <table><tr><th>Index</th><th>Name</th><th>Mcu Clock Frequenc...</th><th>Mcu Clo...</th></tr><tr><td>0</td><td>McuClock...</td><td>FTU4_CLK</td><td>2.4E7</td></tr><tr><td>1</td><td>McuClock...</td><td>SYS_CORE_CLK</td><td>1.5E8</td></tr><tr><td>2</td><td>McuClock...</td><td>SYS_BUS_CLK</td><td>7.5E7</td></tr><tr><td>3</td><td>McuClock...</td><td>ADC0_CLK</td><td>1.2E7</td></tr><tr><td>4</td><td>McuClock...</td><td>FCUART0_CLK</td><td>1.2E7</td></tr><tr><td>5</td><td>McuClock...</td><td>FCUART1_CLK</td><td>1.2E7</td></tr><tr><td>6</td><td>McuClock...</td><td>FCSPI1_CLK</td><td>1.2E7</td></tr><tr><td>7</td><td>McuClock...</td><td>FLEXCAN0_CLK</td><td>2.4E7</td></tr><tr><td>8</td><td>McuClock...</td><td>FLEXCAN2_CLK</td><td>1.2E7</td></tr><tr><td>9</td><td>McuClock...</td><td>FTU1_CLK</td><td>2.4E7</td></tr><tr><td>10</td><td>McuClock...</td><td>FCIIC0_CLK</td><td>1.2E7</td></tr><tr><td>11</td><td>McuClock...</td><td>FTU6_CLK</td><td>2.4E7</td></tr><tr><td>12</td><td>McuClock...</td><td>FCSPI0_CLK</td><td>1.2E7</td></tr></table>			Index	Name	Mcu Clock Frequenc...	Mcu Clo...	0	McuClock...	FTU4_CLK	2.4E7	1	McuClock...	SYS_CORE_CLK	1.5E8	2	McuClock...	SYS_BUS_CLK	7.5E7	3	McuClock...	ADC0_CLK	1.2E7	4	McuClock...	FCUART0_CLK	1.2E7	5	McuClock...	FCUART1_CLK	1.2E7	6	McuClock...	FCSPI1_CLK	1.2E7	7	McuClock...	FLEXCAN0_CLK	2.4E7	8	McuClock...	FLEXCAN2_CLK	1.2E7	9	McuClock...	FTU1_CLK	2.4E7	10	McuClock...	FCIIC0_CLK	1.2E7	11	McuClock...	FTU6_CLK	2.4E7	12	McuClock...	FCSPI0_CLK	1.2E7
Index	Name	Mcu Clock Frequenc...	Mcu Clo...																																																								
0	McuClock...	FTU4_CLK	2.4E7																																																								
1	McuClock...	SYS_CORE_CLK	1.5E8																																																								
2	McuClock...	SYS_BUS_CLK	7.5E7																																																								
3	McuClock...	ADC0_CLK	1.2E7																																																								
4	McuClock...	FCUART0_CLK	1.2E7																																																								
5	McuClock...	FCUART1_CLK	1.2E7																																																								
6	McuClock...	FCSPI1_CLK	1.2E7																																																								
7	McuClock...	FLEXCAN0_CLK	2.4E7																																																								
8	McuClock...	FLEXCAN2_CLK	1.2E7																																																								
9	McuClock...	FTU1_CLK	2.4E7																																																								
10	McuClock...	FCIIC0_CLK	1.2E7																																																								
11	McuClock...	FTU6_CLK	2.4E7																																																								
12	McuClock...	FCSPI0_CLK	1.2E7																																																								
Properties	Property	Value																																																									
	Type	IDENTIFIABLE																																																									
Requirements	ECUC_MCU_00174																																																										

[SWDESG_MCU_201] This container defines a reference point in the Mcu Clock tree. It defines the frequency which then can be used by other modules as an input value. Lower multiplicity is 1, as even in the simplest case (only one frequency is used), there is one frequency to be defined.

3.2.5.12.10.1 Mcu Clock Frequency Select

Variable	McuClockFrequencySelect	
Description	Select clock source for the specific instance of the McuClockReferencePoint container. Note: The clock frequency configured in McuPeripheralClockConfig should be used to export the clock frequency through McuClockReferencePoint. This reference point should be used in the configuration of the module that uses it (SPI, I2C, GPT, etc.). If the configured module has also an internal clock selection (like FlexTimer for example), the clock reference point should be configured taking the internal clock selection into account and the reference used should reflect the clock that finally enters the used peripheral.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	SYS_CORE_CLK
	Origin	Flagchip
	Range	Available configured clock source
Requirements	FUNC_MCU_0051	

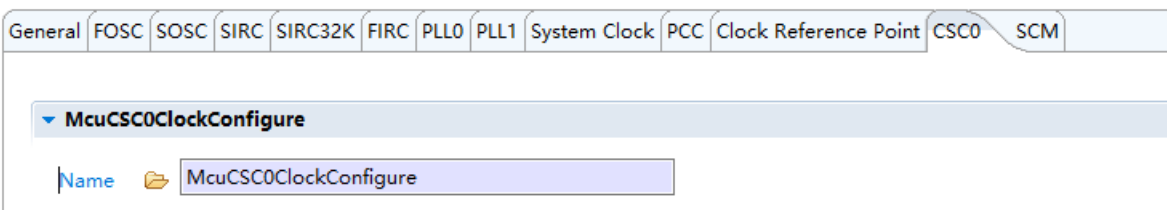
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.10.2 Mcu Clock Reference Point Frequency

Variable	McuClockReferencePointFrequency	
Description	This is the frequency for the specific instance of the McuClockReferencePoint container. It shall be given in Hz. Calculated value.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00175	

[SWDESG_MCU_202] This is the frequency for the specific instance of the McuClockReferencePoint container. It shall be given in Hz.

3.2.5.12.11 McuCSC0ClockConfigure

Container	McuCSC0ClockConfigure	
Description	This container contains the configuration for the CSC0 register.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12 McuSMCClockConfigure

Container	McuSMCClockConfigure	
Description	This container contains the configuration for the SMC register.	

Screenshot	GeneralFOSCOSCOSIRCSIRC32KFIRCPDLL0PLL1System ClockPCCClock Reference PointCSC0SMCSMCSCM ▼ McuSmcClockConfigure Name McuSmcClockConfigure				
	Properties	<table><tr><th>Property</th><th>Value</th></tr><tr><td>Type</td><td>IDENTIFIABLE</td></tr></table>	Property	Value	Type
Property	Value				
Type	IDENTIFIABLE				
Requirements	FUNC_MCU_0051				

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12.1 McuClkOutConfigure

Container	McuClkOutConfigure	
Description	This container contains the configuration for CSC0_CLKOUT_CTRL register.	
Screenshot	▼ McuClkOutConfigure Name McuClkOutConfigure	
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12.2 Enable CLKOUT

Variable	McuClockOutEnable	
Description	CSC0_CLKOUT_CTRL [CLKOUT_EN] - CLKOUT enable unchecked - Clockout disabled. checked - Clockout enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12.3 CLKOUT Divider

Variable	McuClockOutDivider	
Description	Configures the CSC0_CLKOUT_CTRL [CLKOUT_DIV] bit field CLKOUT Divide Ratio. CLKOUT Divide Range is 1~8. Note: implementation specific parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

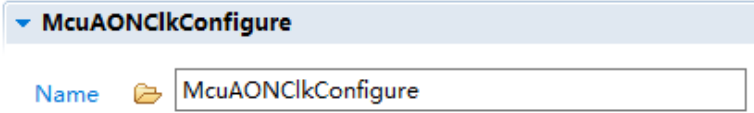
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12.4 CSC0 CLKOUT select

Variable	McuClockOutSelect	
Description	CSC0_CLKOUT_CTRL [CLKOUT_SEL] - CLKOUT select Selects the clock to output on the CLKOUT pin. SCG_CLKOUT FOSC DIVM CLK SIRC DIVM CLK FIRC DIVM CLK PLL0 DIVM CLK AON_CLK which selected by SMISC_AONCLKS[AONCLKSEL] AON32K_CLK which selected by SMISC_AONCLKS[RTCCLKSEL]	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12.5 McuAONClkConfigure

Container	McuAONClkConfigure	
Description	This container contains the configuration for AONCLKSR register.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

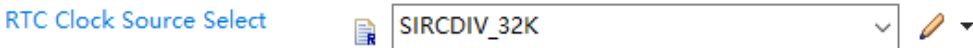
3.2.5.12.12.6 AON32K Clock Source Select

Variable	McuAon32kClkSelect	
Description	This is a write-once parameter SMISC_AONCLKSR[32KAONCLKSEL] - 32 KHz Always on clock source select. Selects Always on 32 kHz clock source for peripherals. 0 - disabled 1 - SIRCDIV_32K 2 - SOSC_32K 3 - SIRC_32K Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	SIRC32K

	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12.7 RTC Clock Source Select

Variable	McuRtcClkSelect	
Description	This is a write-once parameter SMISC_AONCLKS[RTCCLKSEL] - RTC clock source select. Selects RTC clock source for peripherals. 0 - FOSCDIVL 1 - SIRCDIV_32K 2 - SOSC 3 - SIRC32K Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	SIRCDIV_32K
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

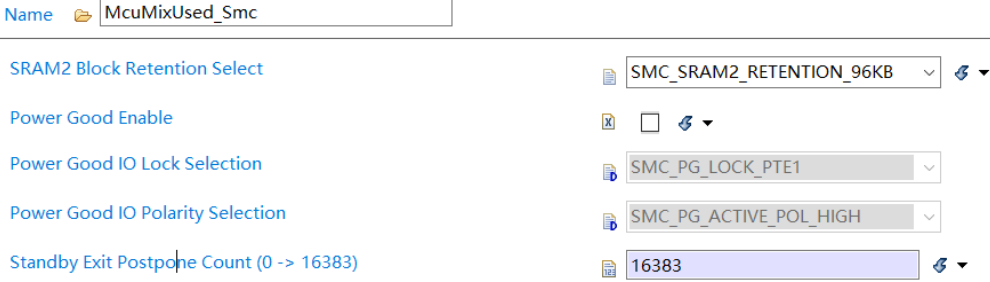
3.2.5.12.12.8 AON Clock Source Select

Variable	McuAONClockSelect	
Description	This is a write-once parameter SMISC_AONCLKS[AONCLKSEL] - AON clock source select Selects AON clock source for peripherals 0 - SIRCDIV_128K 1 - SIRC32K 2 - SIRCDIV_32K 3 - 1 kHz AON clock which is divided by the 128 kHz AON clock Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	SIRC32K
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

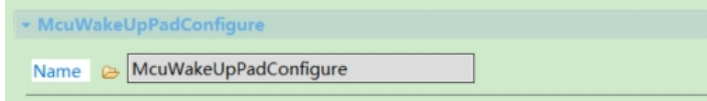
3.2.5.12.12.9 McuMixUsed_Smc

Variable	McuMixUsed_Smc
Description	This container contains the configuration for SMC mixed used register, including: 1. SRAM2 block retention select;

	- Power good enable - Power good IO lock selection - Power good IO polarity selection - Standby exit postpone count	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

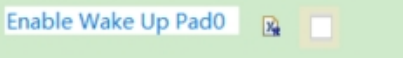
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12.10 McuWakeUpPadConfigure

Container	McuWakeUpPadConfigure	
Description	This container contains making of wakeup source sent to the LP_WAKEUP pad.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

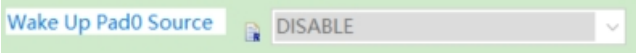
3.2.5.12.12.11 Enable Wake Up Pad0 ~ Enable Wake Up Pad4

Variable	Enable Wake Up Pad0 ~ Enable Wake Up Pad4	
Description	FC7300FXXX: Untouch FC7240FXXX: Optional unchecked - Wake up pad disabled. checked - Wake up pad enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

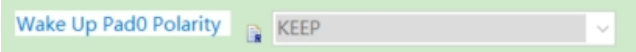
3.2.5.12.12.12 Wake Up Pad0 Source ~ Wake Up Pad4 Source

Variable	Wake Up Pad0 Source ~ Wake Up Pad4 Source	
Description	FC7300FXXX: Untouched FC7240FXXX: Optional Select wake up pad source.	

	DISABLE AONTIMER_TRIGGER RTC_ALARM CMP0_OUTPUT CMP1_OUTPUT PMC_RPM_ENTRY WKU_INTERRUPT TSTMP0_PWM_TRIGGER0 TSTMP0_PWM_TRIGGER1 FCPIT0_PWM_TRIGGER0 FCPIT0_PWM_TRIGGER1 FCPIT0_TRIGGER_OUT0 FCPIT0_TRIGGER_OUT1 FCPIT0_TRIGGER_OUT2 FCPIT0_TRIGGER_OUT3	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Default	DISABLE
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.12.13 Wake Up Pad0 Polarity ~ Wake Up Pad4 Polarity

Variable	Wake Up Pad0 Polarity ~ Wake Up Pad4 Polarity	
Description	FC7300FXXX: Untouched FC7240FXXX: Optional Wake up pad polarity configure KEEP INVERT	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Default	KEEP
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13 McuSCMClockConfigure

Container	McuCSC0ClockConfigure
Description	This container contains the configuration for the SCM register.

Screenshot	GeneralFOSC SOSC SIRC SIRC32K FIRC PLL0 PLL1 System Clock PCC Clock Reference Point CSC0 SCM ▼ McuSCMClockConfigure Name McuSCMClockConfigure				
	Properties	<table><tr><th>Property</th><th>Value</th></tr><tr><td>Type</td><td>IDENTIFIABLE</td></tr></table>	Property	Value	Type
Property	Value				
Type	IDENTIFIABLE				
Requirements	FUNC_MCU_0051				

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.1 McuTraceCLKDivConfiguration

Container	McuTraceCLKDivConfiguration		
Description	This container contains the configuration for the SMISC_TRACECLKDIV register.		
Screenshot	▼ McuTraceCLKDivConfiguration Name McuTraceCLKDivConfiguration		
Properties	Property	Value	
	Type	IDENTIFIABLE	
Requirements	FUNC_MCU_0051		

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.2 Debug Trace Divider Control

Variable	McuTraceDividerEnable		
Description	SMISC_TRACECLKDIV[TRACEDIVEN] - Debug Trace Divider Control 0 - Debug trace divider disabled. 1 - Debug trace divider enabled. Note: Implementation Specific Parameter.If Debug Trace is disabled, the following configurations will be generated but not written to the register.		
Screenshot	Debug Trace Divider Control 		
Properties	Property	Value	
	Type	BOOLEAN	
	SymbolicName	false	
	Default	false	
	Origin	Flagchip	
Requirements	FUNC_MCU_0051		

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

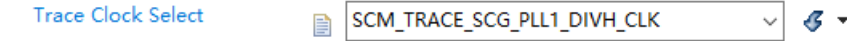
3.2.5.12.13.3 Trace Clock Divider(0 ~ 7)

Variable	McuTraceClockDivider										
Description	Configures the SMISC_TRACECLKDIV[TRACEDIV] bitfield trace clock divider divisor - Divider output clock = Core clock * [(TRACEFRAC+1)/(TRACEDIV+1)].At IO PAD side the value would divide by 2.Note: implementation specific parameter.										
Screenshot	Trace Clock Divider (1 -> 8) 										
Properties	Property	Value									
	Type	INTEGER									
	SymbolicName	false									
	Default	2									

	Origin	Flagchip
Requirements	FUNC_MCU_0051	

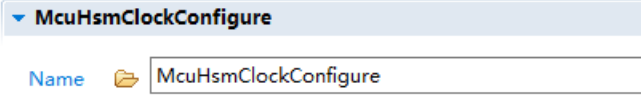
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.4 Trace Clock Select

Variable	McuTraceClockSelect	
Description	Trace Clock Select. 0 - Select the core clock as the trace clock 1 - Select SCG PLL1 DIVH as the trace clock	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	SCM_TRACE_CORE_CLK
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.5 McuHsmClockConfigure

Container	McuHsmClockConfigure	
Description	This container contains the configuration for SCM_HSM_PCC register.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.6 HSM Clock Enable

Variable	HsmClockEnable	
Description	SCM_HSM_PCC[CLKEN], Controls Hsm clock gating Controls the clock gating to the Hsm module. 0 - Clock disabled. 1 - Clock enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

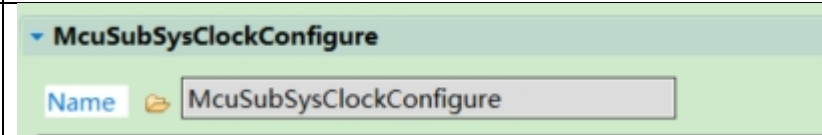
3.2.5.12.13.7 HSM WDG Reset Enable

Variable	HsmWdgRestEnable
-----------------	------------------

Description	SCM_HSM_PCC[WDG_RSTEN] - Hsm watchdog reset Control Controls the Hsm watchdog reset enable/disable. 0 - Disabled. 1 - Enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.8 McuSubSysClockConfigure

Container	McuSubSysClockConfigure	
Description	This container contains the configuration for SCM_SUBSYS_PCC register.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.9 TPU Clock Enable

Variable	TPU Clock Enable	
Description	FC7300FXXX: Untouch FC7240FXXX: Optional SCM_SUBSYS_PCC[CLKEN_TPU], Controls Tpu clock gating Controls the clock gating to the Tpu module. 0 - Clock disabled. 1 - Clock enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.10 HSM Clock Enable

Variable	HsmClockEnable	
Description	FC7300FXXX: Untouch FC7240FXXX: Optional	

	SCM_SUBSYS_PCC[CLKEN_HSM_ENGINE], Controls Hsm clock gating Controls the clock gating to the Hsm module. 0 - Clock disabled. 1 - Clock enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

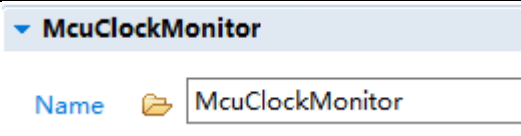
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.13.11 Subsystem WDG Reset Enable

Variable	Subsystem WDG Reset Enable	
Description	FC7300FXXX: Untouch FC7240FXXX: Optional SCM_SUBSYS_PCC[WDG_RSTEN] - Subsystem watchdog reset Control Controls the Subsystem watchdog reset enable/disable. 0 - Disabled. 1 - Enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

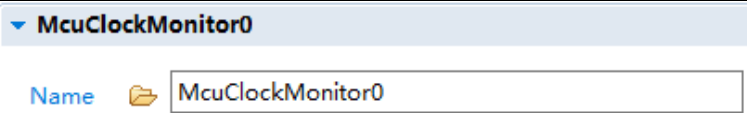
3.2.5.12.14 McuClockMonitor

Container	McuClockMonitor	
Description	The Reset Control Module (MC_CMU) centralizes the different reset sources and manages the reset sequence of the device. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1 McuClockMonitor0

Container	McuClockMonitor0
------------------	------------------

Description	User SIRC to monitor RTC_CLK. SIRC32_1K as period monitor source Make sure SIRC clock and RTC_clock valid before enable.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.1 CMU0 Enable

Variable	ClockMonitor0Enable	
Description	CMU_CTRL[ENABLE], This bit to enable CMU logic. 0 - RTC_CLK monitor disable. 1 - RTC_CLK monitor enabled. Note: Implementation Specific Parameter. except CTRL0 byte0, all others configured register should write when this bit de-asserted.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.2 Low Power Enable

Variable	LowPowerEnable	
Description	CMU_CTRL[LP_EN], Standby mode enable, need STOP_EN asserted. 0 - CMU disable in standby modes. 1 - CMU enable in standby modes. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.3 Stop Enable

Variable	StopEnable	
Description	CMU_CTRL[STOP_EN], Stop mode enable. 0 - CMU disable in stop mode. 1 - CMU enable in Stop mode. Note: Implementation Specific Parameter.	
Screenshot		

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.4 Enable CMU0 Interrupt

Variable	InterruptEnable	
Description	CMU_CTRL[IRQ_EN], Interrupt enable, if this bit asserted. Either LOC or MIS asserted can lead the interrupt. 0 - Interrupt disable. 1 - Interrupt enable. Note: Implementation Specific Parameter.	
Screenshot	Enable CMU0 Interrupt  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.5 Period Monitor Enable

Variable	PerMonitorEnable	
Description	CMU_PERIOD[EN] - Period Monitor mode enable. If this bit asserted, CMU would sample for WINDOW times. 0 - Period monitor is disabled. 1 - Period monitor is enabled. Note: if another period start point is happened during this time. the period monitor counter would clear and restarted.	
Screenshot	Period Monitor Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.6 Reference clock divider

Variable	RefDiv	
Description	The divider for the reference clock.	
Screenshot	Reference clock divider  	
Properties	Property	Value
	Type	Integer(Range)
	SymbolicName	false
	Default	1

	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.7 Minimum REF_WINDOW value(1 ~ 15978297)

Variable	MinRefWindow	
Description	Minimum REF_WINDOW: A lower value of REF_WINDOW results in a shorter checking window and faster response. Minimum REF_WINDOW = rounding up value of $6 + 5 * (f(\text{Reference clock}) / f(\text{Monitored clock}))$.	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	11
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.8 Maximum REF_WINDOW value(0 ~ 15978297)

Variable	MaxRefWindow	
Description	Maximum REF_WINDOW: A higher value of REF_WINDOW results in a longer checking window leading to better accuracy in monitored clock measurement. Maximum REF_WINDOW = MIN value (rounding down) of $(0\text{xfffff} \times (f(\text{Reference clock}) / f(\text{monitor clock}) / 2^{\text{REF_DIV}})$, and 0xfffff	
Screenshot		
Properties	Property	Value
	Type	Float
	SymbolicName	false
	Default	15978297
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.9 REF_WINDOW value

Variable	RefWindow	
Description	Please input a number between Minimum REF_WINDOW value and Maximum REF_WINDOW value Set REF_WINDOW, The Min and Max values are limited values, but in real situations,from the perspective of measurement accuracy and event response time,it is recommended to use a typical value between Min and Max,like Minimum REF_WINDOW value x 100 and Maximum REF_WINDOW value /100, etc.note: If Period Monitor Window function is enabled, the REF_WINDOW*10/(reference clock) less than 1ms	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1100
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.10 Clock deviation

Variable	Clock deviation	
Description	Clock deviation percentage	
Screenshot		
Properties	Property	Value
	Type	Float
	SymbolicName	false
	Default	5.0
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.11 Minimum Monitored Clock Count

Variable	MinClockMonitorCount	
Description	Minimum Monitored Clock Count = (Ideal Monitored Clock Count x 0.95) - 3(MIN)	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1024
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.12 Maximum Monitored Clock Count

Variable	MaxClockMonitorCount	
Description	Maximum Monitored Clock Count = (Ideal Monitored Clock Count x 1.05) + 3 (MAX)	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1158
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.13 Period Monitor Window

Variable	PerMonitorWindow	
Description	CMU_PERIOD[WINDOW] - Period Monitor mode WINDOW. 4bits counter. In Period monitor mode, CMU started at period start point, when CMU complete once operation, period monitor counter increase, when period monitor counter reach WINDOW, CMU stop and wait next period start point.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1

	Origin	Flagchip
Requirements	FUNC_MCU_0051	

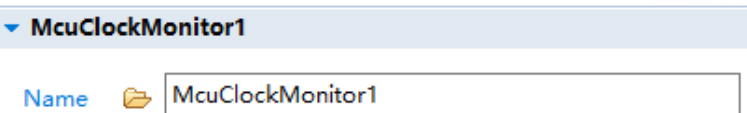
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.1.14 CmuISRNotification

Variable	PerMonitorWindow	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2 McuClockMonitor1

Container	McuClockMonitor1	
Description	User SIRC to monitor FOSC clock. SIRC32_1K as period monitor source Make sure SIRC clock and RTC_clock valid before enable.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

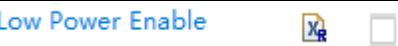
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.1 CMU1 Enable

Variable	ClockMonitor1Enable	
Description	CMU_CTRL[ENABLE], This bit to enable CMU logic. 0 - RTC_CLK monitor disable. 1 - RTC_CLK monitor enabled. Note: Implementation Specific Parameter. except CTRL0 byte0, all others configured register should write when this bit de-asserted.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.2 Low Power Enable

Variable	LowPowerEnable	
Description	CMU_CTRL[LP_EN], Standby mode enable, need STOP_EN asserted. 0 - CMU disable in standby modes. 1 - CMU enable in standby modes. Note: Implementation Specific Parameter.	
Screenshot		

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.3 Stop Enable

Variable	StopEnable	
Description	CMU_CTRL[STOP_EN], Stop mode enable. 0 - CMU disable in stop mode. 1 - CMU enable in Stop mode. Note: Implementation Specific Parameter.	
Screenshot	Stop Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.4 Enable CMU1 Interrupt

Variable	InterruptEnable	
Description	CMU_CTRL[IRQ_EN], Interrupt enable, if this bit asserted. Either LOC or MIS asserted can lead the interrupt. 0 - Interrupt disable. 1 - Interrupt enable. Note: Implementation Specific Parameter.	
Screenshot	Enable CMU1 Interrupt  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.5 Period Monitor Enable

Variable	PerMonitorEnable	
Description	CMU_PERIOD[EN] - Period Monitor mode enable. If this bit asserted, CMU would sample for WINDOW times. 0 - Period monitor is disabled. 1 - Period monitor is enabled. Note: if another period start point is happened during this time. the period monitor counter would clear and restarted.	
Screenshot	Period Monitor Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.6 Reference clock divider

Variable	RefDiv	
Description	The divider for the reference clock.	
Screenshot		
Properties	Property	Value
	Type	Integer(Range)
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.7 Minimum REF_WINDOW value(1 ~ 15978297)

Variable	MinRefWindow	
Description	Minimum REF_WINDOW: A lower value of REF_WINDOW results in a shorter checking window and faster response. Minimum REF_WINDOW = rounding up value of $6 + 5 * (f(\text{Reference clock}) / f(\text{Monitored clock}))$.	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	0
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.8 Maximum REF_WINDOW value(0 ~ 15978297)

Variable	MaxRefWindow	
Description	Maximum REF_WINDOW: A higher value of REF_WINDOW results in a longer checking window leading to better accuracy in monitored clock measurement. Maximum REF_WINDOW = MIN value (rounding down) of $(0\text{xfffff} \times (f(\text{Reference clock}) / f(\text{monitor clock}) / 2^{\text{REF_DIV}}))$, and 0xfffff	
Screenshot		
Properties	Property	Value
	Type	Float
	SymbolicName	false
	Default	0.0
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.9 REF_WINDOW value

Variable	RefWindow	
Description	Please input a number between Minimum REF_WINDOW value and Maximum REF_WINDOW value Set REF_WINDOW, The Min and Max values are limited values, but in real situations, from the perspective of measurement accuracy and event response time, it is recommended to use a typical	

	value between Min and Max, like Minimum REF_WINDOW value x 100 and Maximum REF_WINDOW value /100, etc. note: If Period Monitor Window function is enabled, the REF_WINDOW*10/(reference clock) less than 1ms	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	10
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

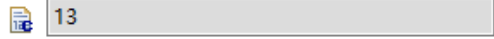
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.10 Minimum Monitored Clock Count

Variable	MinClockMonitorCount	
Description	Minimum Monitored Clock Count = (Ideal Monitored Clock Count x 0.95) - 3 (MIN)	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1024
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

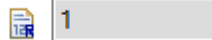
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.11 Maximum Monitored Clock Count

Variable	MaxClockMonitorCount	
Description	Maximum Monitored Clock Count = (Ideal Monitored Clock Count x 1.05) + 3 (MAX)	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1158
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.12 Period Monitor Window

Variable	PerMonitorWindow	
Description	CMU_PERIOD[WINDOW] - Period Monitor mode WINDOW. 4bits counter. In Period monitor mode, CMU started at period start point, when CMU complete once operation, period monitor counter increase, when period monitor counter reach WINDOW, CMU stop and wait next period start point.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip

Requirements	FUNC_MCU_0051
---------------------	---------------

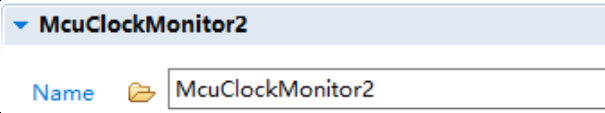
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.2.13 CmulSRNotification

Variable	PerMonitorWindow	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3 McuClockMonitor2

Container	McuClockMonitor2	
Description	User SIRC to monitor FIRC clock. SIRC32_1K as period monitor source Make sure SIRC clock and RTC_clock valid before enable.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.1 CMU2 Enable

Variable	ClockMonitor2Enable	
Description	CMU_CTRL[ENABLE], This bit to enable CMU logic. 0 - RTC_CLK monitor disable. 1 - RTC_CLK monitor enabled. Note: Implementation Specific Parameter. except CTRL0 byte0, all others configured register should write when this bit de-asserted.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.2 Low Power Enable

Variable	LowPowerEnable	
Description	CMU_CTRL[LP_EN], Standby mode enable, need STOP_EN asserted. 0 - CMU disable in standby modes. 1 - CMU enable in standby modes. Note: Implementation Specific Parameter.	
Screenshot		

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.3 Stop Enable

Variable	StopEnable	
Description	CMU_CTRL[STOP_EN], Stop mode enable. 0 - CMU disable in stop mode. 1 - CMU enable in Stop mode. Note: Implementation Specific Parameter.	
Screenshot	Stop Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.4 Enable CMU2 Interrupt

Variable	InterruptEnable	
Description	CMU_CTRL[IRQ_EN], Interrupt enable, if this bit asserted. Either LOC or MIS asserted can lead the interrupt. 0 - Interrupt disable. 1 - Interrupt enable. Note: Implementation Specific Parameter.	
Screenshot	Enable CMU2 Interrupt  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.5 Period Monitor Enable

Variable	PerMonitorEnable	
Description	CMU_PERIOD[EN] - Period Monitor mode enable. If this bit asserted, CMU would sample for WINDOW times. 0 - Period monitor is disabled. 1 - Period monitor is enabled. Note: if another period start point is happened during this time. the period monitor counter would clear and restarted.	
Screenshot	Period Monitor Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.6 Reference clock divider

Variable	RefDiv	
Description	The divider for the reference clock.	
Screenshot		
Properties	Property	Value
	Type	Integer(Range)
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.7 Minimum REF_WINDOW value(1 ~ 15978297)

Variable	MinRefWindow	
Description	Minimum REF_WINDOW: A lower value of REF_WINDOW results in a shorter checking window and faster response. Minimum REF_WINDOW = rounding up value of $6 + 5 * (f(\text{Reference clock}) / f(\text{Monitored clock}))$.	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.8 Maximum REF_WINDOW value(0 ~ 15978297)

Variable	MaxRefWindow	
Description	Maximum REF_WINDOW: A higher value of REF_WINDOW results in a longer checking window leading to better accuracy in monitored clock measurement. Maximum REF_WINDOW = MIN value (rounding down) of $(0\text{xfffff} \times (f(\text{Reference clock}) / f(\text{monitor clock}) / 2^{\text{REF_DIV}})$, and 0xfffff	
Screenshot		
Properties	Property	Value
	Type	Float
	SymbolicName	false
	Default	5592.405
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.9 REF_WINDOW value

Variable	RefWindow	
Description	Please input a number between Minimum REF_WINDOW value and Maximum REF_WINDOW value Set REF_WINDOW, The Min and Max values are limited values, but in real situations, from the perspective of measurement accuracy and event response time, it is recommended to use a typical value between Min and Max, like Minimum REF_WINDOW value x 100 and Maximum REF_WINDOW	

	value /100, etc.note: If Period Monitor Window function is enabled, the REF_WINDOW*10/(reference clock) less than 1ms	
Screenshot	 	
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	10
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.10 Minimum Monitored Clock Count

Variable	MinClockMonitorCount	
Description	Minimum Monitored Clock Count = (Ideal Monitored Clock Count x 0.95) - 3(MIN)	
Screenshot	 	
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1024
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

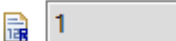
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.11 Maximum Monitored Clock Count

Variable	MaxClockMonitorCount	
Description	Maximum Monitored Clock Count = (Ideal Monitored Clock Count x 1.05) + 3 (MAX)	
Screenshot	 	
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1158
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.12 Period Monitor Window

Variable	PerMonitorWindow	
Description	CMU_PERIOD[WINDOW] - Period Monitor mode WINDOW. 4bits counter. In Period monitor mode, CMU started at period start point, when CMU complete once operation, period monitor counter increase, when period monitor counter reach WINDOW, CMU stop and wait next period start point.	
Screenshot	 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

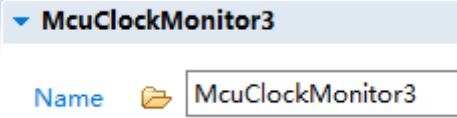
[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.3.13 CmuISRNotification

Variable	PerMonitorWindow	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4 McuClockMonitor3

Container	McuClockMonitor3	
Description	User FIRC to monitor SIRC clock. SIRC32_1K as period monitor source Make sure SIRC clock and FIRC valid before enable.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.1 CMU3 Enable

Variable	ClockMonitor3Enable	
Description	CMU_CTRL[ENABLE], This bit to enable CMU logic. 0 - RTC_CLK monitor disable. 1 - RTC_CLK monitor enabled. Note: Implementation Specific Parameter. except CTRL0 byte0, all others configured register should write when this bit de-asserted.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.2 Low Power Enable

Variable	LowPowerEnable	
Description	CMU_CTRL[LP_EN], Standby mode enable, need STOP_EN asserted. 0 - CMU disable in standby modes. 1 - CMU enable in standby modes. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN

	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.3 Stop Enable

Variable	StopEnable	
Description	CMU_CTRL[STOP_EN], Stop mode enable. 0 - CMU disable in stop mode. 1 - CMU enable in Stop mode. Note: Implementation Specific Parameter.	
Screenshot	Stop Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.4 Enable CMU3 Interrupt

Variable	InterruptEnable	
Description	CMU_CTRL[IRQ_EN], Interrupt enable, if this bit asserted. Either LOC or MIS asserted can lead the interrupt. 0 - Interrupt disable. 1 - Interrupt enable. Note: Implementation Specific Parameter.	
Screenshot	Enable CMU3 Interrupt  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.5 Period Monitor Enable

Variable	PerMonitorEnable	
Description	CMU_PERIOD[EN] - Period Monitor mode enable. If this bit asserted, CMU would sample for WINDOW times. 0 - Period monitor is disabled. 1 - Period monitor is enabled. Note: if another period start point is happened during this time. the period monitor counter would clear and restarted.	
Screenshot	Period Monitor Enable  	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.6 Reference clock divider

Variable	RefDiv
Description	The divider for the reference clock.

Screenshot	Reference clock divider 	
Properties	Property	Value
	Type	Integer(Range)
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.7 Minimum REF_WINDOW value(1 ~ 15978297)

Variable	MinRefWindow	
Description	Minimum REF_WINDOW: A lower value of REF_WINDOW results in a shorter checking window and faster response. Minimum REF_WINDOW = rounding up value of $6 + 5 * (f(\text{Reference clock}) / f(\text{Monitored clock}))$.	
Screenshot	Minimum REF_WINDOW value(1 ~ 15978297) 	
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.8 Maximum REF_WINDOW value(0 ~ 15978297)

Variable	MaxRefWindow	
Description	Maximum REF_WINDOW: A higher value of REF_WINDOW results in a longer checking window leading to better accuracy in monitored clock measurement. Maximum REF_WINDOW = MIN value (rounding down) of $(0\text{xfffff} \times (f(\text{Reference clock}) / f(\text{monitor clock}) / 2^{\text{REF_DIV}})$, and 0xfffff	
Screenshot	Maximum REF_WINDOW value(0 ~ 15978297) 	
Properties	Property	Value
	Type	Float
	SymbolicName	false
	Default	5592.405
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.9 REF_WINDOW value

Variable	RefWindow	
Description	Please input a number between Minimum REF_WINDOW value and Maximum REF_WINDOW value Set REF_WINDOW, The Min and Max values are limited values, but in real situations, from the perspective of measurement accuracy and event response time, it is recommended to use a typical value between Min and Max, like Minimum REF_WINDOW value x 100 and Maximum REF_WINDOW value / 100, etc. note: If Period Monitor Window function is enabled, the $\text{REF_WINDOW} * 10 / (\text{reference clock})$ less than 1ms	
Screenshot	REF_WINDOW value 	

Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	cacluted
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.10 Minimum Monitored Clock Count

Variable	MinClockMonitorCount	
Description	Minimum Monitored Clock Count = (Ideal Monitored Clock Count x 0.95) - 3(MIN)	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1024
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.11 Maximum Monitored Clock Count

Variable	MaxClockMonitorCount	
Description	Maximum Monitored Clock Count = (Ideal Monitored Clock Count x 1.05) + 3 (MAX)	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1158
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.12 Period Monitor Window

Variable	PerMonitorWindow	
Description	CMU_PERIOD[WINDOW] - Period Monitor mode WINDOW. 4bits counter. In Period monitor mode, CMU started at period start point, when CMU complete once operation, period monitor counter increase, when period monitor counter reach WINDOW, CMU stop and wait next period start point.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.4.13 CmulSRNotification

Variable	PerMonitorWindow
-----------------	------------------

Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5 McuClockMonitor4

Container	McuClockMonitor4	
Description	User FIRC to monitor Slow Clock. SIRC32_1K as period monitor source Make sure SIRC clock and RTC_clock valid before enable.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.1 CMU4 Enable

Variable	ClockMonitor4Enable	
Description	CMU_CTRL[ENABLE], This bit to enable CMU logic. 0 - Slow Clock monitor disable. 1 - Slow Clock monitor enabled. Note: Implementation Specific Parameter. except CTRL0 byte0, all others configured register should write when this bit de-asserted.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.2 Low Power Enable

Variable	LowPowerEnable	
Description	CMU_CTRL[LP_EN], Standby mode enable, need STOP_EN asserted. 0 - CMU disable in standby modes. 1 - CMU enable in standby modes. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN

	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.3 Stop Enable

Variable	StopEnable	
Description	CMU_CTRL[STOP_EN], Stop mode enable. 0 - CMU disable in stop mode. 1 - CMU enable in Stop mode. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.4 Enable CMU4 Interrupt

Variable	InterruptEnable	
Description	CMU_CTRL[IRQ_EN], Interrupt enable, if this bit asserted. Either LOC or MIS asserted can lead the interrupt. 0 - Interrupt disable. 1 - Interrupt enable. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.5 Period Monitor Enable

Variable	PerMonitorEnable	
Description	CMU_PERIOD[EN] - Period Monitor mode enable. If this bit asserted, CMU would sample for WINDOW times. 0 - Period monitor is disabled. 1 - Period monitor is enabled. Note: if another period start point is happened during this time. the period monitor counter would clear and restarted.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.6 Reference clock divider

Variable	RefDiv
Description	The divider for the reference clock.

Screenshot	Reference clock divider 	
Properties	Property	Value
	Type	Integer(Range)
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.7 Minimum REF_WINDOW value(1 ~ 15978297)

Variable	MinRefWindow	
Description	Minimum REF_WINDOW: A lower value of REF_WINDOW results in a shorter checking window and faster response. Minimum REF_WINDOW = rounding up value of $6 + 5 * (f(\text{Reference clock}) / f(\text{Monitored clock}))$.	
Screenshot	Minimum REF_WINDOW value(1 ~ 15978297) 	
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.8 Maximum REF_WINDOW value(0 ~ 15978297)

Variable	MaxRefWindow	
Description	Maximum REF_WINDOW: A higher value of REF_WINDOW results in a longer checking window leading to better accuracy in monitored clock measurement. Maximum REF_WINDOW = MIN value (rounding down) of $(0\text{xfffff} \times (f(\text{Reference clock}) / f(\text{monitor clock}) / 2^{\text{REF_DIV}})$, and 0xfffff	
Screenshot	Maximum REF_WINDOW value(0 ~ 15978297) 	
Properties	Property	Value
	Type	Float
	SymbolicName	false
	Default	5592.405
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.9 REF_WINDOW value

Variable	RefWindow	
Description	Please input a number between Minimum REF_WINDOW value and Maximum REF_WINDOW value Set REF_WINDOW, The Min and Max values are limited values, but in real situations, from the perspective of measurement accuracy and event response time, it is recommended to use a typical value between Min and Max, like Minimum REF_WINDOW value x 100 and Maximum REF_WINDOW value / 100, etc. note: If Period Monitor Window function is enabled, the $\text{REF_WINDOW} * 10 / (\text{reference clock})$ less than 1ms	
Screenshot	REF_WINDOW value 	

Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	10
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.10 Minimum Monitored Clock Count

Variable	MinClockMonitorCount	
Description	Minimum Monitored Clock Count = (Ideal Monitored Clock Count x 0.95) - 3(MIN)	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1024
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.11 Maximum Monitored Clock Count

Variable	MaxClockMonitorCount	
Description	Maximum Monitored Clock Count = (Ideal Monitored Clock Count x 1.05) + 3 (MAX)	
Screenshot		
Properties	Property	Value
	Type	Integer
	SymbolicName	false
	Default	1158
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.12.14.5.12 Period Monitor Window

Variable	PerMonitorWindow	
Description	CMU_PERIOD[WINDOW] - Period Monitor mode WINDOW. 4bits counter. In Period monitor mode, CMU started at period start point, when CMU complete once operation, period monitor counter increase, when period monitor counter reach WINDOW, CMU stop and wait next period start point.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

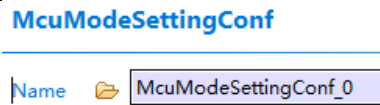
3.2.5.12.14.5.13 CmuISRNotification

Variable	PerMonitorWindow
-----------------	------------------

Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	String
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip
Requirements	FUNC_MCU_0051	

[SWDESG_MCU_193] SWDESG_MCU_193 covers this item.

3.2.5.13 McuModeSettingConf

Container	McuModeSettingConf	
Description	This container contains the configuration for the Mode setting of the MCU. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	SWS_MCU_00165, ECUC_MCU_00123	

[SWDESG_MCU_163] The number of modes and the configuration can be set.

3.2.5.13.1 Mode ID

Variable	McuMode	
Description	This parameter shall represent the ID of the MCU mode.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	true
	Origin	AUTOSAR_ECUC
	Range	N/A
Requirements	ECUC_MCU_00176	

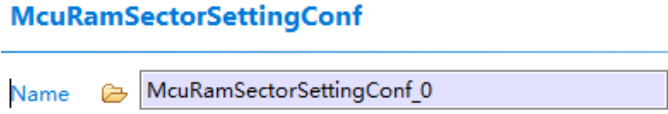
[SWDESG_MCU_196] The parameter represents the MCU Mode settings.

3.2.5.13.2 Operating Mode

Variable	McuPowerMode	
Description	This parameter selects the Power Mode to be used. For valid Mode transitions refers to Power mode state diagram from Reference Manual. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	RUN
	Origin	Flagchip
Requirements	Range	N/A
	ECUC_MCU_00176	

[SWDESG_MCU_196] SWDESG_MCU_196 covers this item.

3.2.5.14 McuRamSectorSettingConf

Container	McuRamSectorSettingConf	
Description	This container contains the configuration for the RAM Sector setting.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	ECUC_MCU_00120	

[SWDESG_MCU_197] This container contains the configuration (parameters) for the RAM Sector setting. RAM Sector ID

3.2.5.15 McuRamSectorId

Variable	McuRamSectorId	
Description	This parameter shall represent the ID of the MCU RAM Sector configuration.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	true
	Origin	Flagchip
Requirements	ECUC_MCU_00120	

[SWDESG_MCU_197] SWDESG_MCU_197 covers this item.

3.2.5.15.1 RAM Default Value

Variable	McuRamDefaultValue	
Description	This parameter shall represent the Data pre-setting to be initialized. Default value is 0xbabababa. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC
Requirements	ECUC_MCU_00177	

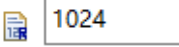
[SWDESG_MCU_198] This parameter shall represent the Data pre-setting to be initialized.

3.2.5.15.2 RAM Section Base Address

Variable	McuRamSectionBaseAddress	
Description	This parameter represents the RAM section base address. The address must be aligned to 4 bytes. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	33554432
	Origin	AUTOSAR_ECUC
	Range	N/A
Requirements	ECUC_MCU_00178	

[SWDESG_MCU_199] This parameter shall represent the MCU RAM section base address.

3.2.5.15.3 RAM Section Size

Variable	McuRamSectionSize	
Description	This parameter represents the RAM section size in bytes. The size must be multiple of 4. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1024
	Origin	AUTOSAR_ECUC
	Range	N/A
Requirements	ECUC_MCU_00179	

[SWDESG_MCU_200] This parameter represents the MCU RAM Section size in bytes.

3.2.5.15.4 RAM Section Base Address Linker Symbol

Variable	McuRamSectionBaseAddrLinkerSym	
Description	This parameter represents the RAM section base address. The address must be aligned to 4 bytes. If this parameter is empty, then the integer values from "McuRamSectionBaseAddress" will be used. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	STRING
	SymbolicName	false
	Origin	Flagchip
Requirements	ECUC_MCU_00179	

[SWDESG_MCU_197] SWDESG_MCU_197 covers this item.

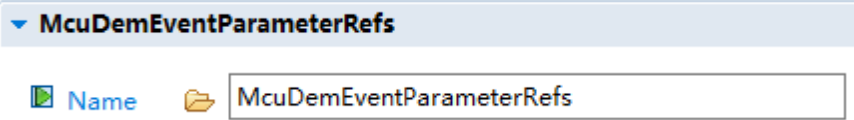
3.2.5.15.5 RAM Section Size Linker Symbol

Variable	McuRamSectionSizeLinkerSym	
Description	This parameter represents the RAM section size in bytes. The size must be multiple of 4. If this parameter is empty, then the integer values from "McuRamSectionSize" will be used. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	STRING
	SymbolicName	false
	Origin	Flagchip
Requirements	ECUC_MCU_00179	

[SWDESG_MCU_197] SWDESG_MCU_197 covers this item.

3.2.5.16 McuDemEventParameterRefs

Container	McuDemEventParameterRefs
Description	Container for the references to DemEventParameter elements which shall be invoked using the API Dem_ReportErrorStatus API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.

Screenshot		
	Property	Value
Properties	Type	IDENTIFIABLE
Requirements	ECUC_MCU_00187	

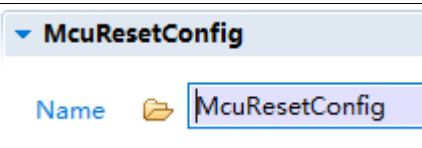
[SWDESG_MCU_194] Container for the references to DemEventParameter elements which shall be invoked using the API.

3.2.5.16.1 MCU_E_CLOCK_FAILURE

Variable	MCU_E_CLOCK_FAILURE	
Description	Reference to configured DEM event to report Timeout failure.	
Screenshot		
Properties	Property	Value
	Type	SYMBOLIC-NAME-REFERENCE
	Origin	Flagchip
Requirements	SWS_MCU_00053, ECUC_MCU_00188	

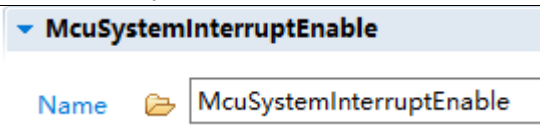
[SWDESG_MCU_195] Reference to configured DEM event to report "Clock source failure".

3.2.5.17 McuResetConfig

Container	McuResetConfig	
Description	The Reset Control Module centralizes the different reset sources and manages the reset sequence of the device. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0048	

[SWDESG_MCU_186] SWDESG_MCU_186 covers this item.

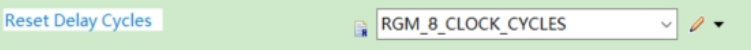
3.2.5.17.1 McuSystemInterruptEnable

Container	McuSystemInterruptEnable	
Description	Configures RGM_SRIE This register can delay the assertion of a system reset and generate an interrupt first, in this way software can perform some procedure before reset. Notice that chip POR reset cannot be delayed by this feature. This feature uses AON32K clock to count the delay. This register is reset on Chip POR only.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0047	

[SWDESG_MCU_214] This container contains the configuration (parameters) for system interrupt settings.

3.2.5.17.1.1 Reset Delay Cycles

Variable	McuResetDelayTime
Description	RGM_SRIE[DELAY] - Reset Delay.

	configures the reset delay time from when the interrupt assert and the reset eventually occur (actually time need to plus 2 clocks for sync). In F7xxx, the clock used is 32K clock which is selected by CSC0 CSC0_AONCLKSR[RTCCLKSEL]. 0 - RGM_8_CLOCK_CYCLES. 1 - RGM_32_CLOCK_CYCLES. 2 - RGM_128_CLOCK_CYCLES. 3 - RGM_512_CLOCK_CYCLES. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	RGM_8_CLOCK_CYCLES
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0047	

[SWDESG_MCU_214] SWDESG_MCU_214 covers this item.

3.2.5.17.1.2 Enable Stop Acknowledge Error Interrupt

Variable	StopAcknowledgeErrorInterrupt	
Description	RGM_SRIE[SACKERR_RIE] - Stop Acknowledge Error Reset Interrupt Enable. 0 - SACKERR reset interrupt disabled. 1 - SACKERR reset interrupt enabled.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0047	

[SWDESG_MCU_214] SWDESG_MCU_214 covers this item.

3.2.5.17.1.3 Enable WatchDog1 Error Interrupt

Variable	Enable WatchDog1 Error Interrupt	
Description	FC7300FXXX: Untouch FC7240FXXX: Optional RGM_SRIE[WDOG1_RIE] - WatchDog1 Reset Interrupt Enable. 0 - WatchDog1 reset interrupt disabled. 1 - WatchDog1 reset interrupt enabled.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0047	

[SWDESG_MCU_214] SWDESG_MCU_214 covers this item.

3.2.5.17.1.4 Enable SYSAP Reset Interrupt

Variable	SysAPResetInterrupt
-----------------	---------------------

Description	RGM_SRIE[SYSAP_RIE] - SYSAP System Reset Interrupt Enable. 0 - SYSAP reset interrupt disabled. 1 - SYSAP reset interrupt enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0047	

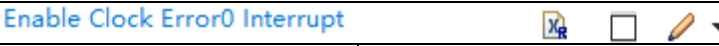
[SWDESG_MCU_214] SWDESG_MCU_214 covers this item.

3.2.5.17.1.5 Enable JTAG Reset Interrupt

Variable	JTAGResetInterrupt	
Description	RGM_SRIE[JTAG] - JTAG generated reset. 0 - Interrupt disabled. 1 - Interrupt enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0047	

[SWDESG_MCU_214] SWDESG_MCU_214 covers this item.

3.2.5.17.1.6 Enable Clock Error0 Interrupt

Variable	ClockError0ResetInterrupt	
Description	RGM_SRIE[CLKERR0_RIE] - Clock Error0 Reset Interrupt Enable. 0 - Interrupt disabled. 1 - Interrupt enabled. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0047	

[SWDESG_MCU_214] SWDESG_MCU_214 covers this item.

3.2.5.17.2 McuPartition0InterruptEnable

Container	McuPartition0InterruptEnable	
Description	Configures RGM_SRIE This register can generate an interrupt before core 0 reset happened.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0056	

[SWDESG_MCU_239] This container contains the configuration (parameters) for CPU0 interrupt settings.

3.2.5.17.2.1 Enable CPU0 xxx Interrupt

Variable	Cpu0SoftwareResetInterrupt/ Cpu0InterruptMonitorTimeoutInterrupt/ Cpu0WatchdogInterrupt/ Cpu0SystemCoreResetInterrupt/ Cpu0LockupInterrupt	
Description	RGM_C0_CFG CPU0 Software Reset Interrupt. 0 - reset interrupt disable. 1 - reset interrupt enable. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0056	

[SWDESG_MCU_239] SWDESG_MCU_239 covers this item.

3.2.5.17.3 McuPartition1InterruptEnable

Container	McuPartition1InterruptEnable	
Description	Configures RGM_SRIE This register can generate an interrupt before core 1 reset happened.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0057	

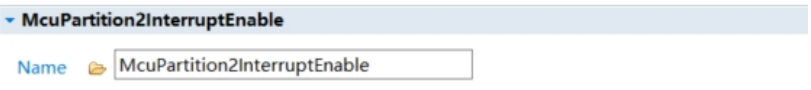
[SWDESG_MCU_240] This container contains the configuration (parameters) for CPU1 interrupt settings.

3.2.5.17.3.1 Enable CPU1 xxx Interrupt

Variable	Cpu1SoftwareResetInterrupt/ Cpu1InterruptMonitorTimeoutInterrupt/ Cpu1WatchdogInterrupt/ Cpu1SystemCoreResetInterrupt/ Cpu1LockupInterrupt	
Description	RGM_C1_CFG CPU0 Software Reset Interrupt. 0 - reset interrupt disable. 1 - reset interrupt enable. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0057	

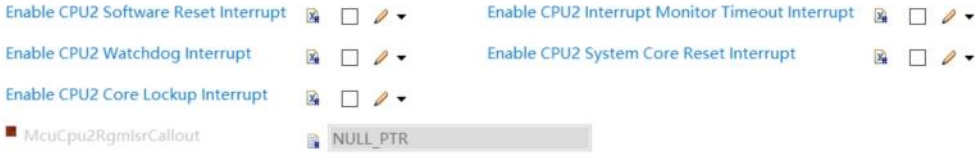
[SWDESG_MCU_240] SWDESG_MCU_240 covers this item.

3.2.5.17.4 McuPartition2InterruptEnable

Container	McuPartition2InterruptEnable	
Description	Configures RGM_SRIE This register can generate an interrupt before core 0 reset happened.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0058	

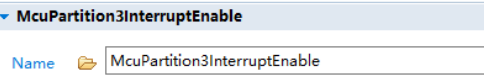
[SWDESG_MCU_241] This container contains the configuration (parameters) for CPU2 interrupt settings.

3.2.5.17.4.1 Enable CPU2 xxx Interrupt

Variable	Cpu2SoftwareResetInterrupt/ Cpu2InterruptMonitorTimeoutInterrupt/ Cpu2WatchdogInterrupt/ Cpu2SystemCoreResetInterrupt/ Cpu2LockupInterrupt	
Description	RGM_C2_CFG CPU0 Software Reset Interrupt. 0 - reset interrupt disable. 1 - reset interrupt enable. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0058	

[SWDESG_MCU_241] SWDESG_MCU_241 covers this item.

3.2.5.17.5 McuPartition3InterruptEnable

Container	McuPartition3InterruptEnable	
Description	Configures RGM_SRIE This register can generate an interrupt before core 0 reset happened.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0059	

[SWDESG_MCU_245] This container contains the configuration (parameters) for CPU5 interrupt settings.

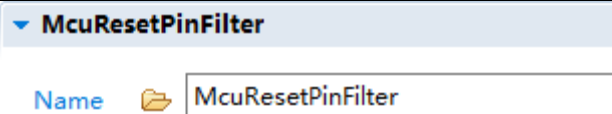
3.2.5.17.5.1 Enable CPU3 xxx Interrupt

Variable	Cpu3SoftwareResetInterrupt/ Cpu3InterruptMonitorTimeoutInterrupt/ Cpu3WatchdogInterrupt/ Cpu3SystemCoreResetInterrupt/ Cpu3LockupInterrupt	
Description	RGM_C3_CFG CPU0 Software Reset Interrupt. 0 - reset interrupt disable. 1 - reset interrupt enable. Note: Implementation Specific Parameter.	

Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0060	

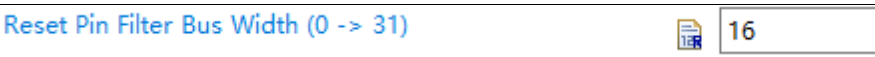
[SWDESG_MCU_245] SWDESG_MCU_245 covers this item.

3.2.5.17.6 McuResetPinFilter

Container	McuResetPinFilter	
Description	Configures RGM_RSTFLT RSTFLT register provide the function to filter the glitch on reset pad. This register is reset on POR only, other reset types will have no effect.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0048	

[SWDESG_MCU_215] This container contains the configuration (parameters) for reset filter settings.

3.2.5.17.6.1 Reset Pin Filter Bus Width

Variable	RSTFLT_BUSW	
Description	RGM_RSTFLT[RSTFLT_BUSW] - Reset Pin Filter Bus Clock Width configuration. Configure the number of bus clocks of reset pin filter Reset signal lengths less than RSTFLTSEL cycles are filtered. Reset signal lengths between RSTFLTSEL and (RSTFLTSEL+1) cycles may be filtered, Reset signal lengths greater than (RSTFLTSEL+1) cycles are not filtered. Bit8--Bit12, 5bits bitfield. rang 0--31.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	16
	Origin	Flagchip
Requirements	FUNC_MCU_0048	

[SWDESG_MCU_215] SWDESG_MCU_215 covers this item.

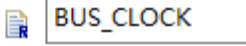
3.2.5.17.6.2 Reset Pin Filter In Low Power Mode

Variable	RSTFLT_AON_LP	
Description	RGM_RSTFLT[RSTFLT_AON_LP] - Reset Pin Filter Using AON32K clock in low power mode. Reset signal length less than 3 AON32K clock will be filtered. In FC4150F512 this field is reserved . 0 - filter function disabled. 1 - filter function enabled.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false

	Origin	Flagchip
Requirements	FUNC_MCU_0048	

[SWDESG_MCU_215] SWDESG_MCU_215 covers this item.

3.2.5.17.6.3 Reset Pin Filter Clock Source

Variable	RSTFLT_CLK_SRC	
Description	RGM_RSTFLT[RSTFLT_AON/RSTFLT_BUS] - Reset Pin Filter ENABLE Using AON32K clock or BUS_CLOCK. 0 - filter choose BUS_CLOCK. 1 - filter choose AON32K_CLOCK.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	BUS_CLOCK
	Origin	Flagchip
	Range	N/A
Requirements	FUNC_MCU_0048	

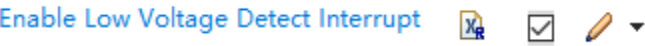
[SWDESG_MCU_215] SWDESG_MCU_215 covers this item.

3.2.5.18 McuPowerControl

Container	McuPowerControl	
Description	Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0049	

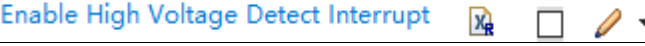
[SWDESG_MCU_216] This container contains the configuration (parameters) for power control settings.

3.2.5.18.1 Enable Low Voltage Detect Interrupt

Variable	McuLowVoltageDetectInterruptEnable	
Description	PMC_CONFIG[LVD_IE] - Low Voltage Detect Interrupt Enable. This bit enables hardware interrupt requests for LVDF. 0 - Hardware interrupt disabled (use polling). 1 - Request a hardware interrupt when LVDF = 1.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0049	

[SWDESG_MCU_216] SWDESG_MCU_216 covers this item.

3.2.5.18.2 Enable High Voltage Detect Interrupt

Variable	McuHighVoltageDetectInterruptEnable	
Description	PMC_CONFIG[HVD_IE] - Low Voltage Detect Interrupt Enable. This bit enables hardware interrupt requests for LVDF. 0 - Hardware interrupt disabled (use polling). 1 - Request a hardware interrupt when LVDF = 1. Note: - Implementation Specific Parameter.	
Screenshot		

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0049	

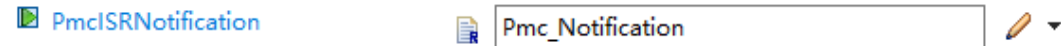
[SWDESG_MCU_216] SWDESG_MCU_216 covers this item.

3.2.5.18.3 Enable 2V5 Domain During RPM

Variable	V25DomainInRpmEnable	
Description	PMC_LVDSC1[LVDIE] - V25 domain enable during RPM. Reduce power mode (RPM): For low-current consumption. This bit enables hardware interrupt requests for LVDF. 0b - V25 regulator are disabled in RPM. 1b - V25 regulator are enabled in RPM. Note: - Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0049	

[SWDESG_MCU_216] SWDESG_MCU_216 covers this item.

3.2.5.18.4 PmcISRNotification

Variable	PmcISRNotification	
Description	Function pointer to callback function. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip
Requirements	FUNC_MCU_0049	

[SWDESG_MCU_216] SWDESG_MCU_216 covers this item.

3.2.5.19 McuWakeupControl

Container	McuWakeupControl	
Description	Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] McuWakeupControl container contains the configuration for the wake up source setting.

3.2.5.19.1 Enable CMP0 as Wakeup Source

Variable	CMP0	
Description	WKU_MWER1[MWE10] - Wakeup Module Control For Module 10. 0 - Disable WKU_INPUT10 as wakeup source. 1 - Enable WKU_INPUT10 as wakeup source.	
Screenshot		

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.2 Enable TSTMP0 as Wakeup Source

Variable	TSTMP0	
Description	WKU_MWER1[MWE13] - Wakeup Module Control For Module 13. 0 - Disable WKU_INPUT13 as wakeup source. 1 - Enable WKU_INPUT13 as wakeup source.	
Screenshot	Enable TSTMP0 as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.3 Enable RTC Alarm as Wakeup Source

Variable	RTC_Alarm	
Description	WKU_MWER1[MWE14] - Wakeup Module Control For Module 14. 0 - Disable WKU_INPUT14 as wakeup source. 1 - Enable WKU_INPUT14 as wakeup source.	
Screenshot	Enable RTC Alarm as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.4 Enable RTC Seconds as Wakeup Source

Variable	RTC_Seconds	
Description	WKU_MWER1[MWE15] - Wakeup Module Control For Module 15. 0 - Disable WKU_INPUT15 as wakeup source. 1 - Enable WKU_INPUT15 as wakeup source.	
Screenshot	Enable RTC Seconds as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.5 Enable AONTIMER as Wakeup Source

Variable	AONTIMER	
Description	WKU_MWER2[MWE18] - Wakeup Module Control For Module 18. 0 - Disable WKU_INPUT18 as wakeup source. 1 - Enable WKU_INPUT18 as wakeup source.	
Screenshot	Enable AONTIMER as Wakeup Source 	

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.6 Enable ADC1_LPWAKE as Wakeup Source

Variable	ADC1_LPWAKE	
Description	WKU_MWER2[MWE18] - Wakeup Module Control For Module 18. 0 - Disable WKU_INPUT18 as wakeup source. 1 - Enable WKU_INPUT18 as wakeup source.	
Screenshot	Enable ADC1 LPWAKE as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.7 Enable ADC2_LPWAKE as Wakeup Source

Variable	ADC2_LPWAKE	
Description	WKU_MWER2[MWE18] - Wakeup Module Control For Module 18. 0 - Disable WKU_INPUT18 as wakeup source. 1 - Enable WKU_INPUT18 as wakeup source.	
Screenshot	Enable ADC2 LPWAKE as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.8 Enable GPIOA as Wakeup Source

Variable	GPIOA	
Description	WKU_MWER2[MWE19] - Wakeup Module Control For Module 19. 0 - Disable WKU_INPUT19 as wakeup source. 1 - Enable WKU_INPUT19 as wakeup source.	
Screenshot	Enable GPIOA as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.9 Enable GPIOB as Wakeup Source

Variable	GPIOB	
Description	WKU_MWER2[MWE0] - Wakeup Module Control For Module0. 0 - Disable WKU_INPUT0 as wakeup source. 1 - Enable WKU_INPUT0 as wakeup source.	
Screenshot	Enable GPIOB as Wakeup Source 	
Properties	Property	Value

	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.10 Enable GPIOC as Wakeup Source

Variable	GPIOC	
Description	WKU_MWER2[MWE21] - Wakeup Module Control For Module 21. 0 - Disable WKU_INPUT21 as wakeup source. 1 - Enable WKU_INPUT21 as wakeup source.	
Screenshot	Enable GPIOC as Wakeup Source   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.11 Enable GPIOD as Wakeup Source

Variable	GPIOD	
Description	WKU_MWER2[MWE22] - Wakeup Module Control For Module 22. 0 - Disable WKU_INPUT22 as wakeup source. 1 - Enable WKU_INPUT22 as wakeup source.	
Screenshot	Enable GPIOD as Wakeup Source   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.12 Enable GPIOE as Wakeup Source

Variable	GPIOE	
Description	WKU_MWER2[MWE23] - Wakeup Module Control For Module 23. 0 - Disable WKU_INPUT23 as wakeup source. 1 - Enable WKU_INPUT23 as wakeup source.	
Screenshot	Enable GPIOE as Wakeup Source   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.13 Enable GPIOF as Wakeup Source

Variable	GPIOF	
Description	WKU_MWER2[MWE22] - Wakeup Module Control For Module 22. 0 - Disable WKU_INPUT22 as wakeup source. 1 - Enable WKU_INPUT22 as wakeup source.	
Screenshot	Enable GPIOF as Wakeup Source   	

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.14 Enable GPIOG as Wakeup Source

Variable	GPIOG	
Description	WKU_MWER2[MWE23] - Wakeup Module Control For Module 23. 0 - Disable WKU_INPUT23 as wakeup source. 1 - Enable WKU_INPUT23 as wakeup source.	
Screenshot	Enable GPIOG as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.5.19.15 Enable GPIOH as Wakeup Source

Variable	GPIOH	
Description	WKU_MWER2[MWE4] - Wakeup Module Control For Module4. 0 - Disable WKU_INPUT4 as wakeup source. 1 - Enable WKU_INPUT4as wakeup source.	
Screenshot	Enable GPIOH as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

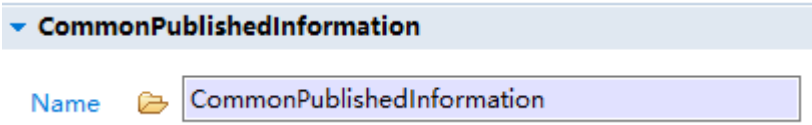
3.2.5.19.16 Enable GPIOI as Wakeup Source

Variable	GPIOI	
Description	WKU_MWER2[MWE4] - Wakeup Module Control For Module5. 0 - Disable WKU_INPUT5 as wakeup source. 1 - Enable WKU_INPUT5 as wakeup source.	
Screenshot	Enable GPIOI as Wakeup Source 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Origin	Flagchip
Requirements	FUNC_MCU_0034	

[SWDESG_MCU_153] SWDESG_MCU_153 covers this item.

3.2.6 CommonPublishedInformation

Container	CommonPublishedInformation
------------------	----------------------------

Description	Common container, aggregated by all modules. It contains published information about vendor and versions.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

[SWDESG_MCU_154] Configuration CommonPublishedInformation container holding the common published information of the module and the container CommonPublishedInformation shall be located in pre-configuration section.

3.2.6.1 AUTOSAR Major Version

Variable	AUTOSAR Major Version	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	4
	Origin	Flagchip
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

[SWDESG_MCU_155] Configuration CommonPublishedInformation container holding the variable ArReleaseMajorVersion.

3.2.6.2 AUTOSAR Minor Version

Variable	AUTOSAR Minor Version	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	6
	Origin	Flagchip
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

[SWDESG_MCU_156] Configuration CommonPublishedInformation container holding the variable ArReleaseMinorVersion.

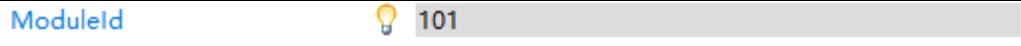
3.2.6.3 AUTOSAR Revision Version

Variable	AUTOSAR Revision Version	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false

	Default	0
	Origin	Flagchip
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

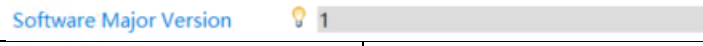
[SWDESG_MCU_157] Configuration CommonPublishedInformation container holding the variable ArReleaseRevisionVersion.

3.2.6.4 ModuleId

Variable	ModuleId	
Description	Module ID of this module from Module List. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	101
	Origin	Flagchip
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

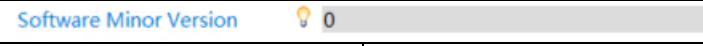
[SWDESG_MCU_158] Configuration CommonPublishedInformation container holding the variable ModuleId.

3.2.6.5 Software Major Version

Variable	Software Major Version	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	1
	Origin	Flagchip
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

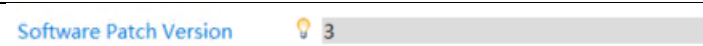
[SWDESG_MCU_159] Configuration CommonPublishedInformation container holding the variable SwMajorVersion.

3.2.6.6 Software Minor Version

Variable	Software Minor Version	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	0
	Origin	Flagchip
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

[SWDESG_MCU_160] Configuration CommonPublishedInformation container holding the variable SwMinorVersion.

3.2.6.7 Software Patch Version

Variable	Software Patch Version	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific. Note: Implementation Specific Parameter	
Screenshot		

Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	3
	Origin	Flagchip
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

[SWDESG_MCU_161] Configuration CommonPublishedInformation container holding the variable SwPatchVersion.

3.2.6.8 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	174
	Origin	Flagchip
Requirements	FUNC_MCU_0007, FUNC_MCU_0008	

[SWDESG_MCU_162] Configuration CommonPublishedInformation container holding the variable VendorId.

Chapter 4 Configuration Guides

4.1 MCU Usage Common Steps

MCU configuration UI is shown below. "General" and "Published Information" is default page, If the element type is "MAP", it would show as a page no matter which container it belongs to.

Mcu

Name*

General | McuEcucPartitionRef | McuResetReasonConf | McuClockSettingConfig | McuModeSettingConf | McuRamSectorConfig | Published Information

ConfigVariant

McuGeneralConfiguration

Name

DevErrorDetect ☒	MultiCoreErrorDetect ☒
McuGetRamStateAPI ☐	McuInitClockAPI ☒
McuNoPLL ☐	McuVersionInfoApi ☐
McuEnableUserModeSupport ☐	McuPerformResetApi ☐
McuCalloutBeforePerformReset ☐	
McuPerformResetCallout NULL_PTR	

McuPublishedInformation

Name

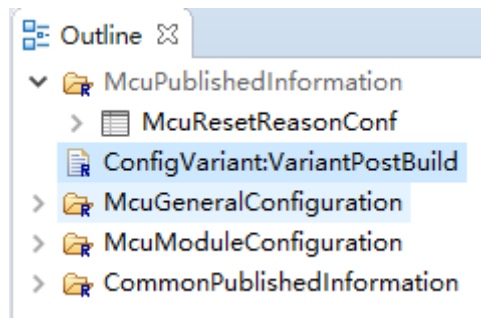
McuModuleConfiguration

Name

Clock Failure Notification

Mcu Number of Mode Settings (1 -> 255)

The MCU configuration outline structure is as follows:



It contains 4 containers: McuPublishedInformation, McuGeneralConfiguration, McuModuleConfiguration and CommonPublishedInformation.

- 1) After the MCU module configuration load, the default setting options will be loaded automatically.
- 2) Configuration Variant. This configuration defines the configuration type. There are two options: PreCompile and PostBuild; choose PostBuild as default.

ConfigVariant

3) General Configuration Setting.

For General configuration setting, check if AUTOSAR API and DET are supported.

Name					
McuGeneralConfiguration					
DevErrorDetect		☐		McuGetRamStateAPI	☐
McuInitClockAPI		☒		McuNoPLL	☐
McuVersionInfoApi		☐		McuEnableUserModeSupport	☐
McuPerformResetApi		☐		McuCalloutBeforePerformReset	☐
McuPerformResetCallout		NULL_PTR			

4) MCU Module Configuration. This container includes the main detailed MCU configuration peripherals, including CMU, PCC, PMC, RCM, SCG, SMC, SMISC, WKU and cortex M7 core instructions.

- ▼ McuModuleConfiguration
 - Mcu Number of Mode Settings:7
 - Mcu Number of RAM Sectors:2
 - Reset Setting:1
 - Clock Failure Notification:DISABLED
 - DisableRCMInit:true
 - DisablePMCAInit:true
 - DisableSMCAInit:true
 - DisableWKUInit:true
 - DisableCMUInit:true
 - EnableCMUInterrupt:false
 - CmuISRNotification:NULL_PTR
 - > McuClockSettingConfig
 - Mcu Disable Dem Report:true
 - > McuDemEventParameterRefs
 - > McuModeSettingConf
 - > McuRamSectorSettingConf
 - > McuResetConfig
 - > McuPowerControl
 - > McuWakeupControl

a) MCU Clock Setting. It's a list type element that supports configuring multiple clock settings.

The MCU_clock_setting function will use one of the clock setting configuration parameters from the clock setting list.

Normally, choose the first 1 as default.

McuClockSettingConfig															
Index	Name	McuCloc...	FOSC En...	FOSC CL...	FOSC cL...	FOSC St...	FOSC By...	FOSC DI...	FOSC DI...	FOSC DI...	FOSC DI...	FOSC DI...	FOSC DI...	FOSC DI...	FOSC DI...
0	MCU_Demo_FOSC24M		☒	☐	☐	☐	☐	1	2	4	2.4E7	2.4E7	1.2E7	6000000.0	☐

The setting contains SCG, PCC, CMU and clock configuration part of CSC and SCM.

The configuration sequence of clock source input is: FOSC → SOSC → SIRC → SIRC32K → FIRC → PLL0 → PLL1.

Note:

- For FIRC or SIRC, the clock is internal clock and SIRC is always on.
Note that the FircISRNotification/SircISRNotification is editable when the (clock Monitor Enable) is enabled. This notification is called when the clock is error or the autotrim is enable and autotrim configuration is error.
- For FOSC or SOSC clock enable, need to check if external oscillators are placed on PCBA.

McuFOSCClockConfig

Name

McuFOSCClockConfig

FOSC Enable

☒

✎

McuSOSCClockConfig

Name

McuSOSCClockConfig

SOSC Enable

☐

✎

Note that the FoscISRNotification/SoscISRNotification is editable when the (clock Monitor Enable) is enabled and (clock monitor reset Enable) is disabled. This notification is called when the clock is error.

- For PLL clock source, make sure FOSC or FIRC has been configured before selection.

McuPLL0Config

Name

McuPLL0Config

PLL0 Enable

☒

✎

PLL0 Clock Monitor Enable

☐

✎

PLL0 Stop Enable

☐

✎

Mcu PLL0 Source Clock

FOSC

✎

PLL0 Pre Divider (1 -> 8)

FIRC

✎

PLL0 Lock

☒

✎

PLL0 Clock Monitor Reset Enable

☐

✎

Note that the PLL0ISRNotification is editable when the (clock Monitor Enable) is enabled and (clock monitor reset Enable) is disabled. This notification is called when the clock is error.

- For system clock, make sure PLL0, FOSC or FIRC clock has been configured before selection. Core clock frequency is the current MCU main clock.

Flagchip Confidential and Proprietary

170

▼ McuSystemClockConfig

Name  McuSystemClockConfig

System Clock Select	 PLL0 
Bus Clock Monitor	 ☐ 
System Core Clock Divider (1 -> 16)	 1 
System Bus Clock Divider (1 -> 16)	 2 
System Slow Clock Divider (1 -> 16)	 4 
SCG ClkOut Select	 FOSC 
NVM Clock Source Select	 SIRC
System Clock Source Frequency (Hz) (1000 -> 150000000)	 1.5E8 
Core Clock Frequency (Hz) (1000 -> 150000000)	 1.5E8 
Bus Clock Frequency (Hz) (1000 -> 75000000)	 7.5E7 
Slow Clock Frequency (Hz) (1000 -> 37500000)	 3.75E7 
SCG ClockOut Frequency (Hz) (0 -> 150000000)	 2.4E7 

Note that the CMU configurations in General sheet of McuClockSettingConfig is about the functionality of clock monitor.

McuClockSettingConfig

Name  MCU_Demo_FOSC24M

General FOSC SOSC SIRC SIRC32K FIRC PLL0 PLL1 System Clock PCC Clock Reference Point CSC0 SCM

McuClockSettingId  0 

▼ McuClockMonitor

Name  McuClockMonitor

▼ McuClockMonitor0

Name  McuClockMonitor0

CMU0 Enable	 ☐ 	Low Power Enable	 ☐
Stop Enable	 ☐	Enable CMU0 Interrupt	 ☐
Period Monitor Enable	 ☐		
Reference clock divider	 1		
Minimum REF_WINDOW value(1 ~ 15978297)	 1881		
Maximum REF_WINDOW value(0 ~ 15978297)	 1.5978297E7		
REF_WINDOW value	 188100		
Minimum Monitored Clock Count	 475		
Maximum Monitored Clock Count	 531		
Period Monitor Window (0 -> 15)	 1		
☒ CmuISRNotification	 NULL_PTR		

- b) Peripheral clock configuration is used for PCC function. It's the dependency of other peripherals. Please make sure to configure the clock gate and function clock if the peripheral has been used.

McuPeripheralClockConfig							
Index	Name	Mcu Peripheral Name	Enable p...	Peripher...	Peripher...	Peripher...	Peripher...
0	McuPeripheralClockConfig_1	DMA_MUX		OFF	1	0	0.0
1	McuPeripheralClockConfig_2	FTU0		FOSCDIV	1	0	2.4E7
2	McuPeripheralClockConfig_3	ADC1		FOSCDIV	1	0	1.2E7
3	McuPeripheralClockConfig_11	FCIIC1		FOSCDIV	1	0	1.2E7
4	McuPeripheralClockConfig_12	FTU1		SIRCDIV	1	0	1.2E7
5	McuPeripheralClockConfig_13	FTU2		PLL0DIV	1	0	1.2E8
6	McuPeripheralClockConfig_14	ADC0		FOSCDIV	1	0	1.2E7
7	McuPeripheralClockConfig_15	RTC		OFF	1	0	0.0
8	McuPeripheralClockConfig_19	PORTA		OFF	1	0	0.0
9	McuPeripheralClockConfig_38	CMP0		OFF	1	0	0.0
10	McuPeripheralClockConfig_39	CMP1		OFF	1	0	0.0
11	McuPeripheralClockConfig_40	CMP2		OFF	1	0	0.0
12	McuPeripheralClockConfig_41	FLEXCAN0		FOSCDIV	1	0	2.4E7

Note that if the CMU or WKU configuration is enabled, must configure the clock gate.

- c) MCU Mode Setting is used for the target MCU running mode to be used. The setting ID will be called by API function `Mcu_SetMode`, and it will call the SMC mode configuration accordingly.

for FC7300 series:

McuModeSettingConf*				
Index	Name	Mode ID	Operating Mode	
0	McuModeSettingConf_0	0	RUN	
1	McuModeSettingConf_1	1	CORELP_SLEEP	
2	McuModeSettingConf_2	2	CORELP_DEEPSLEEP	
3	McuModeSettingConf_3	3	STOP	
4	McuModeSettingConf_4	4	STANDBY_0	
5	McuModeSettingConf_5	5	STANDBY_1	
6	McuModeSettingConf_6	6	STANDBY_2	
7	McuModeSettingConf_7	7	STANDBY_3	

for FC7240:

McuModeSettingConf				
Index	Name	Mode ID	Operating Mode	
0	McuModeSettingConf_0	0	RUN	
1	McuModeSettingConf_1	1	WAIT	
2	McuModeSettingConf_2	2	STOP	
3	McuModeSettingConf_3	3	STANDBY_0	
4	McuModeSettingConf_4	4	STANDBY_1	
5	McuModeSettingConf_5	5	STANDBY_2	
6	McuModeSettingConf_6	6	STANDBY_3	

- d) Clock source reference point.

Some peripherals may have a function clock for external bus clock calculation. It would get the clock source frequency value from the MCU clock reference point like:

`SpiTransmitTimeoutClockRef`

`/Mcu/Mcu/McuModuleConfiguration/McuClockSettingConfig_0/McuClockReferencePoint_FCSP11`

Make sure the module referenced link item has been configured in reference point.

McuClockReferencePoint

Index	Name	Mcu Clock Freq...	Mcu Clo...
0	McuClockReferencePoint_Sys	SYS_CORE_CLK	1.2E8
1	McuClockReferencePoint_Bus	SYS_BUS_CLK	4.0E7
2	McuClockReferencePoint_Can0	FLEXCAN0	2.4E7
3	McuClockReferencePoint_Adc0	ADC0_CLK	1.2E7
4	McuClockReferencePoint_Ftu0	FTU0_CLK	2.4E7
5	McuClockReferencePoint_FCSPI1	FCSPI1_CLK	1.2E7
6	McuClockReferencePoint_FCIIC0	FCIIC0_CLK	1.2E7
7	McuClockReferencePoint_0	FCUART0_CLK	1.2E7

5) Callout and Notification functions:

- Callout: Mcu_PerformReset API has a callout function before resetting:

```
#if (MCU_RESET_CALLOUT_USED == STD_ON)
```

```
/* User callout */
```

```
MCU_RESET_CALLOUT();
```

```
#endif /* (MCU_RESET_CALLOUT_USED == STD_ON) */
```

If user want to use this callout before reset function performed, Fill the function name without parameters at this place

McuPerformResetCallout

- Notification:

CMU, RCM and PMC reserve the notification interface in interrupt handler. To use this function, the user needs to enable the optional item, and add the function name without parameters.

McuPerformResetCallout





McuRgmIsrCallout





PmcISRNotification





CmuISRNotification





Then add callout function as example:

```
void Rgm_Notification(uint32 u32SRS)
```

```
{
```

```
(void)u32SRS;
```

```
}
```

```
void Pmc_Notification(uint32 u32Flag)
```

```
{
```

```
(void)u32Flag;
```

```
}
```

```

void Cmu0Callout_Func(const CMU_InstanceType eInstance, const CMU_InterruptType eStatus)
{
    (void)eInstance;
    (void)eStatus;
}

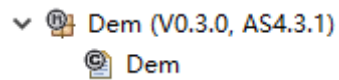
void ResetCallout_Func(void)
{
}

```

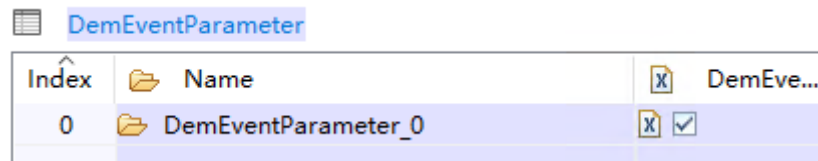
6) Dem parameter dependency:

To add Dem event report in MCU, take the following steps:

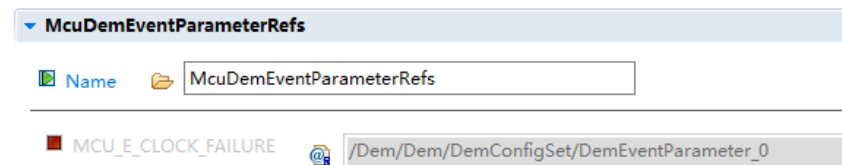
a) Load the Dem module to the project:



b) In DemEventParameter list, configure the required event ID.



c) In McuDemEventParameterRefs, enable MCU_E_CLOCK_FAILURE, and add the reference link from the DEM event parameter setting.



Note: If the tool chain is not EB. The user may manually change the code to add the DEM event ID and reference in MCU.